Investigating memory management with a custom interpreter

Sam Cutter

Contents

1. Analysis 4
1.1. Statement of the investigation 4
1.2. Background 4
1.3. The expert 4
1.4. Initial research 4
1.4.1. How interpreters work 4
1.4.2. Memory management techniques 7
1.4.3. Existing interpreted languages 8
1.4.4. First interview 11
1.4.5. Key components 14
1.5. Further research 15
1.5.1. Prototype 15
1.5.2. Second interview 20
1.6. Objectives 21
1.7. Modelling 25
1.7.1. Grammar 25
1.7.2. Flowchart 26
2. Design 28
2.1. High-level overview 28
2.1.1. Key components 28
2.2. Host programming language 30
2.3. Lexing 30
2.3.1. The lexing algorithm 30
2.3.2. The `Source` data structure 30
2.3.3. The `Token` data structure 31
2.4. Parsing 32
2.4.1. Recursive descent parsing algorithm 32
2.4.2. The `TokenStream` data structure 34
2.4.3. The `Expression` data structure 35
2.4.4. The `Statement` data structure 36
2.5. Execution/evaluation 36
2.5.1. The execution/evaluation algorithm 36
2.5.2. Data structures to represent values 38
2.5.3. The `Stack` data structure 38
2.5.4. The `Environment` data structure 39
2.6. Memory management 41
2.6.1. Associated data structures 41
2.6.2. Naive allocation 41
2.6.3. Garbage collection 41
2.6.4. Reference counting 42
2.7. Statistics tracking 44
2.7.1. Entry logging frequency 44
2.7.2. The `Entry` data structure 44
2.7.3. The `Logger` data structure 44
2.8. User interface 45
2.8.1. REPL mode 45
2.8.2. File mode 45

3. Technical solution 45
4. Testing 46
4.1. Testing table 46
5. Evaluation 49
5.1. Overall effectiveness 49
5.2. Evaluation of objectives 49
5.3. Memory management method analysis 50
5.4. Expert feedback 55
5.5. System improvements 58
5.5.1. Improving the type system 58
5.5.2. More advanced data structures 59
5.5.3. Improving iteration constructs 59
5.5.4. Closures 60
A Prototype source code 61
B Interpreter source code 87
C Tests 156
Bibliography 177

1. Analysis

1.1. Statement of the investigation

Most interpreted programming languages now offer automatic memory management, which moves the burden from the user to the runtime. There are various solutions for managing memory, including reference counting, garbage collection and ownership, however each have their tradeoffs with respect to runtime performance, implementation complexity and learning curve for the developer. This report aims to investigate different memory management techniques, in order to evaluate their relative performance within an interpreted environment.

My report will answer the following questions:

1. How does choice of memory management technique affect the execution time of a program?
2. How does choice of memory management technique affect the amount of memory used by the interpreter?
3. How does choice of memory management technique affect the speed at which objects are cleaned up?

1.2. Background

A translator is a program which translates a sequence of instructions–the *source code*–from one language to another. Typically, the translation is from a human readable, *higher level* language, into a language which can be understood by machines–a *low level* language. There are two main types of translator: compilers and interpreters.

Compilers translate the source code in one go, producing a file which contains the entirety of the translated code, which can then be executed independently.

An interpreter, however, reads the source code, translating then immediately executing one instruction at a time, without producing any translated output.

When an interpreter executes a sequence of instructions, it is responsible for managing the memory it uses. Since the computer doesn't have infinite memory, it's important to make efficient use of what's available, so the interpreter must make correct and useful judgements about which information is still in use (or could be used in the future) by the program.

1.3. The expert

I reached out to Dr. Philip M. Sleat, who taught me further maths and computer science at GCSE level, for assistance with this investigation. He has a PhD in Computer Science, and had a career as a software engineer before moving into teaching.

1.4. Initial research

1.4.1. How interpreters work

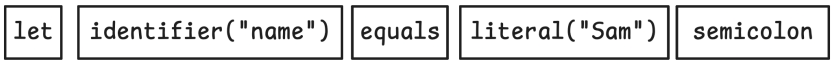
In order to gain an understanding of how interpreters work, I read the first four chapters of *Compilers: Principles, Techniques, and Tools, 2nd edition* [1]. This gave me an understanding of formal grammars, lexical analysis and parsing.

1.4.1.1. Lexical analysis

The lexical analyser works at the front line of the interpreter, reading the raw source code which the user has written, and converting this information into a much cleaner, easier to work with format: a sequence of tokens. A token represents the smallest meaningful unit of a language. For example, consider the following code:

```
let name = "Sam";
```

The above code assigns the string "Sam" to the identifier `name`. Running this through a lexical analyser would produce something like:



Here, each box represents a single token, and the text inside each box represents the type of token. In some cases, for example with `identifier`, additional information needs to be stored, in which case the data is shown in parenthesis.

We see that no information–barring whitespace–has been lost from the original source code. The information has simply been converted into a format which is easier for the next stage of the interpreter to work with.

In order to produce this sequence of tokens, the lexical analyser must understand the *lexical grammar* of the language. The lexical grammar describes exactly which sequences of characters map to which tokens.

Let's imagine a simple language which supports basic arithmetic and logical operations. If, while scanning through the source code, we encounter a + character, we immediately know that this must be a `Plus` token, as no other token begins with the + character. However, if instead we encountered an = character, we don't immediately know which token is being described: it could either be a single equals sign (the `Equal` token), which may represent assigning a value to a variable, or it could be two equals signs right next to one another (`DoubleEqual`), which would then be describing an equality test. In order to determine which token we have, we must look ahead into the source code. In this example, another equals sign would confirm that we are consuming a `DoubleEqual` token, and any other character (or no next character at all) would mean that we have an `Equal` token.

The distance into the source code at which we may need to peek is called the *lookahead*. Most languages only need one or two characters of lookahead, which is fortunate as lexical analysers tend to slow down as lookahead increases. For this reason, I will ensure that my language will only require two characters of lookahead (see Objective 2.3).

More complex tokens, for example numbers, are slightly more complex to read, but it ultimately comes down to consuming characters, and occasionally peeking a number of characters ahead to figure out what's going on.

1.4.1.2. Parsing

The parser takes in the sequence of tokens produced by the lexical analyser, and produces a data structure which represents the semantics of the user's program. This data structure should take into account rules to do with operator precedence and associativity.

Consider the following source code:

```
1 + 2 * 3 - 4
```

As per Section 1.4.1.1, lexical analysis analysis would produce the following:



If a human were to evaluate the code, they would likely group the separate parts of the expression, applying the rules of operator precedence they learnt in primary school. They would first evaluate the central brackets: `1 + 6 - 4`, before evaluating from left to right: `7 - 4`, then `3`.

This method of nesting and evaluating sub-expressions lends itself well to storing the program in a tree-like data structure, usually known as an *abstract syntax tree*. In our example, the abstract syntax tree produced would look something like this:

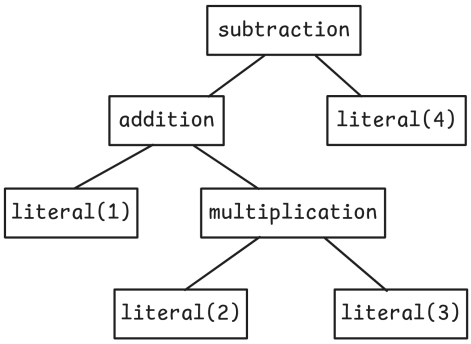


Figure 1: An expression tree.

In this diagram, the internal nodes represent operations, and the leaf nodes represent operands. Once again, the information stored by the tokens has not been changed, simply rearranged into a more useful format.

The process of parsing is slightly more complicated, but in many ways similar to lexical analysis. The general idea is to attempt to match sequences of tokens against pre-described patterns. The set of valid patterns which make up a language is known as its *grammar*.

A grammar contains a set of *non-terminals*, and for each non-terminal, a set of *productions*. The productions are the various 'patterns' which are associated with that non-terminal, and may contain a mixture of *terminals* (actual tokens) and other non-terminals. The parser builds up the abstract syntax tree by attempting to parse a designated 'start' non-terminal, and recursively replacing non-terminals with associated productions, depending on the contents of the source code. This type of parser is called a *recursive descent parser*. I plan on using this type of parser for my interpreter (see Objective 2.5), as it is relatively simple to implement but very powerful.

1.4.1.3. Evaluation

Once an abstract syntax tree (AST) has been produced, it can be directly evaluated. As each internal node represents an operation, in order to evaluate that internal node, its children must first be evaluated. This means that a post-order traversal is appropriate for evaluating the AST.

Once the children for an internal node have been evaluated, the operation which that node describes can be carried out. It's at this point that any type checking should occur, to ensure that operations which are poorly defined (e.g. adding a string to an integer) will not be carried out. I plan on performing type checks at runtime to prevent strange runtime errors appearing (see Objective 1.4). It's also important to check for other invalid operations such as division by zero at this stage. Once the types have been checked, the operation is performed on the values of the children, and the result returned to the parent operation.

Some operations, such as accessing variables, require access to the *environment*. The environment is a collection of *scopes*, arranged in a stack, where each scope contains variable and function definitions. A new scope is added whenever a block (e.g. an if-statement, while-loop etc.) is entered, and the scope is popped off the environment stack when that block is exited. In order to get the value of a variable, the most recent scope is first checked, then its parent, and then the program carries on iterating through the scopes until the original one is reached (or until the variable in question is found).

The environment stack is different to the call stack, in that each call stack frame has its own environment stack contained within it. However, in order to facilitate global variables, the global variable scope exists externally to the call stack.

I plan on using the post-order traversal method to evaluate abstract syntax trees produced by my parser, as it is simple to implement, and is easy to understand conceptually (see Objective 2.6).

1.4.2. Memory management techniques

Any data stored on the heap must be managed, in order to prevent the memory which a program uses from growing unchecked. These are some of the methods commonly used for managing memory.

1.4.2.1. Naive allocation

What is sometimes known as the naive method is not used within any serious programming languages, as it practically useless. Allocated memory is never deallocated, even when obvious that the information it stores will never be needed again. As a consequence, running programs will use huge amounts of memory, the vast majority of which is wasted. I will implement this as one of the memory management options, in order to provide a 'control' for the benchmarks (Objective 2.11.1).

1.4.2.2. Reference counting

Reference counting is a technique which keeps track of how many parts of the program rely on (reference) a piece of information, deallocating the memory once the number of references drops to zero. This technique is generally very effective, as objects' memory is deallocated as soon as the object is no longer needed. However, any situation in which an object references itself, directly or otherwise, cannot be handled by the reference counter.

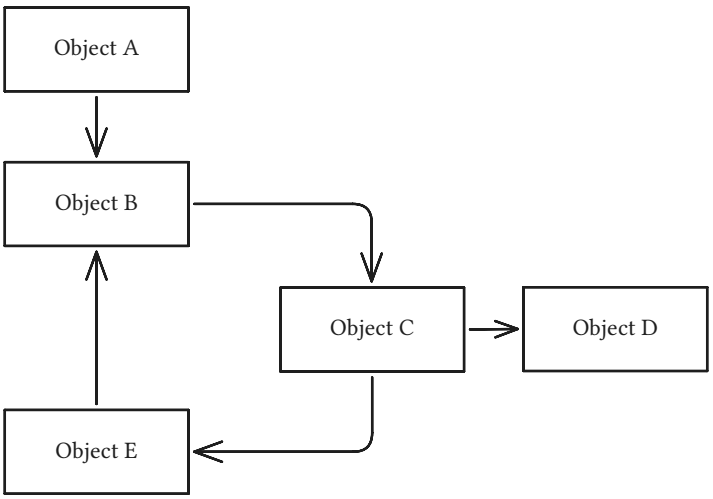


Figure 2: A reference cycle.

Consider Figure 2. Let's imagine that Object A was deleted. Object B's reference count would therefore be decreased by one, however due to the reference from Object E, its count would not fall to zero, meaning that Object B would not be deleted. Therefore, the reference from Object B to C would not be dropped, and neither would all subsequent references. Objects B, C and D and E would not be deleted, despite existing entirely isolated from the rest of the program, completely unreachable.

In my solution, I will implement this as one of my memory management techniques (see Objective 2.11.2), as it is relatively simple to implement, while being generally fast and effective. Furthermore,

my programming language will need to support basic objects which can reference one another (see Objective 1.7), in order to demonstrate the limitations of reference counting.

1.4.2.3. Mark-and-sweep garbage collection

Mark-and-sweep garbage collection maintains a graph of references between objects, and frequently traverses this graph, *marking* all objects which can be reached, and then *sweeping* through the graph, dropping all objects which are unreachable.

In the example given in Figure 2, as soon as the reference from Object A to B is dropped, then Object B will not be reachable, meaning that Objects B, C, D and E will be deleted.

This technique tends to be relatively robust, however frequent traversal of the reference graph may slow down the execution of the program, so much so that the interpreter can sometimes be forced forced to temporarily stop execution of the user's code so that it can run the garbage collector and clean up its memory (these moments are known as 'stop the world moments'). However, the time cost can be justified by the memory saved, especially in programs where there tend to be many reference cycles.

I will implement mark-and-sweep garbage collection in my solution, to demonstrate a more widely used memory management method (see Objective 2.11.3). I will benchmark programs written in my language to see how much the running of the garbage collector itself impacts the performance of the interpreter, as compared to other memory management methods (see Objective 3).

1.4.3. Existing interpreted languages

1.4.3.1. Python

Python is often the language which people learn first, due to the simplicity of its syntax, and the huge community behind it. It's not just used by beginners, though. Python is one of the most used languages for machine learning and data analysis.

Python is a multi-paradigm language, supporting object-oriented programming, imperative programming, as well as aspects of functional programming. It's dynamically (but strongly) typed, with the types of operands being checked at runtime to ensure undefined behaviour doesn't occur.

```
def fibonacci(n):
  if n <= 1:
    return 1

  return fibonacci(n - 2) + fibonacci(n - 1)
```

Shown above is a recursive implementation of getting the nth Fibonacci number in Python. As seen, variables and function definitions are not annotated with types. Unlike languages in the C family, Python does not use any statement terminators such as a semicolon, and it also does not enclose blocks with curly braces.

```
def bubble_sort(items):
  for i in range(0, len(items)):
    swaps_made = False

    for j in range (0, len(items) - i - 1):
      if items[j] > items[j + 1]:
        (items[j], items[j + 1]) = (items[j + 1], items[j])
        swaps_made = True

    if not swaps_made:
      break

  return items
```

Above is another example of some Python code, this time a bubble sort. Control flow is shown through indentation, which makes for fairly elegant code, however this syntax is disliked by many.

Under the hood, Python is first compiled to bytecode, which is then ran in a virtual machine. Python uses a combination of reference counting and garbage collection to manage its memory, primarily employing reference counting, but with some cycle detection features to catch what a typical reference counter cannot.

Advantages

- Syntax is very beginner friendly and readable.
- Dynamic type system provides a lot of flexibility, and leads to more concise code due to lack of type annotations.
- Automatic memory management removes a burden from the programmer.

Disadvantages

- Relatively slow execution times.
- Dynamic typing can lead to unpredictable runtime errors.

As my language will also be dynamically typed and interpreted, it will experience the same issues with runtime errors and speed that Python does. Performance issues introduced by the garbage collector may also occur within my language, but I will allow the programmer to choose their desired method of memory management (see Objective 4.1), so the power will lie with them to decide which memory management method would be the most appropriate. I'd like to mimic the level of flexibility and ease of use that Python provides, by not forcing the user to annotate their variables and functions with types (see Objective 1.3.1), and allowing the types of variables to change throughout the execution of the program (see Objective 1.3.2). I also like the fact that although Python is dynamically typed, it is strongly typed, meaning that it ensures poorly defined operations (such as adding a Boolean and an integer) will not be performed, and an error will instead be thrown at runtime. I will implement these runtime type checks within my interpreter (see Objective 1.4).

1.4.3.2. JavaScript

JavaScript is the language which powers the vast majority of websites on the Internet. Virtually all browsers have a JavaScript engine, whose job is to interpret scripts embedded into the HTML of webpages. This allows developers to create complex web applications, which can fetch data from databases, change the style of the website dynamically, and run complex algorithms.

JavaScript is dynamically typed and, just like Python, supports multiple programming paradigms.

```
function fibonacci(n) {
    if (n <= 1) {
        return 1;
    }

    return fibonacci(n - 2) + fibonacci(n - 1);
}
```

Here, the Fibonacci function is implemented in JavaScript. Notice the C-like syntax, with curly braces and semicolons, and parenthesis surrounding `if` statement conditions.

```
function bubble_sort(items) {
  for (let i = 0; i < items.length; i++) {
    let swaps_made = false;

    for (let j = 0; j < items.length - i - 1; j++) {
      if (items[j] > items[j + 1]) {
        let temp = items[j];
        items[j] = items[j + 1];
        items[j + 1] = temp;

        swaps_made = true;
      }
    }

    if (!swaps_made) {
      break;
    }
  }

  return items;
}
```

The bubble sort algorithm is implemented in JavaScript above. Here, it becomes clear that JavaScript code can be more verbose than Python code, but control flow is often slightly more clear, due to the use of curly braces surrounding blocks of code.

JavaScript engines typically use a combination of interpreting and compiling, known as just-in-time compilation. Initially, the JavaScript program is interpreted, but as it runs, the engine analyses the code, and will compile sections of the code–such as frequently called functions–to achieve speedups. Since the engine has access to more information than a standalone compiler would, programs can often reach faster execution times using this method.

JavaScript uses garbage collection to clean up memory no longer in use, specifically mark-and-sweep garbage collection, which 'marks' all reachable objects and then removes all objects which are unmarked.

Advantages

- The use of just-in-time compilation achieves much faster speeds than purely interpreting the source code.
- Similarly to Python, the lack of type annotations leads to more concise code.
- Robust garbage collection allows the programmer to focus on the logic of their program rather than managing memory.

Disadvantages

- For some types, operations are defined in a confusing manner, for example between a string and an integer, the + operator treats the number as a string and performs string concatenation, whereas - attempts to coerce the string into an integer, then perform the arithmetic subtraction.
- Dynamic typing can lead to difficult-to-trace runtime errors, especially when the `undefined` value is introduced.

Just-in-time compilation would be too complex to implement for this project, so I will stick to purely walking the abstract syntax tree produced by the parser (see Objective 2.6). The memory management that JavaScript provides is generally very effective, so I will provide garbage collection as one of the memory management options (see Objective 2.11.3). I dislike the fact that some operations are poorly defined, and are performed anyway without reporting an error, so I will ensure that my language is strongly typed (see Objective 1.4). I prefer the syntax of JavaScript to Python, as it's often more clear how the code works, so I will base the syntax of my language around that of the C family's (see Objective 1.2).

1.4.4. First interview

Question 1: "Which memory management techniques would be both feasible and valuable to implement and investigate?"

> "With garbage collection, you can have *stop the world moments*, where the interpreter is no longer running the user's code, but it's running all this background stuff to try and clean up its memory. However, you don't get the hassle of memory leaks. You could also think about reference counting, which doesn't deal with reference cycles or self-referential objects, so you'd have to watch out for that. I would stick with mark-and-sweep, reference counting and perhaps seeing what happens when memory is left unmanaged. That sounds quite ambitious to me, but there we go. As a potential extension to the project, you could implement manual memory management. Manual memory management gives you that element of direct control, but also the hassle of problems with memory leaks and things like that. The advantage is that you can often get much faster, more predictable execution times."

Dr. Sleat suggested that I should implement mark and sweep garbage collection (see Objective 2.11.3), reference counting (see Objective 2.11.2), and also naive allocation, where no memory management occurs (see Objective 2.11.1). I will most likely not implement manual memory management, as I think it will be beyond the scope of this project.

He also mentioned some of the advantages and pitfalls of the various techniques. Garbage collectors can cause *stop the world moments*, where execution of the user's program has to temporarily be stopped while the garbage collector cleans up memory, to avoid running out of memory. This could cause issues in time-critical systems. With manual memory allocation, it is very easy to accidentally create bugs which leak memory, since the control of memory is entirely down to the user. However, this is a double edged sword, as code with manual memory management may have more predictable execution times (there is no garbage collector to cause stop the world moments). Dr. Sleat also mentioned the issues with reference counting, namely the fact that it can't deal with reference cycles.

Question 2: "What is the minimum feature set that the language should have in order to be able to evaluate each technique fully?"

> "You're going to need to have some element of global variables, whose state persists for the entire duration of the execution of the code and which you don't want to be cleared up in any garbage collection or memory management. I would suggest that you probably need some functions, in order to create some local variables within those functions, and maybe handle some arguments that are going to be stored on a stack or something like that. And if you've got some functions, then you can probably implement some very simple recursion for something like Fibonacci or factorial, nothing too complicated. And then obviously have some sort of basic object which is able to reference objects in order to test reference counting and reference cycling."

Dr. Sleat told me that I would need a range of features. The first is global variables (see Objective 1.8.4.2), which should not be cleaned up in memory management. Next is functions (see Objective 1.10), which should have local variables and arguments. Functions should support recursion (see Objective 1.10.5). There should also be some objects (see Objective 1.7), which are able to reference other objects (and themselves), so that I can test reference cycle handling.

Question 3: "Which paradigm (object oriented, imperative etc.) would you recommend for my language, and why?"

> "I would try to keep it as simple as possible. You've got a lot to tackle and probably an imperative language is going to be your best option, with very basic objects."

From the answer to this question, it's clear that I should stick to an imperative paradigm (see Objective 1.1). However, I will still need implement the ability to have objects. They will not be advanced objects, however.

Question 4: "Would making my language statically typed be feasible? If so, would it be relevant to this project?"

> "With dynamic typing, you can use a piece of memory however the code wants to use that memory at that moment in time. So when you set up a variable, it's really just pointing to a lump of memory that's going to be used. In terms of the runtime system, that is probably going to be the easiest because you're just allocating a bit of memory and then using it as the bit of imperative code sees fit. From a functionality, ease of use, purism of the language point of view, it is horrible. I mean in Python you can set a variable to an integer, and in the very next line you can set it to a string. But it's probably the simplest. If you have static casting of your data types, then you're going to have to put a bit more effort into making that work. So I would keep it as simple as possible if I were you."

Dr. Sleat suggested that, once again, I should keep things simple. For that reason, I'll keep my language dynamically typed (see Objective 1.3), but perform runtime checks to check that operations are type safe (see Objective 1.4).

Question 5: "As someone who has programmed in both C and Python, what would you say are the relative advantages and disadvantages of both, with respect to performance, ease of use and available features?"

> "C is obviously fast, and very low level, very prone to error, lots of memory leaks all over the place if you're not careful. Python is absolutely great. If you've got a list in Python and you want to add something to the end of it, it's as easy as that. But as you might remember me saying in computer science lessons, it makes for sloppy coding and makes for not very intelligent programmers because they don't know how to do the low level stuff, which is what I think they should know. It depends what the application is, of course, because if you're coding something where you want speed, then C might be the way to go. If you're just knocking up some quick applications and you're not very experienced then Python may be the way to go."

From the answer to this question, it's clear that Python sacrifices low level control and speed for ease of use and flexibility. Since my language is only going to be used for basic testing, it should be fine for it to be closer, in terms of ease of use, to Python. The syntax of my language will be closer to that of C (see Objective 1.2), as Python's indentation style can sometimes be ambiguous.

Question 6: "Once completed, what kinds of algorithms should I implement and benchmark in my language to measure the performance of each technique?"

> "If your programming language can cope with it, doing some recursive calls of a very simple function like a factorial. To see the difference between garbage collection and reference counting, it will also be necessary to implement some programs which introduce reference cycles."

In my programming language, I'll implement a recursive algorithm, such as the Fibonacci function (see Objective 3.1.1). I'll also try to introduce some reference cycles between objects (see Objective 3.1.2) so that I can differentiate between reference counting and garbage collection.

Question 7: "How should I measure the performance of the memory management techniques– which statistics should be tracked?"

> "Obviously, runtime memory usage, and make sure to track the number of objects at any one time. That will help to show the differences between reference counting and garbage collection. You should also track the time that programs take to execute, that kind of thing."

As part of my investigation, I'll measure the memory usage of the interpreter (see Objective 3.2.4), the time that various programs take to execute (see Objective 3.2.1). I may also measure the number of objects at any one moment (see Objective 3.2.2).

Question 8: "What are some common pitfalls which programming languages fall into?"

> "Memory leaks introduced by the runtime system. It does happen. And again it's it's the impact of that garbage collection. If that's too great, then the code could become too slow. How much does it expose to the programmer? How much do you want to expose to the programmer and how much damage can they cause by doing the wrong thing? That clarity of how the memory is handled, I think more than anything."

I learnt here that the runtime system can actually introduce more problems, such as memory leaks, as well as memory management simply having too much of a performance impact on the program. Dr. Sleat also mentioned some safety issues: making sure that the language doesn't expose too much power to the programmer (see Objective 2.11.4) (i.e. allowing the programmer to call the garbage collector directly.) He also says that the most important thing to him is for the language to be clear about exactly how memory is handled.

Question 9: "If you were to design the ideal programming language, which features would you want to include?"

> "If you look at all of these different programming languages, each of them have exciting bits, and bits that you wouldn't have. I actually used to quite enjoy doing memory allocation myself. It only because of that thrill of the chase to try and find out where memory leaks are. Debugging code is actually quite good fun, and it's a skill that's taken away from you with things like Python, probably rightly so, but that's just who I am. But no, I don't think there's anything particular I would want to see in it."

I didn't learn much from the answer of this question, apart from the fact that Dr. Sleat enjoys hunting down memory leaks by hand.

1.4.5. Key components

After completing my initial research, these are the key components for the project:

- **Interpreter.** The main goal for my project is to build an interpreter which executes my custom programming language. It will read in the source code, lexically analyse it, parse it, then execute it.
- **Memory management.** Another key component is memory management for data stored on the heap. The interpreter will support naive allocation, reference counting, and mark-and-sweep garbage collection.
- **Language features and syntax.** The language will be imperative, and the syntax will be similar to that of C's. It will be dynamically typed, but also strongly typed to prevent poorly defined operations.
- **Benchmarking.** Various statistics about the performance of the memory management techniques will be measured, including execution time, memory usage, and the number of objects and stack frames at any one moment.
- **Command-line interface.** The interpreter will provide a command-line interface that supports two modes: REPL for interactively executing one line of code at a time, and a file mode for executing full programs.

1.5. Further research

1.5.1. Prototype

For the prototype, I decided to implement a simple language, which is only able to handle arithmetic and logical expressions. I chose this prototype because it allows me to understand the implementation of almost all parts of the interpreter (lexer, parser, expression evaluator), without having to go too in-depth into any one part.

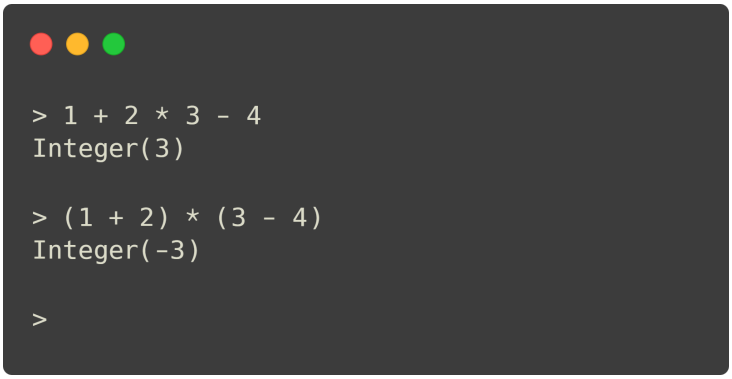


Figure 3: Some arithmetic expressions with support for correct operator precedence.

The prototype is effectively just a calculator: it's unable to handle variable assignments, control flow (other than ternary expressions) or any data structures such as strings. The programmer interacts with the interpreter with a command-line application, using a REPL-like interface, where they input an expression, and the interpreter evaluates and outputs the value of that expression immediately. Any errors which occur throughout the process of lexing, parsing or evaluation are properly caught and reported to the programmer, without crashing the application. While the language is dynamically typed, it is also strongly typed, meaning that poorly defined operations (such as adding an integer to a Boolean), will not be performed.

To assist with creating the prototype, I read chapters 1-6 of *Crafting Interpreters* [2]. This book walks through the process of creating an interpreter from the ground up, without using any 'compiler-compilers' or tools which automatically produce a lexer/parser from a language description. The book first implements a tree-walk interpreter using Java, and then implements a bytecode interpreter in C. I used the section which implements the tree-walk interpreter to inform the design of my prototype. This primarily helped me to understand how the lexing and parsing processes which I learned about in *Compilers: Principles, Techniques and Tools* [1] are translated into code. While my algorithms are very similar to the ones described in *Crafting Interpreters*, the data structures which I use to represent tokens and expressions are different to those in the book: I made sure to take full advantage of Rust's type system, so that invalid states are not possible to represent. This ensures that weird runtime errors don't crop up.

The code for the prototype is given in Appendix A, and is also available in the repository for this project [3].

1.5.1.1. The lexer

The lexer (Appendix A.4) reads in the raw source code from the programmer, and converts it into a stream of `Tokens`.

The lexer relies on a wrapper around the source code, called `Source` (Appendix A.7). This helps to simplify the lexer code, by exposing useful helper methods, such as `matches`, which only advances the stream if the next character matches a specific target character.

As the lexer reads through the source code, it bundles together information into units called `Tokens` (Appendix A.8). A `Token` is comprised of some data, and a location. The data field is where the bulk of the useful information is stored: information about the type of token (e.g. a left parenthesis token), as well as any data the token needs to store (e.g. a number if the token is an integer token) is kept here. The location is simply for debugging purposes later on.

The way in which the lexer works is described in more detail in Section 1.4.1.1. It comes down to scanning through the source code, character by character, and deciding which type of token is currently being described, ideally only having to use one or two characters of lookahead to make the judgement.

Occasionally, the programmer may input invalid characters. In this case, the lexer reports a useful error. For example, if the programmer attempts to use a single equals sign–which is not valid (assignment is not implemented in the prototype, whereas equality checks using the double equals are)–then the programmer will be informed that only a single equals character was found, whereas another equals character was expected, as shown in Figure 4. Other errors, such as characters which are not supported, or keywords which have not been defined (the prototype doesn't support identifiers) will be reported in a similar way.

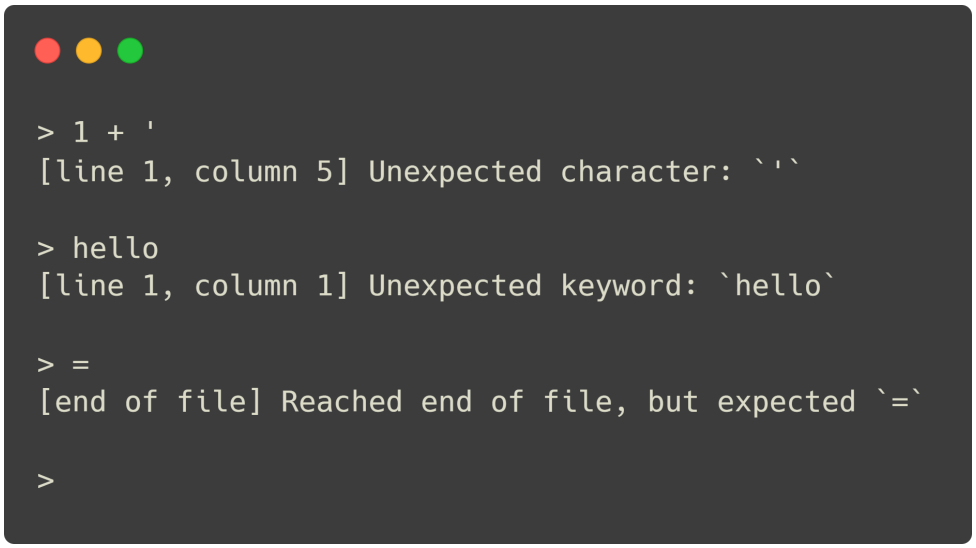


Figure 4: Error handling at the lexer level.

For the (lexically valid) expression `1 + 2 * 3 - 4`, the following tokens are produced:

```
Token { data: Integer(1), location: Location { index: 0, line: 1, column: 1 } }
Token { data: Plus,       location: Location { index: 2, line: 1, column: 3 } }
Token { data: Integer(2), location: Location { index: 4, line: 1, column: 5 } }
Token { data: Star,       location: Location { index: 6, line: 1, column: 7 } }
Token { data: Integer(3), location: Location { index: 8, line: 1, column: 9 } }
Token { data: Minus,      location: Location { index: 10, line: 1, column: 11 } }
Token { data: Integer(4), location: Location { index: 12, line: 1, column: 13 } }
```

We see that for integers, the value of that integer is stored in the `data` field of the `Token`, and that no information from the source code has been lost in the transformation to this new format.

What I learned

It was really useful learning how exactly a simple lexer should be implemented in code. There were two important lessons which I learnt from making the prototype lexer. The first one is the importance of having an abstraction over the source code (Objective 2.2). I found that interacting directly with the source code string could be quite annoying, and could introduce some weird bugs if

the state of the source code was not all synced up in one place. The helper methods which I built to assist with lexing proved really useful, and made for much more readable code. The second thing I learned was how to best represent tokens and their data. After playing around with multiple options, I settled on using an `enum` for the token data, with a variant for each type of token. This results in a data structure which can be easily worked with by the parser (Objective 2.7).

1.5.1.2. The parser

The parser (Appendix A.6) takes the token stream produced by the lexer, and outputs an abstract syntax tree.

Similarly to the lexer, the parser uses a wrapper over its input, named `TokenStream` (Appendix A.9). It's packaged with similar helper methods to simplify the code, and make consuming specific tokens easier.

The parser aims to produce an `Expression` tree. An expression can take multiple forms, for example a binary expression, a unary expression or a literal expression. An expression may be composed of sub-expressions, with the 'base' expression being just a literal value (an integer, float or Boolean). The parser produces the expression tree by recursively applying the rules of the grammar, from the top down, making it a recursive descent parser. The grammar which I developed for the parser is as follows:

```
expression -> ternary

ternary -> logical ("?" logical ":" logical)?

logical -> equality (("&&" | "||") equality)*

equality -> comparison (("!=" | "==") comparison)*

comparison -> bitwise (">" | ">=" | "<" | "<=") bitwise)*

bitwise -> term (("&" | "|") term)*

term -> factor (("+" | "-") factor)*

factor -> unary (("*" | "/") unary)*

unary -> ("!" | "-")? primary

primary -> "(" expression ")"
    | INTEGER
    | FLOAT
    | "true" | "false"
```

The grammar fully encapsulates the associativity and relative precedence of all of the operators, with ternary operators having the weakest binding, and parenthesis having the strongest binding. This grammar also eliminates *left-recursion*, which would have made writing the parser more difficult if not eliminated.

When a set of syntactically invalid tokens are given to the parser, a detailed error is reported to the programmer. These error handling methods handle situations such as undefined unary operators, missing operands to expressions, or unclosed parenthesis.

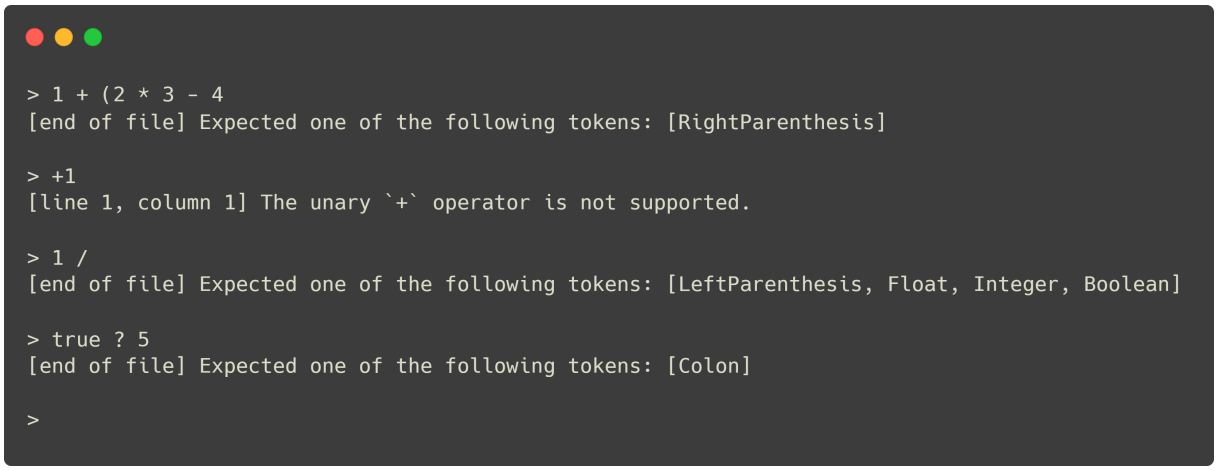


Figure 5: Error reporting at the parser level.

Continuing with the example given in the previous section, the expression tree for `1 + 2 * 3 - 4` is as follows:

```
Binary {
    left: Binary {
        left: Literal(Integer(1)),
        operator: Add,
        right: Binary {
            left: Literal(Integer(2)),
            operator: Multiply,
            right: Literal(Integer(3)),
        },
    },
    operator: Subtract,
    right: Literal(Integer(4)),
}
```

We see that the expression tree structure shown in Figure 1 is mirrored in the output of the parser, with correct precedence preserved.

What I learned

Of all the algorithms which make up the interpreter, the parser was the one which I initially assumed would be the most difficult to implement, so it was reassuring to see that recursive descent is not actually particularly complex. The main lesson which I will take forward into the implementation of the final interpreter is how to handle errors. While I only implemented fairly simple error handling in the prototype, *Crafting Interpreters* mentioned some more comprehensive error handling methods: all errors should reported, not just the first error which is found. Furthermore, any errors which are simply the result of a previous error (cascaded errors) should not be reported. Also, the importance of maintaining an abstracted layer over the list of tokens became obvious (Objective 2.4). The helper methods I developed were crucial for implementing the parser, as they made the code much more readable and easier to write.

1.5.1.3. The evaluator

The evaluator (Appendix A.3) traverses the abstract syntax tree produced by the parser, and evaluates it.

The data structure on which the evaluator operates is an expression tree. As described in the previous section, expressions are often composed of sub-expressions. This data structure lends itself to being recursively evaluated.

First, the top–or root–node is visited. Every node visit follows the following process: if the node has any children, then those children are first visited and evaluated. Once the children have been evaluated, the operation which the node represents will be performed on the values of its children. This value is then returned. For literal values (e.g. integers, Booleans etc.), the inner value is simply returned.

Since I decided to make my prototype strongly typed, operations are at this point type checked to ensure that poorly defined operations are not carried out. This means that for every operation, code must be written to handle every possible valid set of operand types. This leads to very verbose code, but since the host language (Rust) is statically typed there doesn't seem to be a simpler way. If the types of the operands to an operation are invalid (e.g. subtracting a float from an integer) then a type error is returned and propagated up the evaluation stack. Ultimately, this will result in no value being returned to the programmer. Another type of error which can occur at this stage is a division by zero: an appropriate error is reported in this case.

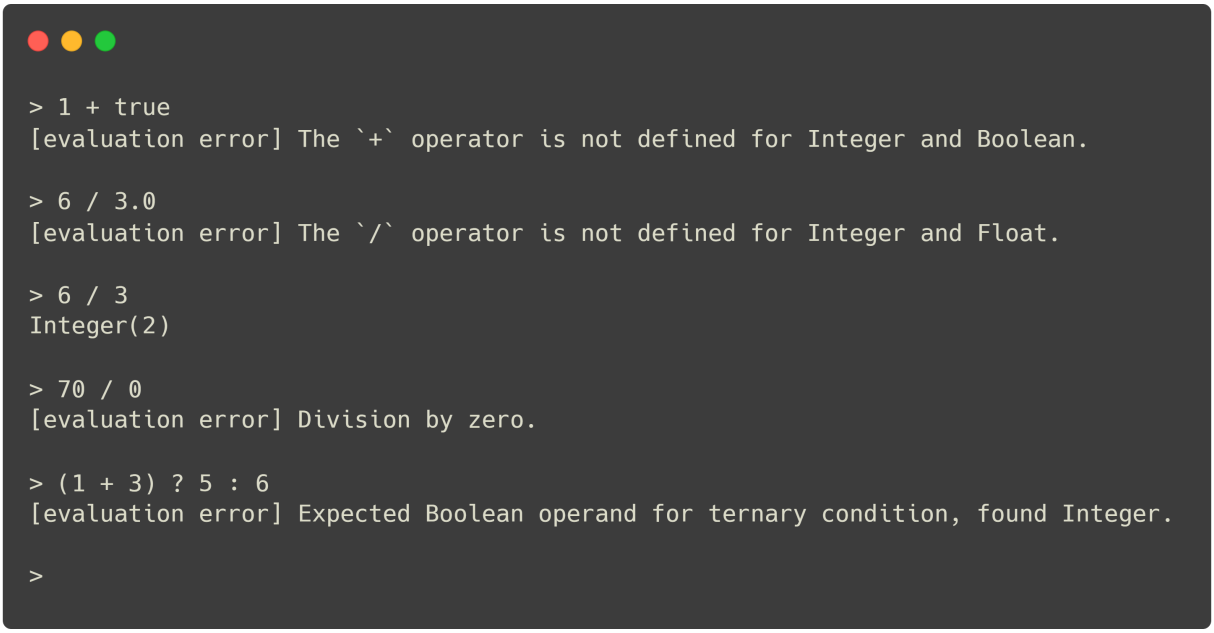


Figure 6: Error reporting at the evaluator level.

If no errors occur, then the result of evaluating the expression tree is output to the programmer along with the data type of the value.

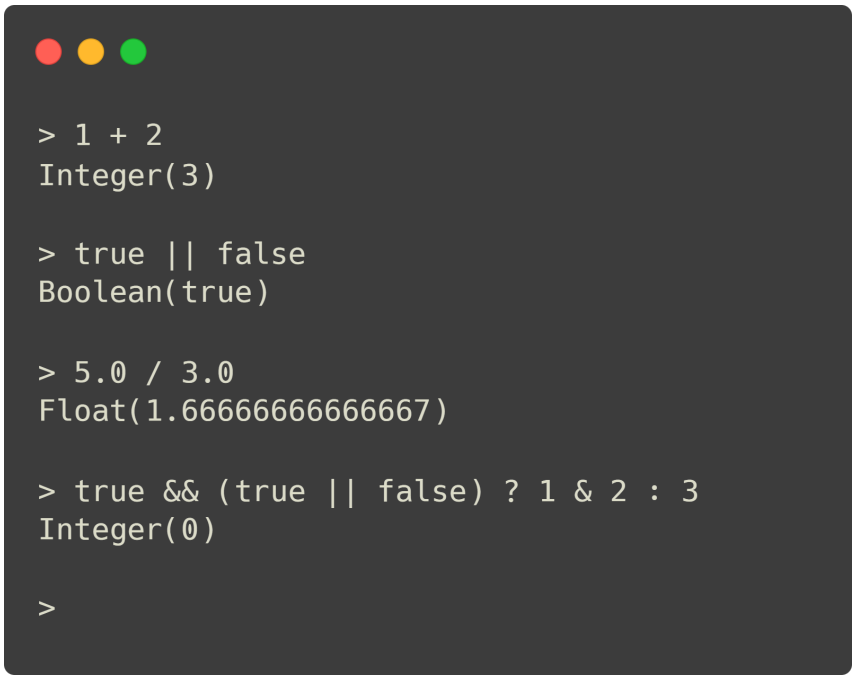


Figure 7: The output from the evalutor.

What I learned

The first thing I learned is that adding more operations and data types makes for significantly more work in the evaluation stage. I will keep my operations (Objective 1.5) and data types (Objective 1.6) to the bare minimum when creating my final program.

1.5.2. Second interview

At this point, I sent Dr. Sleat my prototype as well as my draft objectives to gather feedback. I asked the following questions:

- What features do you like about my prototype?
- Are errors clear and easy to understand?
- Within the current set of features, what do you dislike, and what could be improved (this could be to do with error handling, how type checking is performed etc.)?

He responded with the following:

> "Sam
>
> This is beginning to look very interesting indeed. I've had a play around with your prototype and can see that it reports correct answers for similarly typed pairs of numbers. I've also had a go at testing some of the Boolean expressions and this looks good too. What intrigued me is the decision to only allow operators between identically typed variables. This comes out in Objective 1.5.1. I'm guessing that from a memory management perspective it makes it a little simpler not to allow 4 + 9.8 for example?
>
> Your objectives look reasonable. My concern would be that there is a lot here to get done. While I have no doubt in your abilities, do you have a plan for which objectives take priority if you get short on time? Have you planned out subsequent prototypes and defined which objectives will be met in each? This might help you to still achieve your overall objectives with a partially implemented language.

> By all means keep using me to test/critique… I'm guessing that when you have a bigger prototype, I might be able to give more concrete feedback.
>
> Kind regards
>
> Phil"

Dr. Sleat pointed to my decision to enforce very strict type checking for operations. I agree that this can be quite cumbersome, syntax-wise, however I would prefer that my language be very explicit about the types which will arise from operations, even if this comes at the cost of a slightly more verbose syntax. Despite this, I will need ways to cast between values of different types, and some string formatting methods (Objective 1.11)

He also raised the concern about how much there is to get done. I hear these concerns, and I have a plan in place for which objectives will be completed, at what points in time.

I also made my prototype available to other students in my computer science class. It was suggested that I should add support for more operators, such as the right-associative exponent operator. This will be quite interesting to implement, programming language design–wise, so I will implement this within the final interpreter (see Objective 1.5.1.5).

1.6. Objectives

In my project, I will create an interpreter for a custom programming language, and then test various memory management options to see which is the most efficient.

1. There will be a language which the interpreter implements.
    1.1. The language will be imperative.
    1.2. The language will have syntax similar to that of C.
        1.2.1. Blocks, including subroutine bodies, the bodies of `if`-statements and other control flow structures, will be enclosed in braces.
        1.2.2. Statements will be terminated by a semicolon.
        1.2.3. Parameter lists to subroutines will be enclosed in parenthesis.
        1.2.4. Arithmetic and logical operations will be denoted in infix notation.
        1.2.5. Line comments will begin with a double slash.
        1.2.6. Block comments enclosed with `/*` and `*/` will be supported.
    1.3. The language will be dynamically typed.
        1.3.1. The types of variables and the return types of subroutines do not need to be specified by the programmer.
        1.3.2. Variables will be able to change type at runtime.
        1.3.3. Subroutines may return different data types (or no value at all) across different calls to the function.
    1.4. The language will be strongly typed: operations will be type checked to ensure that operations are not performed on incompatible data types.
    1.5. There will be operators.
        1.5.1. There will be arithmetic infix operators, which may only be performed on a pair of integers, or a pair of floats.
            1.5.1.1. The `+` operator adds the right operand to the left operand. This may also be performed on a pair of strings.
            1.5.1.2. The `-` operator subtracts the right operand from the left operand.
            1.5.1.3. The `*` operator multiplies the left operand by the right operand.
            1.5.1.4. The `/` operator divides the left operand by the right operand.
            1.5.1.5. The `^` raises the left operand to the power of the right operand.

1.5.2. The prefix - operator arithmetically negates its operand, and may only be performed on an integer or a float.
1.5.3. There will be ordering operators, which may only be performed on a pair of integers, a pair of floats, a pair of strings.
1.5.3.1. The > operator checks whether the left operand is greater than the right operand.
1.5.3.2. The >= operator checks whether the left operand is greater than or equal to the right operand.
1.5.3.3. The < operator checks whether the left operand is less than the right operand.
1.5.3.4. The <= operator checks whether the left operand is less than or equal to the right operand.
1.5.4. There will be equality operators, which may be performed on any pair of values of the same type.
1.5.4.1. The == operator checks whether the left operand is equal to the right operand.
1.5.4.2. The != operator checks whether the left operand is not equal to the right operand.
1.5.5. There will be logical operators, which may only be performed on two Boolean values.
1.5.5.1. The && operator checks whether both the left operand and the right operand evaluate to the Boolean true value.
1.5.5.2. The || operator checks whether at least one of the left operand or the right operand evaluates to the Boolean true value.
1.5.6. There will be bitwise operators, which may only be performed on two Boolean values, or two integers.
1.5.6.1. The & operator performs a bitwise AND with the left operand and the right operand.
1.5.6.2. The | operator performs a bitwise OR with the left operand and the right operand.
1.5.7. There will be the prefix ! operator, which is the logical negation operator for a Boolean operand, and the bitwise negation operator for an integer operand.
1.5.8. There will be the ternary `<condition> ? <if_true> : <if_false>` operator, which evaluates to `<if_true>` if `<condition>` is true, and `<if_false>` otherwise.
1.5.9. The language will respect operator precedence as described in Section 1.7.1.
1.6. There will be built-in data types.
1.6.1. There will be the integer type, which stores a 32-bit signed integer.
1.6.2. There will be the float type, which stores a 64-bit signed floating point number.
1.6.3. There will be the Boolean type, which stores a Boolean value.
1.6.4. There will be an string type.
1.7. There will be user-defined objects, which are composed of data items.
1.7.1. User-defined objects will be stored on the heap.
1.7.2. User-defined objects will store a number of data items, each of which has an identifier.
1.7.3. Data items will be accessible via the combination of an object name and item identifier.
1.7.4. User-defined objects will have no templates, or classes.
1.7.5. User-defined objects will have no methods defined on them.
1.7.6. User-defined objects will have no access restrictions on their data items.
1.8. There will be user-definable variables.
1.8.1. Variables will be stored on the stack.

1.8.2. Variables will have an identifier.
1.8.3. Variables will either store a primitive value, or an object reference.
1.8.4. Variables will only exist within the scope they have been defined in.
1.8.4.1. There will be local variables, which are defined within subroutines.
1.8.4.2. There will be global variables, which are defined at the outer layer of the program.
1.8.5. Variables will be mutable.
1.9. There will be control flow constructs.
1.9.1. There will be `if` statements.
1.9.1.1. An `if` statement evaluates a condition, and executes the following block if the condition evaluates to the Boolean true value.
1.9.2. There will be `else` statements.
1.9.2.1. An `else` statement must appear after an `if` statement.
1.9.2.2. The following block will be executed if previous `if` statement conditions have not passed.
1.9.3. There will be `while` loops.
1.9.3.1. A `while` loop evaluates a condition, then executes the following block if the condition evaluates to the Boolean true value.
1.9.3.2. This will happen repeatedly, until the condition evaluates to the Boolean false value.
1.10. There will be user-definable subroutines.
1.10.1. Subroutines will have an identifier.
1.10.2. Subroutines will have a list of parameters.
1.10.2.1. Parameters will not have set data types.
1.10.2.2. Subroutines must be called with the number of arguments equal to the number of parameters.
1.10.3. Subroutines will have a body, which is the list of statements that should be executed when the function is called.
1.10.4. Subroutines will be able to be called.
1.10.5. Subroutines will be able to call subroutines, including themselves (recursion).
1.10.6. Subroutines may return a value, with no set data type.
1.11. There will be native subroutines, which are provided by the language.
1.11.1. There will be a `print` function, which prints to standard output.
1.11.2. There will be a `format` function, which allows injecting values of various types into strings.
1.11.3. There will be an `input` function, which receives input from standard input.
1.11.4. There will be an `int` function, which attempts to cast the input to an integer.
1.11.5. There will be a `float` function, which attempts to cast the input to a float.
2. There will be an interpreter, which implements the language.
2.1. The interpreter will execute the program from source.
2.2. The lexer will not interact with the source code directly, instead wrapping it in an abstraction.
2.3. The lexer will only require two characters of lookahead.
2.4. The parser will not interact with the token list directly, instead wrapping it in an abstraction.
2.5. The parser will use recursive descent parsing.
2.6. Expressions will be evaluated using a post-order traversal of the AST.
2.7. Tokens will comprised of token data (which will be stored using an `enum`), and the token's location.

2.8. Any errors which occur throughout the interpretation process will be reported to the programmer.
    2.8.1. If the error occurred while lexing or parsing, the line and column at which the error occurred will be displayed to the programmer.
    2.8.2. The cause of the error will be displayed to the programmer.
2.9. There will be a stack, which stores a collection of stack frames.
    2.9.1. Each time a subroutine is called, a new stack frame will be pushed onto the stack.
    2.9.2. Each time a subroutine returns, its stack frame will be popped off of the stack.
    2.9.3. Each stack frame will store a collection of local variables.
    2.9.4. Each stack frame will store any arguments which were passed to it.
2.10. There will be a heap, which stores a collection of unorganised data.
    2.10.1. Only objects may be stored on the heap.
    2.10.2. Data on the heap will not automatically cleaned up when no longer needed (unless using a memory management technique).
2.11. There will be three options for managing memory stored on the heap.
    2.11.1. One option will be naive allocation, where no memory is ever freed.
    2.11.2. One option will be reference counting.
        2.11.2.1. Each value stored in the heap will store how many references there are to it.
        2.11.2.2. When an object references another object, the reference count of the object it references will be incremented.
        2.11.2.3. When an object is deleted, the reference counts of any objects it references will be decremented.
        2.11.2.4. When the reference count for a value drops to zero, the value will be deleted.
    2.11.3. One option will be mark and sweep garbage collection.
        2.11.3.1. The garbage collector will 'mark' every object in the heap which is reachable from the stack.
        2.11.3.2. Every object which is unmarked will be deleted.
        2.11.3.3. The garbage collector will be called at regular intervals.
    2.11.4. The programmer will not be able to invoke the memory management technique themself.

3. The performance of the memory management techniques will be benchmarked.
3.1. A variety of algorithms will be implemented in the algorithm.
    3.1.1. The Fibonacci function will be implemented, in order to test out recursion.
    3.1.2. A program which introduces reference cycles between objects will be implemented.
3.2. Statistics about the execution of the program will be tracked.
    3.2.1. The execution time of the program.
    3.2.2. The number of objects at any one moment.
    3.2.3. The number of stack frames at any one moment.
    3.2.4. The memory usage of the interpreter at any one moment.

4. The programmer will interact with the interpreter through a command-line application.
4.1. The programmer will pass in their desired memory management mode as an argument.
4.2. The command-line application will have a 'REPL' mode.
    4.2.1. The programmer will then be able to input one line of source code at a time.
    4.2.2. This line will then be executed by the interpreter.
    4.2.3. The environment (variables, function definitions etc.) will be maintained between each line.

4.2.4. Any errors will be reported to the programmer, and will not crash the command-line application.

4.3. The command-line application will have a file mode.

4.3.1. The programmer will pass in a file path as an argument to the command-line application.

4.3.2. The application will read the contents of the file, which will then be executed by the interpreter.

4.3.3. If an error occurs, then interpretation of the file will end.

4.3.4. Statistics about the execution of the program will be written to a CSV file.

4.4. If the programmer passes a file path as an argument to the command-line application, then the file mode will be enabled. Otherwise, the REPL mode will be enabled.

1.7. Modelling

1.7.1. Grammar

The grammar for my programming language is as follows. It is written in a form similar to BNF, like `non-terminal -> productions`, and uses EBNF operators.

```
program -> statement*

statement -> variableDeclaration
           | functionDefinition
           | returnStatement
           | ifStatement
           | whileLoop
           | block
           | expressionStatement

expressionStatement -> expression ";"

variableDeclaration -> "let" IDENTIFIER ("=" expression)? ";"

functionDefinition -> "fu" IDENTIFIER "(" (IDENTIFIER ("," IDENTIFIER)*)? ")" block

returnStatement -> "return" expression? ";"

ifStatement -> "if" expression block ("else" (block | ifStatement))?

whileLoop -> "while" expression block

block -> "{" statement* "}"

expression -> assignment

assignment -> (call ".")? IDENTIFIER "=" assignment
            | ternary

ternary -> logical ("?" logical ":" logical)?

logical -> equality (("&&" | "||") equality)*

equality -> comparison (("!=" | "==") comparison)*

comparison -> bitwise ((">" | ">=" | "<" | "<=") bitwise)*
```

```
bitwise -> term (("&" | "|") term)*

term -> factor (("+" | "-") factor)*

factor -> unary (("*" | "/") unary)*

unary -> ("!" | "-")? exponent

exponent -> call ("^" exponent)?

call -> primary ( ("(" (expression ("," expression)*)? ")") | ("." IDENTIFIER) )*

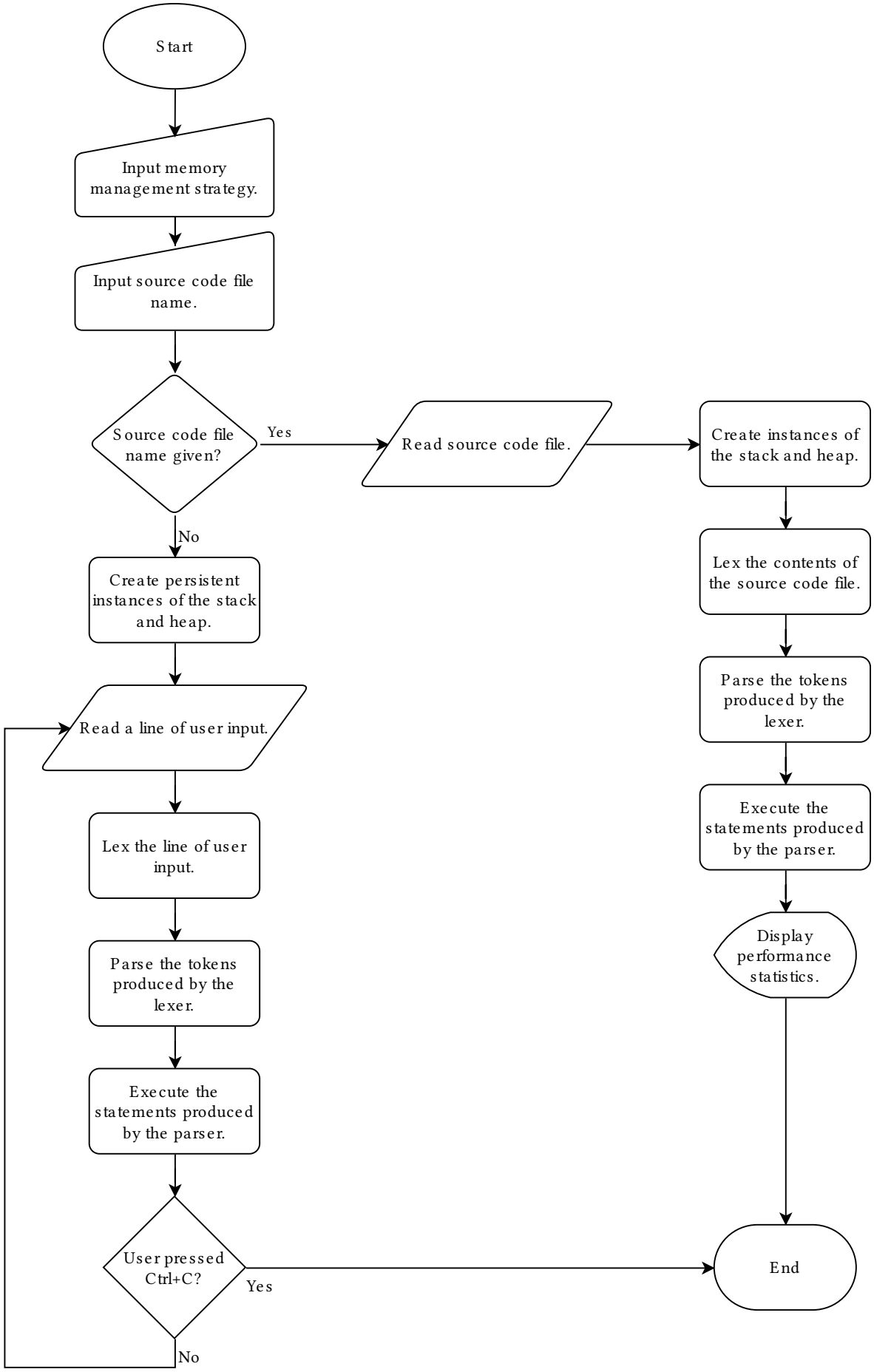
primary -> "(" expression ")"
         | STRING
         | INTEGER
         | FLOAT
         | IDENTIFIER
         | object
         | "true" | "false"

object -> "{" (IDENTIFIER ":" expression ("," IDENTIFIER ":" expression)*)? "}"
```

It defines the various forms of statements and expressions that are valid within the language, and encodes operator precedence and associativity. `assignment` expressions have the least precedence, whereas `primary` expressions have the most precedence. The grammar ignores whitespace, because it can all be safely ignored. I plan on using this grammar to write my parser, as each non-terminal within the grammar can typically correspond to one subroutine in the parser source code (within a recursive-descent parser).

1.7.2. Flowchart

The flowchart below describes the high-level operation of the interpreter. The user will be able to specify a memory management technique (either garbage collected, reference counted, or naive) which will be used to manage the heap for the execution of their program. If the user decided to run an entire file, then statistics about the performance of the program will be displayed after execution finishes.


Start
Input memory management strategy.
Input source code file name.
Source code file name given?
Yes
Read source code file.
Create instances of the stack and heap.
Lex the contents of the source code file.
Parse the tokens produced by the lexer.
Execute the statements produced by the parser.
Display performance statistics.
End
No
Create persistent instances of the stack and heap.
Read a line of user input.
Lex the line of user input.
Parse the tokens produced by the lexer.
Execute the statements produced by the parser.
User pressed Ctrl+C?
Yes
No

2. Design

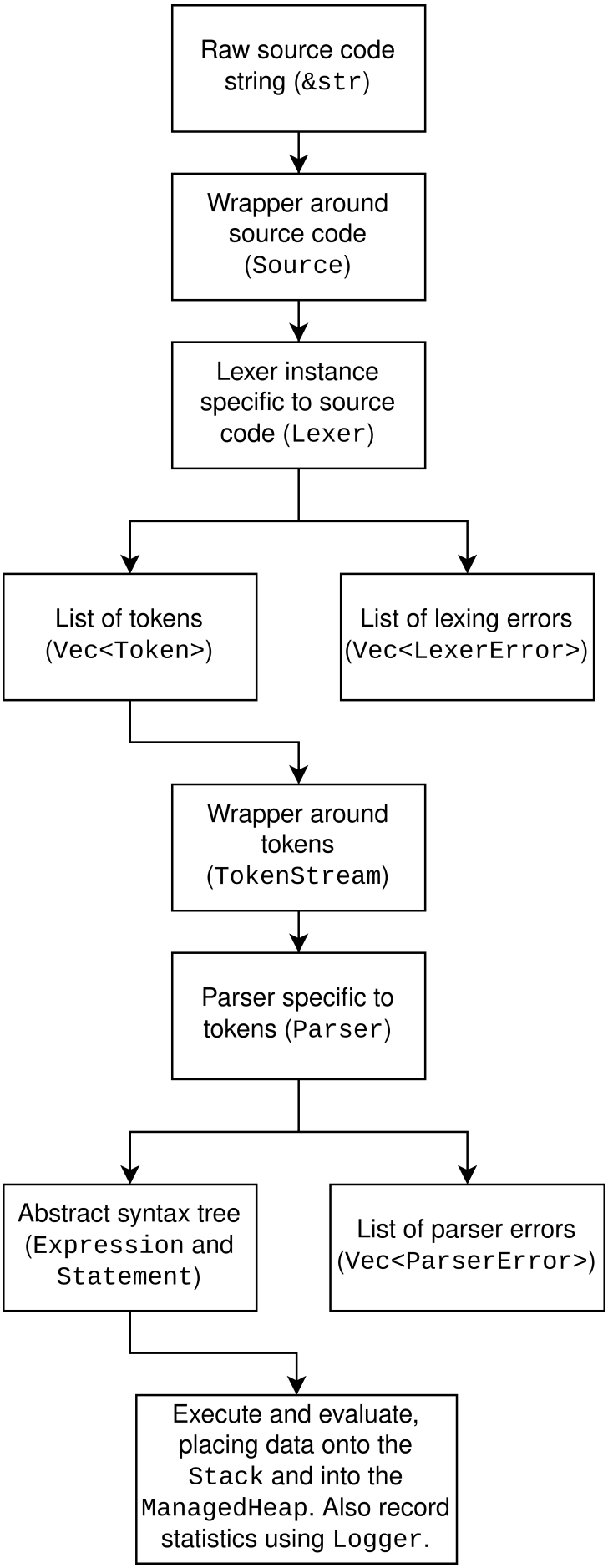
2.1. High-level overview

The purpose of the program is to read in user-written code, and then to attempt to execute this code, by performing successive transformations on the source code string, forming a structure which is much easier to execute. If, at any point, errors (which occurred as a result of user mistakes) occurred, they should be reported in a useful and informative format.

2.1.1. Key components

- The `Lexer` transforms the source code string (`Source`) into a sequence of tokens (`Token`).
- The `Parser` transforms the sequence of tokens (`TokenStream`) into an abstract syntax tree (encoded with `Statement` and `Expression`.)
- The execution stage executes statements (`Statement`) and evaluates expressions (`Expression`). It relies on the `Stack`, the `Environment`, and the `ManagedHeap`. Values resulting from evaluation of expressions are passed around in the `Value` data structure.
- Adjacent to the execution stage is memory management, which operates on the `ManagedHeap`. The `ManagedHeap` is either a `NaiveHeap`, a `ReferenceCountedHeap` or a `GarbageCollectedHeap`.
- Throughout the execution stage, execution statistics (`Entry`) are also logged by the `Logger`.
- The entire process is all controlled by the `main` function, which interacts with the user through the terminal, and calls the relevant components.

The diagram below shows how data flows through the system, and how the source code string is transformed into an abstract syntax tree.


Raw source code string (&str)
Wrapper around source code (Source)
Lexer instance specific to source code (Lexer)
List of tokens (Vec<Token>)
List of lexing errors (Vec<LexerError>)
Wrapper around tokens (TokenStream)
Parser specific to tokens (Parser)
Abstract syntax tree (Expression and Statement)
List of parser errors (Vec<ParserError>)
Execute and evaluate, placing data onto the Stack and into the ManagedHeap. Also record statistics using Logger.

2.2. Host programming language

I decided to write my interpreter using the Rust programming language, due to its advanced static type system, and its memory management model. The type system is *algebraic*, meaning that I can construct data structures using *sum* types (`enum` in Rust) and *product* types (`struct` in Rust), and then use `match` statements, to force me to consider all possible scenarios when writing code.

I am familiar with Rust, having used it in projects before, so it seemed like a suitable choice.

No external libraries (other than the Rust standard library) are required to write this interpreter.

2.3. Lexing

Lexing is the process which transforms the source code string into a sequence of tokens, which are then used by the parser.

2.3.1. The lexing algorithm

The lexer can be modelled as a finite state machine, where each new character advances the lexer into a new state. If a character occurs which is not accounted for in the state transition diagram, the lexer transitions into an invalid state, and emits an error.

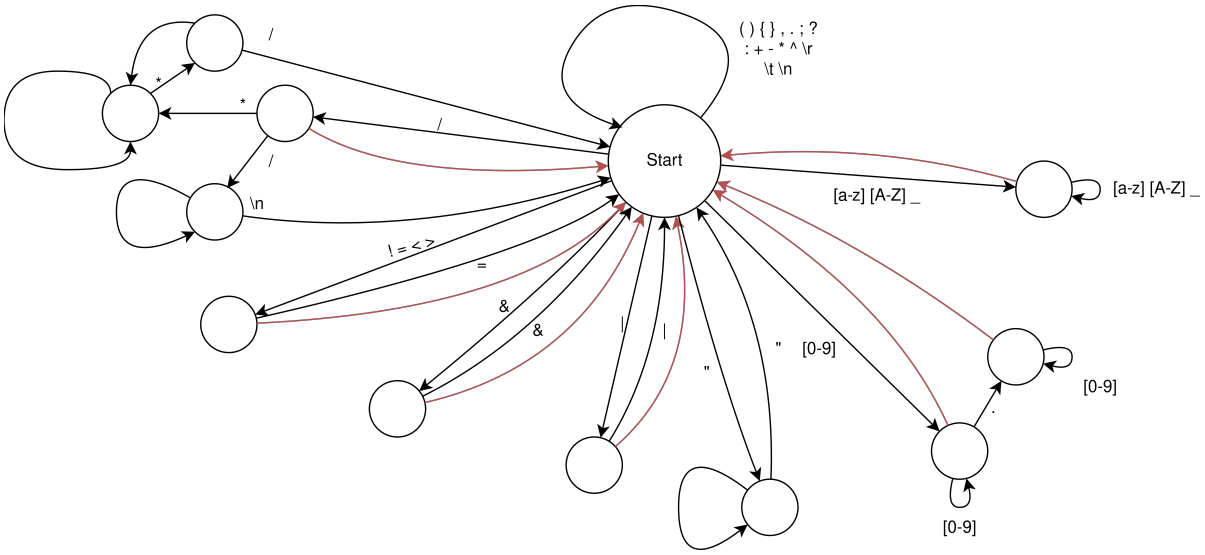


Figure 8: The state transition diagram for my lexer.

In the diagram above, a red arrow means that the source code string should be un-advanced. This isn't quite how the lexer is implemented in the code: in reality, the lexer is able to peek multiple characters into the future, in order to decide which state to transition into. It uses the `peek_after` method on the `Source` data structure to do this.

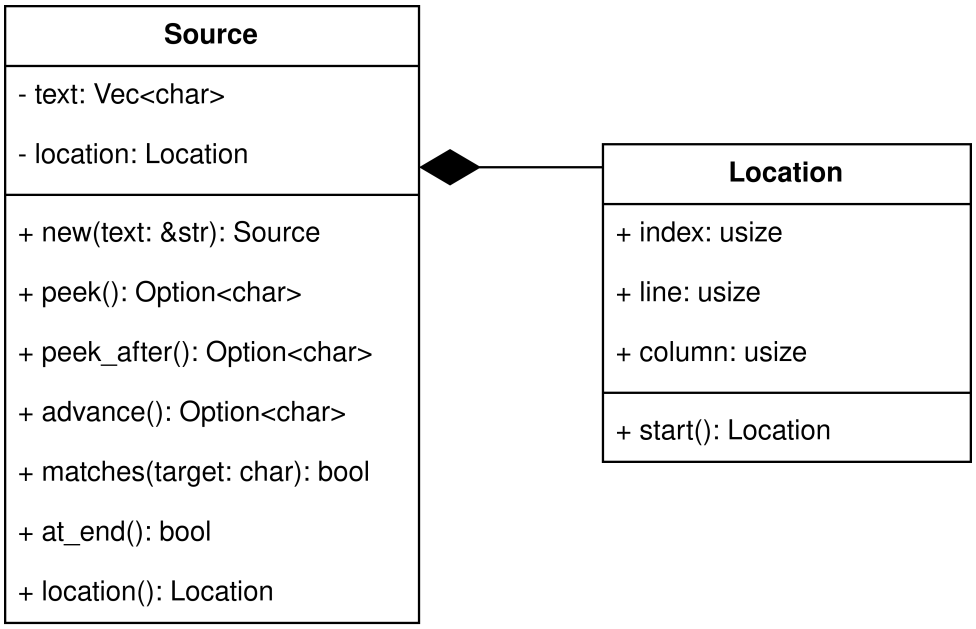
An arrow with no symbol attached to it can accept any character.

I have not included the tokens that the lexer emits on this diagram, as the diagram would become too complex and difficult to read.

The state transition diagram uses the longest token principle, where it always tries to consume the longest token before consuming the next token. For example, in the string `if x == 10`, the `==` is lexed as `DoubleEqual` rather than two of `Equal`.

2.3.2. The `Source` data structure

I decided to create a wrapper around the source code string, to consolidate all state into one place. The `Source` data structure depends on the `Location` data structure, in order to keep track of how much of the source code string has already been consumed.



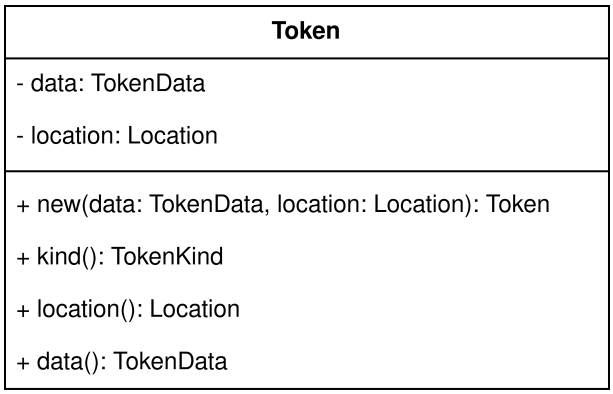
The data structure has a few useful helper methods:

- `peek` returns the next character, without advancing the position.
- `peek_after` returns the character after the next character, without advancing the position.
- `advance` returns the next character and advances the position.
- `matches` advances the position only if the next character is equal to a target. It also returns a Boolean representing whether the next character matched the target.
- `at_end` returns whether the entire source code string has been consumed.
- `location` returns the current location.

When the position is advanced, the `index` and `column` fields are incremented, and if the character is a newline character, then the `column` is reset to `1` and the `line` incremented.

2.3.3. The `Token` data structure

Tokens are what the lexer outputs. They represent the smallest useful unit of the language.



The `Token` structure requires on two `enum`s: `TokenData` and `TokenKind`.

- `TokenData` represents all the possible types of token, as well as their associated data. As such, it has a variant for each type of token: `LeftParenthesis`, `RightParenthesis`, `LeftBrace`, `RightBrace`, `Comma`, `Dot`, `Semicolon`, `QuestionMark`, `Colon`, `Plus`, `Minus`, `Star`, `Slash`, `Exponent`, `Bang`, `BangEqual`, `Equal`, `DoubleEqual`, `Greater`, `GreaterEqual`, `Less`, `LessEqual`, `Ampersand`, `DoubleAmpersand`, `Pipe`, `DoublePipe`, `String(String)`, `Float(f64)`, `Integer(i32)`,

`Boolean(bool)`, `If`, `Else`, `While`, `Return`, `Let`, `Fu`, `Identifier(String)`. Any variants denoted with brackets have associated data, and the type of that associated data is inside the brackets.
- `TokenKind` is a *shadow* of `TokenData`, and has the exact same variants, just with no associated data.

The `TokenData` structure has a method to extract its `TokenKind`, which is useful in the parsing stage for pattern matching.

2.4. Parsing

2.4.1. Recursive descent parsing algorithm

Recursive descent parsing attempts to transform a stream of tokens into an abstract syntax tree. It scans the list of tokens from left to right, consuming one token at a time.

The parser is designed around a grammar, which defines the syntax of the language. The grammar for Slang is given in Section 1.7.1, but is repeated below for convenience.

```
program -> statement*

statement -> variableDeclaration
           | functionDefinition
           | returnStatement
           | ifStatement
           | whileLoop
           | block
           | expressionStatement

expressionStatement -> expression ";"

variableDeclaration -> "let" IDENTIFIER ("=" expression)? ";"

functionDefinition -> "fu" IDENTIFIER "(" (IDENTIFIER ("," IDENTIFIER)*)? ")" block

returnStatement -> "return" expression? ";"

ifStatement -> "if" expression block ("else" (block | ifStatement))?

whileLoop -> "while" expression block

block -> "{" statement* "}"

expression -> assignment

assignment -> (call ".")? IDENTIFIER "=" assignment
            | ternary

ternary -> logical ("?" logical ":" logical)?

logical -> equality (("&&" | "||") equality)*

equality -> comparison (("!=" | "==") comparison)*

comparison -> bitwise ((">" | ">=" | "<" | "<=") bitwise)*

bitwise -> term (("&" | "|") term)*

term -> factor (("+" | "-") factor)*
```

```
factor -> unary (("*" | "/") unary)*

unary -> ("!" | "-")? exponent

exponent -> call ("^" exponent)?

call -> primary ( ("(" (expression ("," expression)*)? ")") | ("." IDENTIFIER) )*

primary -> "(" expression ")"
         | STRING
         | INTEGER
         | FLOAT
         | IDENTIFIER
         | object
         | "true" | "false"

object -> "{" (IDENTIFIER ":" expression ("," IDENTIFIER ":" expression)*)? "}"
```

Each rule in the grammar is given in the format **non-terminal** `->` **production**. A *non-terminal* is a syntactic variable which represents a concept in a programming language, for example `whileLoop`. The right-hand-side of the rule (after the `->`) is the production. A production is a way that this non-terminal can appear in the source code string. A non-terminal can have multiple productions, and these are separated by the `|` symbol, to represent a union. The metalanguage that my grammar is defined in supports the following operators:

- `|` means "or".
- `*` means "zero or more of".
- `+` means "one or more of".
- `?` means "zero or one of".
- `()` groups syntax together.
- `->` separates the left-hand-side from the right-hand-side.

Productions can contain other non-terminals (including themselves), and *terminals*. A terminal is an actual token.

An example of a rule is:

```
ifStatement -> "if" expression block ("else" (block | ifStatement))?
```

This reads as: `ifStatement` is a non-terminal, and is constructed as follows: the string `if` (a terminal), followed by an expression, and then a block. Optionally, an else-statement may be attached, which is constructed with: the string `else`, followed by either a block or an if-statement. As seen, this is a recursive production.

The parser starts by attempting to parse a 'start' symbol (in the case of Slang, `program`), and then decides which production to attempt to parse. For the grammar I wrote, the appropriate production can always be selected by looking at only the next token.

In code, I implement the parser by adding a subroutine for each rule. The code then looks very similar to the grammar itself, as the subroutine for each production simply calls the appropriate subroutines for other non-terminals, occasionally accessing the `TokenStream` to consume an actual terminal/token.

Continuing the example of `ifStatement`, a pseudocode version of its parsing algorithm looks like this:

```
PROCEDURE IfStatement()
  Consume(If)

  condition <- Expression()

  executeIfTrue <- Block()

  executeIfFalse <- null

  IF Tokens.PeekKind() = Else
    Consume(Else)

    IF Tokens.PeekKind() = If
      executeIfFalse <- IfStatement()
    ELSE IF Tokens.PeekKind() = LeftBrace
      executeIfFalse <- Block()
    ELSE
      THROW ParserError
    ENDIF
  ENDIF

  RETURN {condition, executeIfTrue, executeIfFalse}
ENDPROCEDURE
```

Here, `Consume` is a subroutine which consumes the next token, as long as its kind matches the given kind. This corresponds to `TokenStream::consume`. `Expression()` and `Block()` are subroutines which parse an expression and a block, respectively. `Tokens.PeekKind()` gets the kind of the next token; this (more or less) corresponds to `TokenStream::Matches` in the code.

Similar subroutines are written for each of the non-terminals.

One issue which is very important in parsing is error handling. I chose to use 'panic-mode', where, after an error is encountered, the parser blindly consumes tokens, until a token which signals a fresh start is encountered. In my program, these tokens are semicolons, and the keywords `if`, `fu`, `let` and `while`. An error is also emitted and logged. Choosing this methods allows more unique errors to be picked up, while also minimising 'cascaded' errors (errors which only appear as a result of a previous error).

2.4.2. The `TokenStream` data structure

The `TokenStream` structure is a wrapper around a list of tokens. It provides useful helper methods which allow the parser to do easy pattern matching.

TokenStream
- tokens: VecDeque<Token>
+ new(tokens: Vec<Token>): TokenStream
+ peek(): Option<&Token>
+ advance(): Option<Token>
+ only_take(targets: &[TokenKind]): Option<Token>
+ matches(targets: &[TokenKind]): bool
+ binary_operator(targets: &[BinaryOperator]): Option<(BinaryOperator, Location)>
+ unary_operator(targets: &[UnaryOperator]): Option<(UnaryOperator, Location)>
+ consume_identifier(): Result<String, ParserError>
+ consume(kind: TokenKind): Result<Token, ParserError>
+ at_end(): bool

The underlying list of tokens is a `VecDeque`, which is a queue-like structure provided by the Rust standard library that allows pushing and popping from both ends of the list. This means that methods such as `advance` can consume tokens from the start of the list.

The `peek` method allows the parser to look at the next unconsumed token, without actually advancing the token stream. This method returns a reference to the token.

The `advance` method advances the token stream and returns the next unconsumed token.

The `only_take` method takes in a list of targets, each of which has type `TokenKind`. It advances the token stream and returns the next unconsumed token only if the next token has kind equal to one of the targets.

The `matches` method is very similar to `only_take`, apart from instead of returning the next unconsumed token, a Boolean value is returned indicating whether the token stream was advanced.

The `binary_operator` method takes in a list of targets, where each target is a `BinaryOperator`. If the next token is a binary operator and matches one of the targets, the token stream is advanced and the binary operator is returned.

The `unary_operator` method takes in a list of targets, where each target is a `UnaryOperator`. If the next token is a unary operator and matches one of the targets, the token stream is advanced and the unary operator is returned.

The `consume_identifier` method attempts to consume the next token, but will only do so if it is an identifier. If it is an identifier, then the string contained within it is returned, otherwise an error is returned.

The `consume` method attempts to consume the next token, but only if it has a kind equal to the target. An error is returned if the next token doesn't match the target.

The `at_end` method returns whether the token stream has been fully consumed.

2.4.3. The `Expression` data structure

The `Expression` structure is an `enum` which has a variant for every possible type of expression:

- `Ternary` stores a condition, a left operand and a right operand, each of which are expressions.
- `Binary` stores a left operand, an operator, and a right operand.

- `Unary`stores an operator and an operand.
- `Call` stores a function and a list of arguments.
- `Assignment` stores an identifier and a value to be assigned.
- `Grouping` stores a contained expression.
- `Literal` stores a value.
- `Variable`stores an identifier.
- `GetField` stores an expression representing an object, and a field identifier.
- `SetField` stores an expression representing an object, a field identifier, and a value expression to be assigned.
- `Object` stores a key-value structure mapping strings to expressions.

The main method on this structure is the `evaluate` method, which contains all of the code needed to evaluate any one of these expressions.

Two other structures (`BinaryOperator` and `UnaryOperator`) are used to represent the operations within the `Binary` and `Unary` variants of `Expression`.

2.4.4. The `Statement` data structure

The `Statement` structure is an `enum` which has a variant for every possible type of statement:

- `VariableDeclaration` represents a variable declaration, and stores an identifier and an (optional) expression.
- `IfStatement` represents an if-statement, and stores a condition (expression), and two statements representing the possible execution branches of the if-statement.
- `FunctionDefinition` represents a function definition, and stores an identifier, a list of parameter identifiers, and an expression representing the body of the function.
- `Return` stores an (optional) return value, and represents a return statement.
- `WhileLoop` stores a condition and an expression representing its body. It represents a while-loop.
- `Block` stores a list of statements. It can represent the body of an if-statement, while-loop or function.
- `Expression` stores a single expression, and represents an expression statement.

The main method on this structure is the `execute` method, which contains all of the code needed to execute any one of these statements.

2.5. Execution/evaluation

2.5.1. The execution/evaluation algorithm

The execution/evaluation algorithm performs a post-order traversal of the abstract syntax tree. Each `Expression` node represents an expression to be evaluated, and each `Statement` node represents a statement to be executed.

The parser initially produces a list of statements to be executed. The execution algorithm begins by searching through all of these statements for function definitions, and executing these statements first, thus declaring all functions that could be used by the program. This ensures that functions can be used before they are declared in the code. The algorithm then executes each of the other statements in order.

The process for executing an individual statement is as follows:

- If the statement is a variable declaration, then evaluate the initialiser (if given), perform processes required for reference counting (detailed in Section 2.6.4), then use the `Stack::top` method to get the most recent environment of the most recent stack frame, and then use `Environment::define` to define the variable with its initialiser. A `ControlFlow::Continue` is emitted.

- If the statement is a function definition, then use the `Stack::top` method to get the most recent environment of the most recent stack frame, and then use `Environment::define` to define the function. A `ControlFlow::Continue` is emitted.
- If the statement is an if-statement, evaluate the condition. If the condition is a Boolean true value, execute the block of the if-statement. If the condition is a Boolean false value, then execute the block of the else-statement (if given). Otherwise, emit an error.
- If the statement is a while-loop, continue to evaluate the condition, and while the condition is a Boolean true value execute the block of the while-loop. A `ControlFlow::Continue` is emitted when the condition evaluates to the Boolean false value.
- If the statement is a block, first enter a new scope using the `Stack::enter_scope` method, then execute all function definitions within the block. Afterwards, execute each of the other statements in sequence. If any of the statement executions emits a `ControlFlow::Break`, then stop executing the statements. Perform processes required for reference counting (detailed in Section 2.6.4). Then exit the scope using `Stack::exit_scope`, and perform processes required for garbage collection (detailed in Section 2.6.3). If any of the statements emitted a `ControlFlow::Break`, then that is emitted, otherwise a `ControlFlow::Continue` is emitted.
- If the statement is an expression statement, then evaluate the expression and emit a `ControlFlow::Continue`.
- If the statement is a return statement, then emit a `ControlFlow::Break` with the value of the evaluated expression.

In order to execute statements, expressions must also be evaluated.

- If the expression is a ternary expression, the condition is evaluated. If the condition evaluates to the Boolean true value, then the left operand is evaluated and returned. If the condition evaluates to the Boolean false value, then the right operand is evaluated and returned. Otherwise, an error is returned.
- If the expression is a binary expression, the left and right operands are evaluated, and the operation is applied to them, as long as operation is defined between those types. If not, an error is returned.
- If the expression is a unary expression, the operand is evaluated and the operation is applied to it, as long as the operation is defined on the type of the operand. If not, an error is returned.
- If the expression is a function call:
  - If the function is user-defined, the arguments are evaluated, a new stack frame is pushed, and then variables are defined, whose identifiers are the parameter name, and whose values are the values of the evaluated arguments. If the number of arguments does not match the number of parameters, an error is returned. The body of the function is then executed. Reference counting processes are then performed (detailed in Section 2.6.4), and the stack frame is popped.
  - If the function is native, then the appropriate code is ran on the arguments to the function, depending on which native function has been called.
- If the expression is a grouping, the interior expression is evaluated and returned.
- If the expression is a literal, the interior value is returned.
- If the expression is a variable, the value of that variable is retrieved from the stack and returned. If the variable does not exist, an error is returned.
- If the expression is a field get, then the object expression is evaluated. If it is in fact an object, or object reference, then the field is looked up, and its value returned (or an error returned). If not, an error is returned.
- If the expression is a field set, the object expression is evaluated. If it is in fact an object, or an object reference, then the field is set, to the value of the evaluated operand, and the previous value (if there was one) is returned.

- If the expression is an object, an object is constructed with the evaluated fields, and returned.

Throughout this entire process, errors which occur during the evaluation and execution process are returned and propagated up the call stack.

As suggested by the algorithm, execution and evaluation lend themself to recursive implementation, which is how I decided to implement this.

2.5.2. Data structures to represent values

The `Value` structure is an `enum` which can represent all possible values within the language. It has a variant for each data type:

- The `String` variant stores a `String`, and represents a string.
- The `Float` variant stores an `f64`, and represents a floating point number.
- The `Integer` variant stores an `i32`, and represents an integer.
- The `Boolean` variant stores a `bool`, and represents a Boolean.
- The `Function` variant stores a `Function`, and represents a function (may be user-defined or native).
- The `ObjectReference` variant stores a `Pointer`, and represents a reference to an object.
- The `Object` variant stores an `Object`, and represents an object.

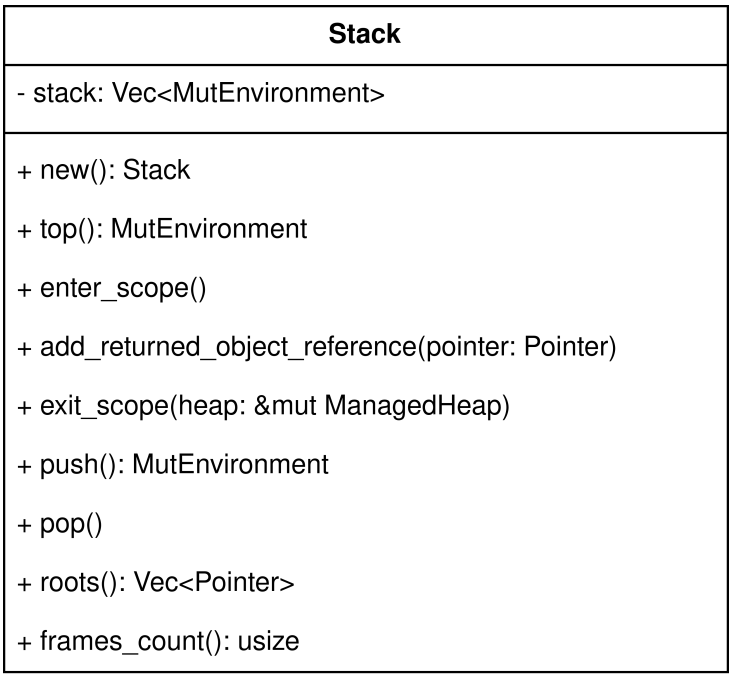
Each variant also has a corresponding variant in the `Type` structure. The `Type` structure is almost identical to the `Value` structure, but it is effectively just a flag indicating the type of a value, and doesn't store the value itself. `Object` and `ObjectReference` have the same type.

Functions are stored in a structure called `Function`, which is also an `enum`. It has a variant for each type of function:

- The `UserDefined` variant represents a user-defined function. It has a `parameters` field, which is a list of the names of the parameters. It also has a `block` field, which is the body of the function.
- The `Native` variant represents a native function, which is stored as `NativeFunction`, which is also an `enum`. It has a variant for each native function: `Print`, `Format`, `Input`, `Int`, `Float`.

2.5.3. The `Stack` data structure

The `Stack` is the data structure within which all variables are stored. The underlying list is implemented as `Vec`, a structure provided by the Rust standard library which is a list, but allows pushing and popping like a stack.



Each stack frame is, within itself, another stack. This is because, within a function call, there may be multiple scopes (i.e. if there is an if-statement). Each function call gets its own stack frame, but each scope (any time there is a block, indicated by curly braces) gets its own *environment*.

The `top` method returns a reference to the innermost stack frame, meaning the most recently pushed stack frame.

The `enter_scope` method creates a new `Environment`, and links it to the most recent environment in the most recent stack frame. This method should be called any time an if-statement, while-loop etc. is entered.

The `add_returned_object_reference` adds a pointer to a special list stored inside the most recent environment in the most recent stack frame. The reason for this is explained in Section 2.6.4.

The `exit_scope` method should be called whenever a scope is exited, for example when an if-statement is exited. It decrements all references stored in the most recent environment's `returned_object_references` list, and replaces the most recent environment with its parent. The way in which environments are chained together is explained in Section 2.5.4.

The `push` method pushes a new frame onto the stack, and returns a mutable reference to the new environment that is created inside of it. This should be called whenever a new function is entered.

The `pop` method pops a stack frame off of the stack.

The `roots` method gathers the roots from each stack frame in the stack. This is used within garbage collection.

The `frames_count` method returns the number of frames currently on the stack. This is useful for statistics tracking.

2.5.4. The `Environment` data structure

`Environment` is a data structure which stores all of the variables for a single scope.

Environments are chained together, so that variables declared inside of enclosing scopes can still be accessed inside inner scopes. Chaining works as follows: each environment has a parent (unless it is the global scope). The environment stores a mutable reference to this parent. When a new scope is

created within a stack frame (via the `Stack::enter_scope` method), a new scope is created, with its parent set to the most recent environment within that stack frame. When a new stack frame is created (and therefore a new scope is created for that stack frame), the new scope has its parent set to the global scope. This means that all scopes are somewhat connected, in a similar way to a tree, however instead of each parent referencing its children, each child references its parents.

Variables are stored in a key-value data structure (`HashMap`), which maps strings to values. The value is actually optional, as my language allows declaring variables without initialising them.

The `returned_object_references` stores all pointers which this environment has received from functions that have been called. This is necessary for reasons explained in Section 2.6.4.

Environment
- parent: Option<MutEnvironment>
- scope: HashMap<String, Option<Value>>
- returned_object_references: Vec<Pointer>
+ new(): Environment
+ define(identifier: String, value: Option<Value>)
+ assign(identifier: String, value: Option<Value>): Result<Option<Value>, EnvironmentError>
+ get(identifier: &str): Result<Value, EnvironmentError>
+ global(self_reference: MutEnvironment): MutEnvironment
+ roots(): Vec<Pointer>
+ returned_object_references(): &Vec<Pointer>
+ add_returned_object_references(pointer: Pointer)
+ parent(pointer: Pointer): Option<MutEnvironment>
+ values(): Vec<Value>

The `define` method defines a new variable, and inserts it into the most recent scope.

The `assign` method assigns a value to an already-existing variable. In order to find the correct variable to mutate, it first checks its own key-value data structure, and if the variable is not present then it hands off the task to its parent. This process continues up the chain of environments, until either the variable is found, or the global scope is reached and the variable is not found, in which case an error is propagated back up. This method also returns the previous value stored in that variable (if there was one), which is helpful for memory management, specifically reference counting.

The `get` method retrieves the value stored within a variable. To find the variable, it uses the same process as `assign`, working its way up the chain of environments.

The `global` method finds the global scope by recursively traversing the chain of parents.

The `roots` method gathers all roots from itself, and all of its parents, by traversing the chain of parents up to the global scope. This is used in garbage collection.

The `returned_object_references` method returns a reference to the `returned_object_references` list. Used for reference counting.

The `add_returned_object_reference` method adds a new pointer to the `returned_object_references` list. Used for reference counting.

The `parent` method returns a mutable reference to the parent of the current environment.

The `values` method returns a list of all values stored within the current environment.

2.6. Memory management

2.6.1. Associated data structures

The `ManagedHeap` data structure is a wrapper around all of the specific types of heap, and is implemented as an `enum`. It is always either a `NaiveHeap`, a `ReferenceCountedHeap`, or a `GarbageCollectedHeap`.

All types of heap rely on `Pointer`, which is an alias for a special smart pointer provided by the Rust standard library which supplies shared ownership with interior mutability. The `Pointer` references `HeapObject`, which stores the object itself, along with metadata required for the memory management methods to work (`marked` for garbage collection, `reference_count` for reference counting).

Most methods on the various heaps are explained in detail below, but there are a few extra methods:

- `ReferenceCountedHeap::conditionally_decrement` is a wrapper around the `decrement` method, which accepts a `Value` instead of an `Object`, only attempting to decrement the reference count if the `Value` is an instance of an `ObjectReference`.
- The `objects_count` method is included on all heaps, and is used for statistics tracking.
- `ManagedHeap::get_technique_code` is used for debugging purposes, and returns a string: `rc` if reference counting is being used, `na` if naive allocation is being used, and `gc` if garbage collection is being used.

../images/Memory management.png

2.6.2. Naive allocation

Naive allocation does not have any cleanup procedure whatsoever: memory for objects is allocated on the heap and never deallocated. The allocation algorithm (named `allocate` in the code) is as follows:

1. Iterate through each property, p of the object, X.
2. If p is of type object:
   2.1. Allocate memory for p.
   2.2. Replace p with a pointer to the allocated memory for p.
3. Place X onto the heap.
4. Return a pointer to X.

As no memory for objects is ever cleaned up, the heap continues to grow until the program is terminated.

2.6.3. Garbage collection

I decided to use mark-and-sweep garbage collection for my project, as it is relatively easy to understand, and is very robust.

The algorithm for allocating memory on the heap is identical to the naive allocation algorithm

The algorithm for mark-and-sweep is as follows:

```
PROCEDURE Manage(roots)
  FOR EACH root IN roots
    Traverse(root)
  ENDFOR

  FOR object IN heap
    IF NOT object.IsMarked()
      REMOVE object FROM heap
    ENDIF
  ENDFOR

  FOR object IN heap
    object.UnMark()
  ENDFOR
ENDPROCEDURE

PROCEDURE Traverse(root)
  IF root.IsMarked() THEN RETURN

  root.Mark()

  FOR EACH property IN root
    IF property IS ObjectReference
      Traverse(property)
    ENDIF
  ENDFOR
ENDPROCEDURE
```

In the code, `Manage` is called `manage`, and `Traverse` is called `traverse`.

The process starts from a list of roots, which are all of the directly reachable objects. For an object to be directly reachable means that there is a reference directly to that object that exists somewhere on the stack. To get all of these roots, the `roots` method on the `Stack` data structure is used.

After the roots are collected, each root is traversed individually. Traversing a root means "marking" it (implemented as updating a Boolean flag from false to true), then traversing each object that it references. This lends itself to a recursive implementation, which is what I did.

The manage procedure is called every time that a stack frame is exited. This is frequent enough to keep the size of the heap relatively small, but infrequent enough that it doesn't slow down execution of the program too much.

2.6.4. Reference counting

The algorithm for reference counting is very similar to the one used for naive allocation and garbage collection, but it has a small modification:

1. Iterate through each property, p of the object, X.
2. If p is of type object:
   2.1. Allocate memory for p.
   2.2. Replace p with a pointer to the allocated memory for p.
3. If instead p is of type object reference.
   3.1. Increment the reference count for p.
4. Place X onto the heap.
5. Return a pointer to X.

Whenever memory for a new object is allocated, the reference counts of any objects which this new object references, must be incremented.

The general principle for reference counting is that each object has a reference count, and this reference count is incremented whenever a new reference is created to that object, and decremented whenever that reference goes out of scope. When the reference count of an object drops to 0, then the memory for that object is deallocated, and all objects which that object references have their reference counts decremented. This lends itself to a recursive implementation.

The main complexity with reference counting is introduced by the number of different places that these increment and decrement procedures need to be called.

Incrementing the reference count needs to happen in the following scenarios:
- When assigning an object reference to a variable.
- When setting the value of a field in an object to an object reference.
- When assigning an object reference (which was passed through as an argument) to a parameter.
- When allocating memory for an object.
- When defining a variable, and initialising it to an object reference.
- When returning an object reference from a function.

Decrementing the reference count must happen in the following cases:
- When the reference count of an object drops to 0, so all objects it references must have their counts decremented.
- When exiting a stack frame, so all object references which were returned from functions called within this stack frame must be decremented (more on this below).
- When a variable is mutated, and the previous value was an object reference.
- When the value of a field on an object is updated, and the previous value was an object reference.
- When a stack frame is exited, so all variables which directly reference an object go out of scope.
- When a variable is redefined (shadowing), and the previous value was an object reference.
- When a scope is exited, so all variables which directly reference an object go out of scope.

There is a specific problem which arises from returning object references from functions. Within the function, f, which is returning the object reference x, f cannot know how x will be used: x could be assigned to a variable in the calling function, c, or it could be discarded, like:

```
fu c() {
  f();
}

fu f() {
  let x = {};
  return x;
}
```

If x is in fact used, then its reference count should somehow end up as 1, but if it is not used, then its reference count should end up as 0, so that its memory can be deallocated.

One solution to this problem would be to decrement the reference count of x to 0 when f returns, and then if x is in fact used in c, then its reference count would be incremented back up to 1. Otherwise, it would stay at 0 and be deallocated. However, this doesn't quite work: if x has its reference count decremented to 0 at any point, then it will be immediately deallocated and will not be usable.

The solution which I landed on, was to maintain a list l, within the stack frame of c, of all object references given to c by functions it called (i.e. f). When f returns, it increments the reference count

of x, and then in c, x is added to the list l. When c returns, all object references within l have their reference counts decremented. This ensures that object references returned from functions live for the correct amount of time.

2.7. Statistics tracking

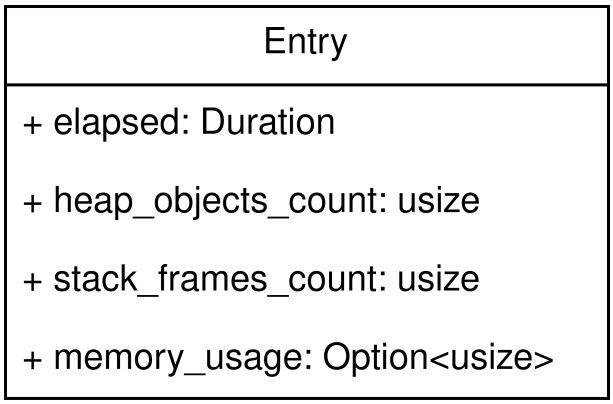
The statistics tracking module is essential for benchmarking the performance of the programming language, its associated memory management techniques, and the programs written in it.

2.7.1. Entry logging frequency

When deciding how frequently to log an entry, there were to main options: either logging an entry at regular time intervals (e.g. every millisecond), or logging an entry every time that some sort of checkpoint was passed (e.g. every time an expression is evaluated). I opted for logging an entry every time that a statement is executed, specifically before that statement is executed. I also log an entry after the entire program has finished executing.

2.7.2. The `Entry` data structure

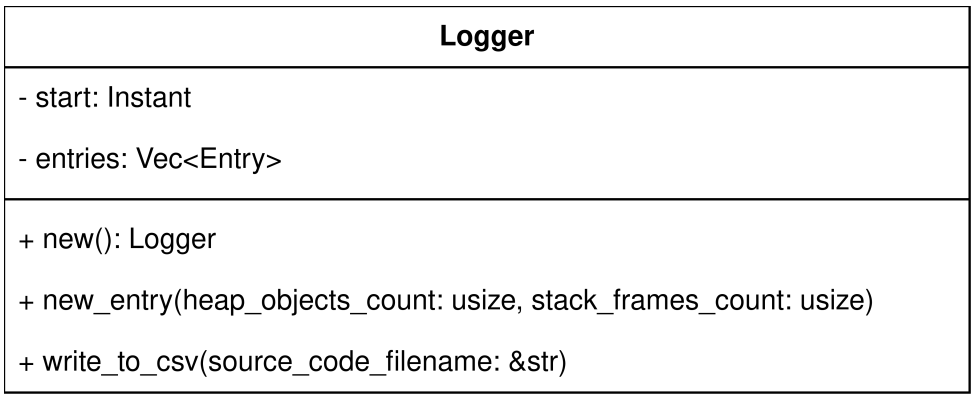
This data structure represents a single entry into the statistics log, and is a simple record structure.



The memory usage of the interpreter is measured in bytes. This field is optional, as it is sometimes not possible to retrieve the memory usage of the interpreter, namely if the operating system that the program is being run on is not Linux.

2.7.3. The `Logger` data structure

This data structure represents an ordered collection of entries. The program is able to add new entries through the `Logger` and this structure is responsible for writing its data to a CSV file.



The `start` field represents the instant that the logger is created, and this field is initialised with a value when the `Logger` is created.

Every time the `new_entry` method is called, the `elapsed` field for the new `Entry` is calculated based on how much time has elapsed since `start`.

Memory usage can only be calculated when the host operating system is Linux, and is determined by accessing the `/proc/self/status` file, and looking for the line beginning with `VmRSS:`. The memory usage in this file is given in KB, so is multiplied by 1000 to convert into bytes.

When `new_entry` is called, a new `Entry` is created and pushed onto the end of the `new_entry` structure.

When `write_to_csv` is called, all data is written, one entry per line, to a CSV file which has the name `<source_code_filename>.csv`.

2.8. User interface

The user interacts with the program through a command-line application. When the user runs the program, they must specify two pieces of information, which they communicate via command line arguments. Firstly, they must decide which memory management method they would like to use. This is either naive mode (`na`), reference counting (`rc`), or garbage collection (`gc`). They then must specify the name of a file they'd like to run. This argument can be left blank, in which case a REPL is opened.

Examples of valid commands are as follows:

- `slang na test.slang` - runs the file `test.slang` in naive mode.
- `slang rc` - opens a REPL in reference counting mode.
- `slang gc file.txt` - runs the file `file.txt` in garbage collection mode.

Both REPL mode and file mode use a common `run` function to run source code. The `run` function takes in the source code string to be ran, an instance of a `Stack`, an instance of a `ManagedHeap`, and an instance of a `Logger`. This allows REPL mode to maintain state between lines of code executed, as it can create a stack and heap first, and then pass a mutable reference to these into the `run` function.

2.8.1. REPL mode

REPL (Read-Evaluate-Print-Loop) mode works as follows:

1. A user types a single line of input.
2. If the user pressed `Ctrl+C`, then exit.
3. Otherwise, the line of code is executed.
4. Return to step 1.

Throughout the REPL, a persistent environment is maintained, meaning that if the user declares a variable in one line, they are able to use it in another. Also, any errors will not crash the REPL; the user will still be able to enter and execute lines of code. This includes lexer errors, parser errors, and errors which occur in evaluation.

2.8.2. File mode

In file mode, the contents of a file are read, and then executed in their entirety. Any errors will halt the execution of the file. In any case, the performance statistics will be written to a CSV file at the end of execution by the `Logger`.

3. Technical solution

The source code for my interpreter can be found in full in Appendix B.

Below are some interesting sections of my code which implement particularly complex functionality.

File name	Description	Document section
`src/lexer.rs`	Implements the finite state machine for the lexer described in Section 2.3.1.	Appendix B.9
`src/parser.rs`	Implements recursive-descent parsing to produce an abstract syntax tree.	Appendix B.11
`src/expression.rs`	Implements recursive evaluation of expressions via a post-order traversal of the abstract syntax tree. Also implements some reference counting functionality.	Appendix B.8
`src/statement.rs`	Implements the execution of statements. Also implements some reference counting functionality.	Appendix B.14
`src/heap/garbage_collected.rs`	Implements mark-and-sweep garbage collection.	Appendix B.3

4. Testing

4.1. Testing table

The test data and expected outcome files can be found in Appendix C.

Test	Objectives	Description	Test data	Expected outcome	Pass/ fail
0	1.2	C-like syntax.	`t00.slang`	`t00.out`	Pass
1	1.3.2	Dynamically typed variables.	`t01.slang`	`t01.out`	Pass
2	1.3.3	Dynamically typed function return types.	`t02.slang`	`t02.out`	Pass
3	1.4	Strongly typed.	`t03.slang`	`t03.out`	Pass
4	1.5.1	Arithmetic infix operators.	`t04.slang`	`t04.out`	Pass
5	1.5.1.4	Division by zero.	`t05.slang`	`t05.out`	Pass
6	1.5.2	The prefix `-` operator.	`t06.slang`	`t06.out`	Pass
7	1.5.3	The ordering operators.	`t07.slang`	`t07.out`	Pass
8	1.5.4	The equality operators.	`t08.slang`	`t08.out`	Pass
9	1.5.5	The logical operators.	`t09.slang`	`t09.out`	Pass
10	1.5.6	The bitwise operators.	`t10.slang`	`t10.out`	Pass
11	1.5.7	The prefix `!` operator.	`t11.slang`	`t11.out`	Pass
12	1.5.8	The ternary operator.	`t12.slang`	`t12.out`	Pass
13	1.5.9	Order of operator precedence.	`t13.slang`	`t13.out`	Pass
14	1.7	Creating objects.	`t14.slang`	`t14.out`	Pass

Test	Objectives	Description	Test data	Expected outcome	Pass/ fail
15	1.7.3	Accessing object fields.	`t15.slang`	`t15.out`	Pass
16	1.8	Initialising variables with no value.	`t16.slang`	`t16.out`	Pass
17	1.8	Initialising variables with a value.	`t17.slang`	`t17.out`	Pass
18	1.8.4	Variable scope.	`t18.slang`	`t18.out`	Pass
19	1.8.5, 1.3	Variable mutability.	`t19.slang`	`t19.out`	Pass
20	1.9.1	`if`-statements.	`t20.slang`	`t20.out`	Pass
21	1.9.2	`else`-statements.	`t21.slang`	`t21.out`	Pass
22	1.9.3	`while`-loops.	`t22.slang`	`t22.out`	Pass
23	1.10.1	Declaring subroutines.	`t23.slang`	`t23.out`	Pass
24	1.10.2.2	Subroutine parameter counts.	`t24.slang`	`t24.out`	Pass
25	1.10.5, 3.1.1	Recursion.	`t25.slang`	`t25.out`	Pass
26	1.10.6	Subroutine return values.	`t26.slang`	`t26.out`	Pass
27	1.11.1	`print` function.	`t27.slang`	`t27.out`	Pass
28	1.11.2	`format` function.	`t28.slang`	`t28.out`	Pass
29	1.11.3	`input` function.	`t29.slang`	`t29.out`	Pass
30	1.11.4	`int` function.	`t30.slang`	`t30.out`	Pass
31	1.11.5	`float` function.	`t31.slang`	`t31.out`	Pass
32	2.8.1	Error location reporting.	`t32.slang`	`t32.out`	Pass
33	2.8.2	Descriptive error messages.	`t33.slang`	`t33.out`	Pass
34	2.9.1, 2.9.2	Stack frame pushing and popping.	`t34.slang`	`t34.out`	Pass
35	2.10	The heap.	`t35.slang`	`t35.out`	Pass
36	2.11.1	Naive allocation.	`t36.slang` (in `na` mode).	`t36.out`	Pass
37	2.11.2	Reference counting.	`t37.slang` (in `rc` mode).	`t37.out`	Pass
38	2.11.2, 3.1.2	Reference cycles with reference counting.	`t38.slang` (in `rc` mode).	`t38.out`	Pass
39	2.11.2	Return values with reference counting.	`t39.slang` (in `rc` mode).	`t39.out`	Pass
40	2.11.2	Variable mutation with reference counting.	`t40.slang` (in `rc` mode).	`t40.out`	Pass

Test	Objectives	Description	Test data	Expected outcome	Pass/ fail
41	2.11.3	Garbage collection.	`t41.slang` (in `gc` mode).	`t41.out`	Pass
42	2.11.3, 3.1.2	Reference cycles with garbage collection.	`t42.slang` (in `gc` mode).	`t42.out`	Pass
43	2.11.3	Return values with garbage collection.	`t43.slang` (in `gc` mode).	`t43.out`	Pass
44	2.11.3	Variable mutation with garbage collection.	`t44.slang` (in `gc` mode).	`t44.out`	Pass
45	3.2	Execution statistics are tracked.	`t45.slang` (in `na` mode).	`t45.slang .csv` created, number of heap objects gradually grows, number of stack frames alternates between 1 and 2.	Pass
46	4.1	Memory management mode selection.	Run executable, with `na`, `rc` and `gc` as input. In each instance, run `print( MEMORY_MANAGEMENT );`	Correct code is displayed.	Pass
47	4.2	REPL mode.	Run executable with no file name, and input lines of code.	Interactive prompt entered and lines of code executed.	Pass
48	4.2.3	Environment maintained for REPL mode.	Run executable with no file name, run `let x = 1;`, press enter, and then run `print(x);`.	`1` printed to screen.	Pass
49	4.2.4	Errors do not crash REPL mode.	Run executable with no file name, run `print("hello);`.	Error printed to screen, but prompt still available.	Pass
50	4.3	File mode.	Run executable with file name.	File is executed.	Pass

Test	Objectives	Description	Test data	Expected outcome	Pass/ fail
51	4.3.2	Invalid file path given.	Run executable with file name which doesn't exist.	Erorr is reported.	Pass
52	4.3.3	Errors end evaluation in file mode.	`t52.slang`	`t52.out`	Pass
53	4.3.4	Statistics written to CSV file.	Run the executable (in file mode).	A CSV file is created.	Pass
54	4.4	User can select mode.	Run the executable with a file name, and then run the executable with no file name.	In the first case, the file is interpreted. In the second case, a shell is created.	Pass

5. Evaluation

5.1. Overall effectiveness

I was able to successfully create an interpreter for a custom programming language, and implement multiple memory management methods. The programming language is general enough to solve many problems, and the memory management techniques have been implemented correctly.

5.2. Evaluation of objectives

Below are the objectives I decided on for my project (sub-objectives are omitted), and my analysis of how successfully each one was met.

Objective	Success
1	All objectives in this section were met successfully, however some were more difficult to complete than others. Objective 1.4 was particularly tedious to implement, as every possible combination of type and operation had to be handled manually due to Rust's strict type system. Objective 1.5.1.5 was interesting grammatically, as the exponent operator is right associative, but most operators were left-associative. Objective 1.8.4.2 was slightly challenging to implement. I had to redesign how I stored environments and stack frames in order to allow every environment to link to the global scope.
2	Again, all objectives were completely met in this section. There were some improvements that could be made, however. Objective 2.8.2 was implemented satisfactorily, however I could have added more advanced logic to figure out to a greater extent what the programmer was trying to do, and which mistake they made. Objective 2.11.2 was very challenging to implement, and required a lot of debugging to root out misconceptions I'd made about scope and object lifetimes.

Objective	Success
3	All objectives were successfully completed for this section.
4	All objectives were met in this section. Objective 4.2.1 was met, however the user input experience could have been made better: currently users are not able to use the backspace key, or the arrow keys, meaning that they must enter a line of code perfectly the first time round.

5.3. Memory management method analysis

At this point, I can answer the questions I set out in Section 1.1. They are repeated below for convenience.

1. How does choice of memory management technique affect the execution time of a program?
2. How does choice of memory management technique affect the amount of memory used by the interpreter?
3. How does choice of memory management technique affect the speed at which objects are cleaned up?

To answer these questions, I wrote the following program:

```
fu fib(n) {
    let temp = {tag: n};
    x = {tag: temp};

    if n <= 1 { return n; }
    else { return fib(n - 1) + fib(n - 2); }
}

let x = {};

print(fib( [HARDCODED VALUE] ));
```

This program creates calculates the nth Fibonacci number, while also creating and reassigning objects frequently. I repeatedly ran this program, swapping out `[HARDCODED VALUE]` for the numbers 0 to 25, and recording the amount of time the program took to run. I did this for each memory management method.

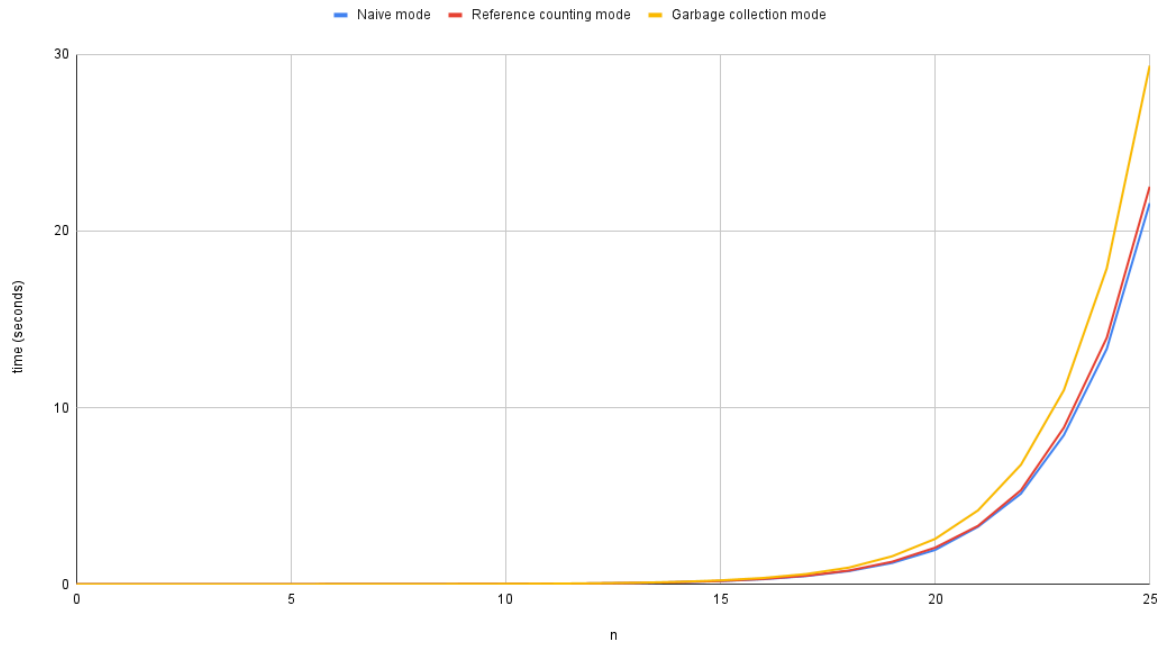


Figure 9: Execution time of `fib(n)` for different values of `n`.

As seen, naive allocation mode is slightly faster than reference counting, but only just. This means that reference counting performs very well–almost as well as if it had not been doing any memory management whatsoever. Garbage collection, however, is consistently slower than both other modes.

Taking a look into memory usage, for `fib(25)`, naive allocation mode allocated 306.5 megabytes, whereas reference counting and garbage collection allocated 75.08 and 75.09 megabytes respectively. This shows that using any memory management whatsoever leads to a drastic decrease in the amount of memory allocated.

Answering the third question required looking at the number of objects stored in the heap at any given time. The graphs below plot the number of objects in the heap against time. `fib(10)` has been called in each case.

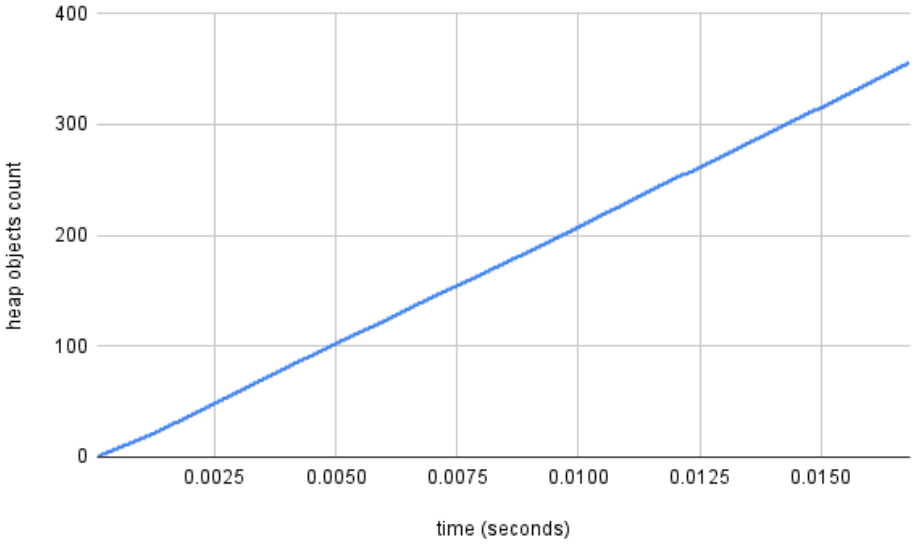


Figure 10: Heap objects count for `fib(10)` in naive allocation mode.

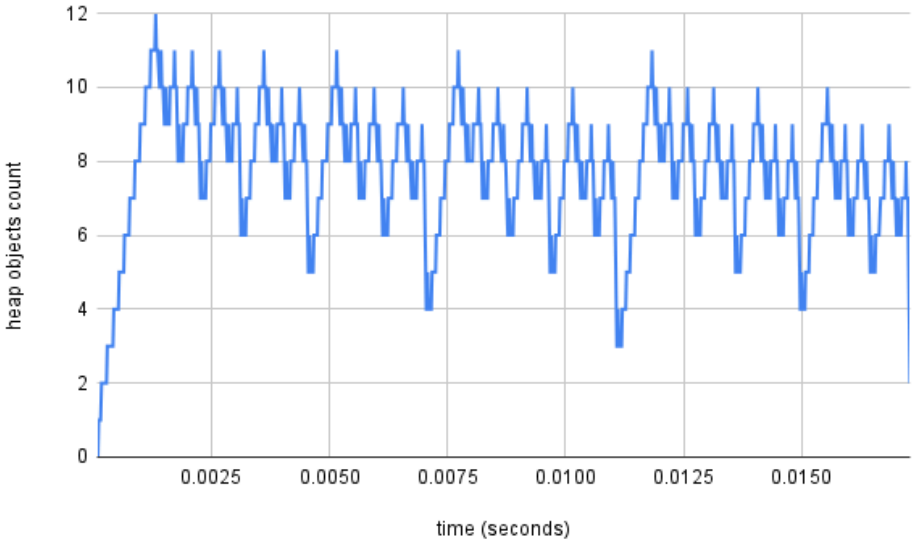


Figure 11: Heap objects count for `fib(10)` in reference counting mode.

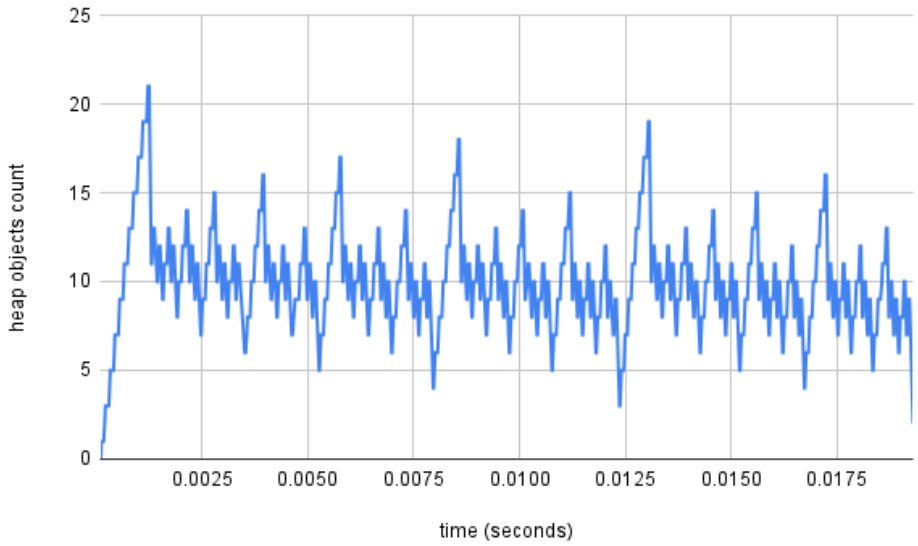


Figure 12: Heap objects count for `fib(10)` in garbage collection mode.

There are a few things to note here. As expected, when using naive allocation the heap grows unchecked, never deallocating any objects. this leads to over 300 objects being allocated, which are never deallocated.

Garbage collection and reference counting, however, are able to rapidly clean up objects when they are no longer in use. When using reference counting, a maximum of 12 objects are stored in the heap at any one moment, and this rises to 21 with garbage collection. Comparing the two graphs highlights the advantage that reference counting holds over garbage collection: objects are deallocated the moment that they are no longer needed.

Within the specific `fib` program I wrote, the disparity between the two methods arises due to the line `x = {tag: temp};`. This line mutates the value of `x`, overwriting its previously stored value with a new object. In reference counting mode, the previously stored object has its reference count immediately decremented, deallocating it instantaneously, but in garbage collection mode the garbage collector is not called until the scope is exited (i.e. when the function returns). This leads to the previous value of `x` existing in the heap for far longer than is necessary.

The situation where garbage collection excels is in programs where reference counts are introduced. For reasons explained in Section 1.4.2.2, reference counting is unable to clean up reference cycles. Naive allocation obviously doesn't even attempt to.

I wrote the following program to demonstrate this:

```
let i = 0;

while i < 100 {
    let a = {};
    let b = {};

    a.other = b;
    b.other = a;

    i = i + 1;
}
```

This repeats the following process 100 times: create two objects, and introduce a cycle by making both objects reference one another.

The graphs below plot the number of heap objects against time, for each memory management mode.

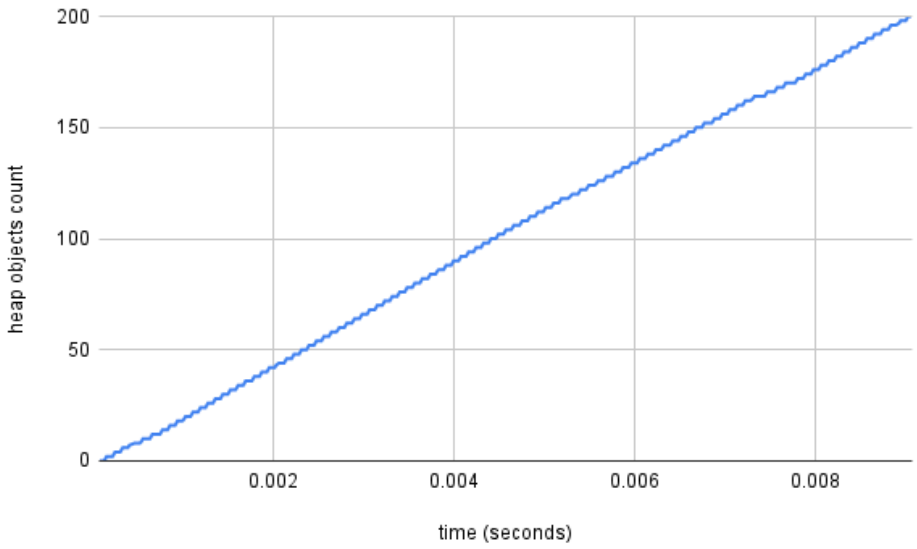


Figure 13: Heap objects count for reference cycling in naive allocation mode.

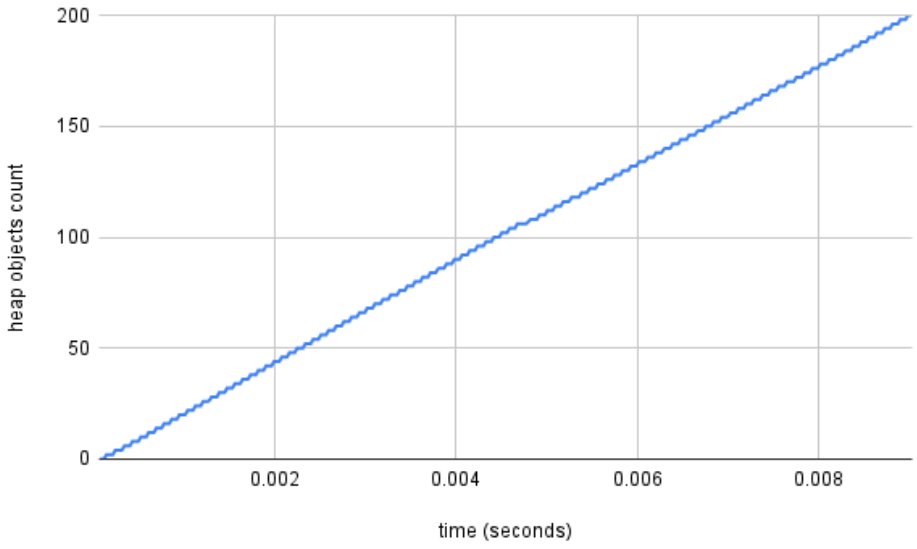


Figure 14: Heap objects count for reference cycling in reference counting mode.

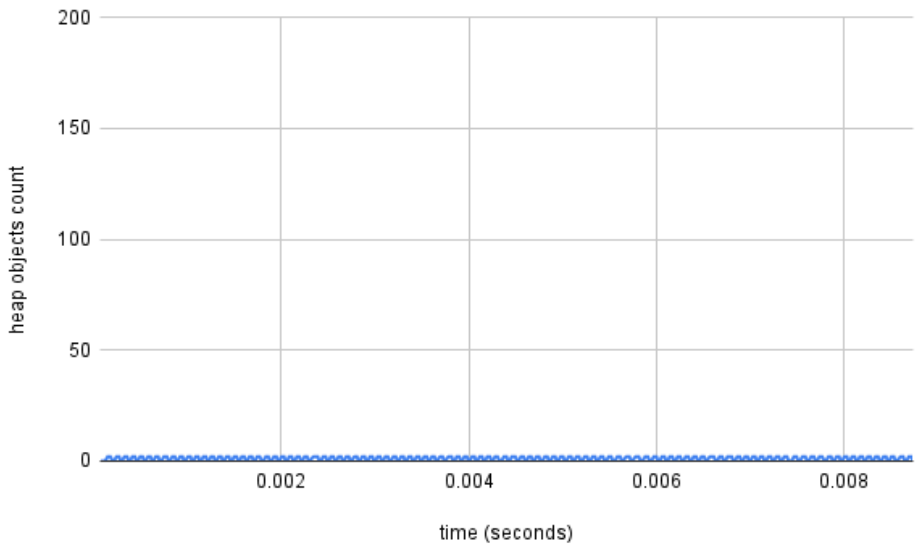


Figure 15: Heap objects count for reference cycling in garbage collection mode.

As seen, both reference counting and naive allocation do not clean up the reference cycles, leaving all objects allocated for the duration of the program even when they are obviously not needed anymore. Garbage collection is able to immediately clean up the reference cycles after each iteration, keeping the number of objects to 2 at a maximum.

5.4. Expert feedback

I emailed Dr. Sleat to ask for some feedback on my interpreter, and he responded with the following:

> "Sam
>
> Here is some feedback on Slang.
>
> Overall I am very impressed. Within a short period of time you have created language that is reasonably functional and stable. This is a great start to allow you to explore how a stack based run-time system works and also to explore features of a language and how those impact the runtime. To have functions, objects as well as iteration and control flow, including with recursion, gives you a good start for further developing Slang. Personally (as an ex–C programmer) I appreciated the use of `;` to delimit statements and `{}` to define scope. In my view this is more explicit than the indentation-based approach of languages such as Python.
>
> **Potential improvements to the language**
>
> I have the following suggestions for areas where you might like to improve the language.
>
> 1. Checking type of parameters to functions. There is no type declaration in the function definition and this can cause runtime errors if the wrong type is passed.
> 2. Beef up native functions to accept multiple parameters. This would be useful for `print()` and `format()` for example to avoid the coder having to either use lots of repeated `format()`s for multiple variable outputs. Indeed if print() allowed for multiple parameters (even if all string types) this would avoid having to use format().
> 3. Casting to strings is related to point 2.

4. Having the ability to change the type of a variable is fine for dev/test languages but can cause untold damage in real-life programming environments. I would stick with strong variable type definitions.
5. I would relax the rules around type interchangeability of integers and floats. You should be able to write `x = 5 + 8.3;`
6. Arrays and/or lists would be nice.
7. Some additional flexibility around iteration/condition e.g. break/continue.
8. How about including some built in functions that exposes the underlying GC state e.g. `memstats()` might return how much memory is currently allocated. This could be useful for debugging and understanding the GC abilities more thoroughly.

Review of garbage collection

I tried a couple of programs using recursion to assess the garbage collection. To be honest, I struggled to see how the `.csv` files that were generated added value to the programmer but, with further explanation and understanding on my part, I'm sure these would allow me to review the GC more accurately.

That being said, `slangtestA.txt` recursively calls a function to a depth decided by an input. I tested it to a depth of 414 before getting the error: `thread 'main' has overflowed its stack`. I'm not sure what the limits are here but 414 recursions didn't seem very many to me.

With `slangtestB.txt` I tried to create a cycle between two objects. This appeared to work but again, I'm not sure of the GC going on in the background. I made a couple of assumptions (a) that I can reference one object from the other - the language appeared to be 'loose' enough to allow me to do this where I assigned "a.pointertob = b" (b) that setting an object to 0 either calls some GC at that point or subsequently. I ran this to 10000 iterations without runtime failure. I tested this with all three runtime options `gc`, `rc` and `na`. No problems.

Sam…this is, in my opinion, a very good piece of work. Many thanks for sharing it with me and please do shout if I can be of any further help.

Keep in touch.

Kind regards

Doc"

He attached the following `.txt` files:

slangtestA.txt

```
// ------------------------------------------------------------
// TEST A: Recursive allocation of temporary objects
// What it does:
//   - Recursively goes down to 0
//   - On each call, allocates a small object { n: depth }
//   - When the function returns, those objects should become unreachable
// ------------------------------------------------------------

fu allocDown(depth) {
  if depth <= 0 {
    return 0;
  }

  // Allocate a brand-new object on each recursive call
```

```
  let temp = { n: depth, tag: "temp" };

  // Make a tiny use of it - don't want any GC to collect unused objects
  if temp.n == -1 {
    print("Impossible");
  }

  // Don't like this, perhaps improve with more parameters to print statement and
intelligent casting?
  let op = format("Recursing...", depth);
  print(op);

  // Recurse
  return allocDown(depth - 1);
}

let D = 0;
D = int(input("Depth ?"));

print("Starting allocDown...");
allocDown(D);
print("Finished allocDown.");
```

slangtestB.txt

```
// ------------------------------------------------------------
// TEST B: Create a tiny cycle and drop it in a loop
// What it does:
//   - Creates two objects that point at each other (a cycle)
//   - Drops the outside reference
//   - Repeats many times to see whether memory grows
// ------------------------------------------------------------

fu makeCycle() {
  let a = { name: "A" };
  let b = { name: "B" };

  // Create a cycle: A <-> B
  a.pointertob = b;
  b.pointertoa = a;

  return a;   // caller holds the only outside reference
}

let i = 0;
let tick = 1000;

print("Starting cycle test...");
while i < 100000 {
  let x = makeCycle();

  // Drop the last outside reference: the cycle is now "floating"
  x = 0;

  // Print progress every 1000 iterations
  tick = tick - 1;
  if tick <= 0 {
    print(format("Made and dropped cycles: ", i));
```

```
    tick = 1000;
  }

  i = i + 1;
}
print("Finished cycle test.");
```

The feedback that Dr. Sleat provided was really helpful. Going point-by-point through his suggestions:

1. I would love to introduce a more robust type system, where the programmer can specify types of function arguments, function return types, variables etc. but it unfortunately wasn't feasible for the amount of time I had to write the interpreter.
2. This would be a very quick fix to implement for the `print` function.
3. Currently the programmer can cast any data type to a string via use of the `format` function, but this can be unwieldy. Adapting the type system for easier string concatenation and casting could be a solution.
4. Again, I would have wanted to construct a static type system if I'd had the time.
5. I'm not sure that I agree with Dr. Sleat on this point. It can be a bit frustrating having to cast an integer to a float before performing a floating point operation, but I think that this is justified by clarity over what type will be returned from the operation.
6. I definitely agree with introducing some more complex data structures, such as arrays and/or lists. This would allow many more programs to be written.
7. Again, adding additional statements such as `break` would be massively helpful.
8. Unfortunately this functionality does already exist, I just forgot to tell this to Dr. Sleat in my email to him. The programmer can access statistics about the program execution through global variables such as `HEAP_OBJECTS_COUNT` and `STACK_FRAMES_COUNT`.

The `slangtestA.txt` program was interesting, as Dr. Sleat discovered a recursion limit. This recursion limit was not implemented by me, and seems to be as a result of the operating system, or runtime. On my machine, I was able to reach 305 recursions, less than his 414.

Dr. Sleat wasn't able to analyse the `.csv` files to extract any useful information, so I ran `slangtestB.txt` and took a look through the resulting `.csv` file. It confirmed to me that garbage collection was indeed working correctly, with the number of objects stored on the heap flicking back and forth between 0 and 2.

5.5. System improvements

5.5.1. Improving the type system

Currently, Slang is strongly typed (meaning that operations are type-checked before being performed) but dynamically typed, meaning that variables can change type throughout the course of the program, and functions do not have defined return or argument types. This leads to a lot of flexibility when writing code, which can be useful, but often leads to type errors occurring.

If I had the time, I would have liked to make Slang statically typed. The concept of static typing only really exists within compiled languages, because type checking occurs at compile-time in statically typed languages. Therefore, I would have to first write a compiler for Slang. This would be a very large project in of itself, and would likely involve interfacing with LLVM.

After writing a compiler for Slang, I would update the grammar and lexical grammar to ensure that variable definitions and function definitions have type annotations. From here, the process of checking whether a program is valid would be very similar to the process of executing the abstract syntax tree, but with no actual data values. By performing a post-order traversal of the tree, the

types of the operands to each operation are checked. Any type errors are reported to the user, and the source code is not compiled if errors occur.

5.5.2. More advanced data structures

One of the most useful features that I could introduce would be a more advanced data structure such as an array or a list. This would allow many new types of languages to be written much more easily.

Firstly, I would add the required tokens, and write the lexer code to detect them. For list indexing, as well as list construction, I would likely use the left and right square brackets.

Next, I would need to add two new types of expression to the parser: the list indexing operation, and the list construction operation. List construction would be represented by the following new grammar rule:

```
list -> "[" (expression ("," expression)*)? "]"
```

An additional production would then be added to `primary`, which would be `| list`.

For list indexing, I would need to update the `call` grammar rule to the following:

```
call -> primary (
    "(" (expression ("," expression)*)? ")"
    | "." IDENTIFIER
    | "[" expression "]"
  )*
```

The parser would be updated to reflect these rules.

Next, I'd need to add new variants to `Expression` and `Value` to represent lists and their operations.

After this, the various operations (concatenation, appending, indexing etc.) would be implemented on the new list type.

5.5.3. Improving iteration constructs

I would like to add additional functionality for iteration, including `for`-loops, `break`-statements and `continue`-statements.

To implement this, I would introduce the following keywords: `for`, `break` and `continue`, and write the lexer code to recognise them.

In the parser, I would add two new types of statement:

```
breakStatement -> "break" ";"
continueStatement -> "continue" ";"
```

For-loops are slightly more complicated. The grammar rule would look as follows:

```
forLoop -> "for" "(" variableDeclaration ";" expression ";" expression ")" block
```

I'm using the syntax that is used in C#, JavaScript and many other C-like languages.

Three new productions would then be added to `statement`, which would be `| breakStatement | continueStatement | forLoop`.

For each of these rules, the parser would be adapted to be able to parse them.

After, new `Statement` variants would be created, and the code to execute them written. `break`- and `continue`-statements simply alter the `ControlFlow` signal that is emitted from a statement execution. A `for`-loop initially runs its `variableDeclaration`, and continues to execute its `block` as long as the first `expression` evaluates to true, evaluating the second `expression` at the end of each loop.

5.5.4. Closures

I would like to make Slang functions truly first class. Currently, functions can be returned from other functions, passed as arguments to functions, assigned to variables, and used in expressions. However, functions are unable to *close* around environments. This is best illustrated with the following example:

```
fu get_adder(a) {
  fu add(x) {
    return x + a;
  }

  return add;
}
```

Here, `get_adder` is a higher-order function that returns a *new* function, whose job is to add `a` to its argument. It could be used as follows:

```
let add_5 = get_adder(5);

print(add_5(10));
```

Here, a function called `add_5` is created, which adds `5` to its argument. `15` is printed to the screen.

This code will not currently run in my interpreter, as when `add_5(10)` is ran, the line `return x + a;` is ran. `x` is replaced with `10`, but `a` is not defined within that scope. This is because the scope within `get_adder` has been lost.

To fix this, I would need to attach the environment that the function was defined in to the definition of the function, in order to access any variables that could have been used in that function.

A Prototype source code

The prototype has the following directory structure.

```
.
├── Cargo.lock
├── Cargo.toml
└── src
    ├── expression.rs
    ├── lexer.rs
    ├── main.rs
    ├── parser.rs
    ├── source.rs
    ├── token.rs
    └── token_stream.rs

1 directory, 9 files
```

A.1 `Cargo.lock`

```
# This file is automatically @generated by Cargo.
# It is not intended for manual editing.
version = 4

[[package]]
name = "slang_prototype"
version = "0.1.0"
```

A.2 `Cargo.toml`

```
[package]
name = "slang_prototype"
version = "0.1.0"
edition = "2024"

[dependencies]
```

A.3 `src/expression.rs`

```
use std::fmt::Display;

pub enum EvaluationError {
    InvalidBinaryTypes {
        left: SlangType,
        operator: BinaryOperator,
        right: SlangType,
    },
    InvalidUnaryType {
        operator: UnaryOperator,
        operand: SlangType,
    },
    NonBooleanTernaryCondition {
        condition: SlangType,
    },
    DivisionByZero,
}

impl Display for EvaluationError {
    fn fmt(&self, f: &mut std::fmt::Formatter<'_>) -> std::fmt::Result {
        match self {
```

```
            Self::InvalidBinaryTypes {
                left,
                operator,
                right,
            } => write!(
                f,
                "[evaluation error] The `{}` operator is not defined for {} and {}.",
                operator.raw(),
                left,
                right
            ),
            Self::InvalidUnaryType { operator, operand } => write!(
                f,
                "[evaluation error] The unary {} operator is not defined for {}.",
                operator.raw(),
                operand
            ),
            Self::NonBooleanTernaryCondition { condition } => write!(
                f,
                "[evaluation error] Expected Boolean operand for ternary condition,
found {}.",
                condition
            ),
            Self::DivisionByZero => {
                write!(f, "[evaluation error] Division by zero.")
            }
        }
    }
}

#[derive(Debug)]
pub enum Expression {
    Ternary {
        condition: Box<Expression>,
        left: Box<Expression>,
        right: Box<Expression>,
    },
    Binary {
        left: Box<Expression>,
        operator: BinaryOperator,
        right: Box<Expression>,
    },
    Unary {
        operator: UnaryOperator,
        operand: Box<Expression>,
    },
    Grouping(Box<Expression>),
    Literal(Literal),
}

impl Expression {
    pub fn evaluate(self) -> Result<Literal, EvaluationError> {
        match self {
            Self::Ternary {
                condition,
                left,
```

```
                right,
            } => Expression::evaluate_ternary(condition, left, right),

            Self::Binary {
                left,
                operator,
                right,
            } => Expression::evaluate_binary(left, operator, right),

            Self::Unary { operator, operand } => Expression::evaluate_unary(operator,
operand),

            Self::Grouping(expression) => expression.evaluate(),

            Self::Literal(literal) => Ok(literal),
        }
    }

    fn evaluate_ternary(
        condition: Box<Expression>,
        left: Box<Expression>,
        right: Box<Expression>,
    ) -> Result<Literal, EvaluationError> {
        let condition = condition.evaluate()?;

        if let Literal::Boolean(condition) = condition {
            if condition {
                return left.evaluate();
            } else {
                return right.evaluate();
            }
        } else {
            return Err(EvaluationError::NonBooleanTernaryCondition {
                condition: condition.slang_type(),
            });
        }
    }

    fn evaluate_binary(
        left: Box<Expression>,
        operator: BinaryOperator,
        right: Box<Expression>,
    ) -> Result<Literal, EvaluationError> {
        let operands = (left.evaluate()?, right.evaluate()?);

        Ok(match operator {
            BinaryOperator::Add => match operands {
                (Literal::Integer(left), Literal::Integer(right)) =>
Literal::Integer(left + right),
                (Literal::Float(left), Literal::Float(right)) => Literal::Float(left
+ right),
                (left, right) => Err(EvaluationError::InvalidBinaryTypes {
                    left: left.slang_type(),
                    operator: operator,
                    right: right.slang_type(),
                })?,
```

```
            },

            BinaryOperator::Subtract => match operands {
                (Literal::Integer(left), Literal::Integer(right)) =>
Literal::Integer(left - right),
                (Literal::Float(left), Literal::Float(right)) => Literal::Float(left
- right),
                (left, right) => Err(EvaluationError::InvalidBinaryTypes {
                    left: left.slang_type(),
                    operator: operator,
                    right: right.slang_type(),
                })?,
            },

            BinaryOperator::Multiply => match operands {
                (Literal::Integer(left), Literal::Integer(right)) =>
Literal::Integer(left * right),
                (Literal::Float(left), Literal::Float(right)) => Literal::Float(left
* right),
                (left, right) => Err(EvaluationError::InvalidBinaryTypes {
                    left: left.slang_type(),
                    operator: operator,
                    right: right.slang_type(),
                })?,
            },

            BinaryOperator::Divide => match operands {
                (Literal::Integer(left), Literal::Integer(right)) => {
                    if right == 0 {
                        return Err(EvaluationError::DivisionByZero);
                    }

                    Literal::Integer(left / right)
                }
                (Literal::Float(left), Literal::Float(right)) => {
                    if right == 0.0 {
                        return Err(EvaluationError::DivisionByZero);
                    }

                    Literal::Float(left / right)
                }
                (left, right) => Err(EvaluationError::InvalidBinaryTypes {
                    left: left.slang_type(),
                    operator: operator,
                    right: right.slang_type(),
                })?,
            },

            BinaryOperator::EqualTo => match operands {
                (Literal::Integer(left), Literal::Integer(right)) => {
                    Literal::Boolean(left == right)
                }
                (Literal::Float(left), Literal::Float(right)) =>
Literal::Boolean(left == right),
                (Literal::Boolean(left), Literal::Boolean(right)) => {
                    Literal::Boolean(left == right)
```

```
                }
                (left, right) => Err(EvaluationError::InvalidBinaryTypes {
                    left: left.slang_type(),
                    operator: operator,
                    right: right.slang_type(),
                })?,
            },

            BinaryOperator::NotEqualTo => match operands {
                (Literal::Integer(left), Literal::Integer(right)) => {
                    Literal::Boolean(left != right)
                }
                (Literal::Float(left), Literal::Float(right)) =>
Literal::Boolean(left != right),
                (Literal::Boolean(left), Literal::Boolean(right)) => {
                    Literal::Boolean(left != right)
                }
                (left, right) => Err(EvaluationError::InvalidBinaryTypes {
                    left: left.slang_type(),
                    operator: operator,
                    right: right.slang_type(),
                })?,
            },

            BinaryOperator::GreaterThan => match operands {
                (Literal::Integer(left), Literal::Integer(right)) =>
Literal::Boolean(left > right),
                (Literal::Float(left), Literal::Float(right)) =>
Literal::Boolean(left > right),
                (left, right) => Err(EvaluationError::InvalidBinaryTypes {
                    left: left.slang_type(),
                    operator: operator,
                    right: right.slang_type(),
                })?,
            },

            BinaryOperator::GreaterThanOrEqualTo => match operands {
                (Literal::Integer(left), Literal::Integer(right)) => {
                    Literal::Boolean(left >= right)
                }
                (Literal::Float(left), Literal::Float(right)) =>
Literal::Boolean(left >= right),
                (left, right) => Err(EvaluationError::InvalidBinaryTypes {
                    left: left.slang_type(),
                    operator: operator,
                    right: right.slang_type(),
                })?,
            },

            BinaryOperator::LessThan => match operands {
                (Literal::Integer(left), Literal::Integer(right)) =>
Literal::Boolean(left < right),
                (Literal::Float(left), Literal::Float(right)) =>
Literal::Boolean(left < right),
                (left, right) => Err(EvaluationError::InvalidBinaryTypes {
                    left: left.slang_type(),
```

```
                    operator: operator,
                    right: right.slang_type(),
                })?,
            },

            BinaryOperator::LessThanOrEqualTo => match operands {
                (Literal::Integer(left), Literal::Integer(right)) => {
                    Literal::Boolean(left <= right)
                }
                (Literal::Float(left), Literal::Float(right)) =>
Literal::Boolean(left <= right),
                (left, right) => Err(EvaluationError::InvalidBinaryTypes {
                    left: left.slang_type(),
                    operator: operator,
                    right: right.slang_type(),
                })?,
            },

            BinaryOperator::AND => match operands {
                (Literal::Boolean(left), Literal::Boolean(right)) => {
                    Literal::Boolean(left && right)
                }
                (left, right) => Err(EvaluationError::InvalidBinaryTypes {
                    left: left.slang_type(),
                    operator: operator,
                    right: right.slang_type(),
                })?,
            },

            BinaryOperator::OR => match operands {
                (Literal::Boolean(left), Literal::Boolean(right)) => {
                    Literal::Boolean(left || right)
                }
                (left, right) => Err(EvaluationError::InvalidBinaryTypes {
                    left: left.slang_type(),
                    operator: operator,
                    right: right.slang_type(),
                })?,
            },

            BinaryOperator::BitwiseAND => match operands {
                (Literal::Integer(left), Literal::Integer(right)) =>
Literal::Integer(left & right),
                (Literal::Boolean(left), Literal::Boolean(right)) =>
Literal::Boolean(left & right),
                (left, right) => Err(EvaluationError::InvalidBinaryTypes {
                    left: left.slang_type(),
                    operator: operator,
                    right: right.slang_type(),
                })?,
            },

            BinaryOperator::BitwiseOR => match operands {
                (Literal::Integer(left), Literal::Integer(right)) =>
Literal::Integer(left | right),
                (Literal::Boolean(left), Literal::Boolean(right)) =>
```

```
Literal::Boolean(left | right),
                (left, right) => Err(EvaluationError::InvalidBinaryTypes {
                    left: left.slang_type(),
                    operator: operator,
                    right: right.slang_type(),
                })?,
            },
        })
    }

    fn evaluate_unary(
        operator: UnaryOperator,
        operand: Box<Expression>,
    ) -> Result<Literal, EvaluationError> {
        let operand = operand.evaluate()?;

        Ok(match operator {
            UnaryOperator::Minus => match operand {
                Literal::Integer(operand) => Literal::Integer(-operand),
                Literal::Float(operand) => Literal::Float(-operand),
                _ => Err(EvaluationError::InvalidUnaryType {
                    operator,
                    operand: operand.slang_type(),
                })?,
            },
            UnaryOperator::NOT => match operand {
                Literal::Integer(operand) => Literal::Integer(!operand),
                Literal::Boolean(operand) => Literal::Boolean(!operand),
                _ => Err(EvaluationError::InvalidUnaryType {
                    operator,
                    operand: operand.slang_type(),
                })?,
            },
        })
    }
}

#[derive(Debug)]
pub enum Literal {
    Float(f64),
    Integer(i32),
    Boolean(bool),
}

impl Literal {
    pub fn slang_type(&self) -> SlangType {
        match self {
            Self::Float(_) => SlangType::Float,
            Self::Integer(_) => SlangType::Integer,
            Self::Boolean(_) => SlangType::Boolean,
        }
    }
}

#[derive(Debug)]
pub enum SlangType {
```

```
    Float,
    Integer,
    Boolean,
}

impl Display for SlangType {
    fn fmt(&self, f: &mut std::fmt::Formatter<'_>) -> std::fmt::Result {
        match self {
            Self::Float => write!(f, "Float"),
            Self::Integer => write!(f, "Integer"),
            Self::Boolean => write!(f, "Boolean"),
        }
    }
}

#[derive(Debug, PartialEq, Eq)]
pub enum BinaryOperator {
    // Arithmetic operators
    Add,
    Subtract,
    Multiply,
    Divide,

    // Logical operators
    EqualTo,
    NotEqualTo,
    GreaterThan,
    GreaterThanOrEqualTo,
    LessThan,
    LessThanOrEqualTo,
    AND,
    OR,

    // Bitwise operators
    BitwiseAND,
    BitwiseOR,
}

impl BinaryOperator {
    pub fn raw(&self) -> String {
        match self {
            Self::Add => "+",
            Self::Subtract => "-",
            Self::Multiply => "*",
            Self::Divide => "/",

            Self::EqualTo => "==",
            Self::NotEqualTo => "!=",
            Self::GreaterThan => ">",
            Self::GreaterThanOrEqualTo => ">=",
            Self::LessThan => "<",
            Self::LessThanOrEqualTo => "<=",
            Self::AND => "&&",
            Self::OR => "||",

            Self::BitwiseAND => "&",
```

```
            Self::BitwiseOR => "|",
        }
        .to_string()
    }
}

#[derive(Debug, PartialEq, Eq)]
pub enum UnaryOperator {
    Minus,
    NOT,
}

impl UnaryOperator {
    pub fn raw(&self) -> String {
        match self {
            Self::Minus => "-",
            Self::NOT => "!",
        }
        .to_string()
    }
}
```

A.4 src/lexer.rs

```
use std::{
    error::Error,
    fmt::{Debug, Display},
};

use crate::{
    source::{GeneralLocation, Location, Source},
    token::{Token, TokenData},
};

pub enum LexerError {
    UnexpectedCharacter {
        location: Location,
        character: char,
        expected: Option<char>,
    },
    UnexpectedEndOfFile {
        expected: char,
    },
    UnknownKeyword {
        location: Location,
        keyword: String,
    },
}

impl Display for LexerError {
    fn fmt(&self, f: &mut std::fmt::Formatter<'_>) -> std::fmt::Result {
        match self {
            Self::UnexpectedCharacter {
                location,
                character,
                expected,
            } => write!(
```

```
                f,
                "{} Unexpected character: `{}`{}",
                location,
                character,
                match expected {
                    Some(expected) => format!(" (expected `{}`)", expected),
                    None => String::new(),
                }
            ),
            Self::UnexpectedEndOfFile { expected } => {
                write!(
                    f,
                    "{} Reached end of file, but expected `{}`",
                    GeneralLocation::EndOfFile,
                    expected
                )
            }
            Self::UnknownKeyword { location, keyword } => {
                write!(f, "{} Unexpected keyword: `{}`", location, keyword)
            }
        }
    }
}

impl Debug for LexerError {
    fn fmt(&self, f: &mut std::fmt::Formatter<'_>) -> std::fmt::Result {
        write!(f, "{}", self)
    }
}

impl Error for LexerError {}

pub struct Lexer {
    source: Source,
    tokens: Vec<Token>,
    current_token_start: Location,
}

impl Lexer {
    pub fn new(source: Source) -> Self {
        Self {
            source: source,
            tokens: Vec::new(),
            current_token_start: Location::start(),
        }
    }

    pub fn lex(mut self) -> (Vec<Token>, Vec<LexerError>) {
        let mut errors = Vec::new();

        while let Some(character) = self.source.advance() {
            let result = match character {
                '(' => Ok(self.add_token(TokenData::LeftParenthesis)),
                ')' => Ok(self.add_token(TokenData::RightParenthesis)),
                '?' => Ok(self.add_token(TokenData::QuestionMark)),
                ':' => Ok(self.add_token(TokenData::Colon)),
```

```
                // Arithmetic operators
                '+' => Ok(self.add_token(TokenData::Plus)),
                '-' => Ok(self.add_token(TokenData::Minus)),
                '*' => Ok(self.add_token(TokenData::Star)),
                '/' => Ok(self.add_token(TokenData::Slash)),

                // Logical and bitwise operators
                '!' => Ok(self.handle_bang()),
                '=' => self.handle_equal(),
                '>' => Ok(self.handle_greater()),
                '<' => Ok(self.handle_less()),
                '&' => Ok(self.handle_ampersand()),
                '|' => Ok(self.handle_pipe()),

                character if character.is_ascii_digit() =>
Ok(self.handle_number(character)),

                // Identifiers and keywords
                character if character.is_ascii_alphabetic() || character == '_' => {
                    self.handle_word(character)
                }

                // Whitespace
                ' ' | '\r' | '\t' | '\n' => Ok(()),

                // Unexpected characters
                _ => Err(LexerError::UnexpectedCharacter {
                    location: self.current_token_start,
                    character: character,
                    expected: None,
                }),
            };

            if let Err(error) = result {
                errors.push(error);
            }

            self.current_token_start = self.source.location();
        }

        (self.tokens, errors)
    }

    fn add_token(&mut self, data: TokenData) {
        self.tokens.push(Token::new(data, self.current_token_start));
    }

    fn handle_bang(&mut self) {
        if self.source.matches('=') {
            self.add_token(TokenData::BangEqual);
        } else {
            self.add_token(TokenData::Bang);
        }
    }
```

```
fn handle_equal(&mut self) -> Result<(), LexerError> {
    if let Some(character) = self.source.peek() {
        if character == '=' {
            self.source.advance();
            Ok(self.add_token(TokenData::DoubleEqual))
        } else {
            Err(LexerError::UnexpectedCharacter {
                location: self.current_token_start,
                character,
                expected: Some('='),
            })
        }
    } else {
        Err(LexerError::UnexpectedEndOfFile { expected: '=' })
    }
}

fn handle_less(&mut self) {
    if self.source.matches('=') {
        self.add_token(TokenData::LessEqual);
    } else {
        self.add_token(TokenData::Less);
    }
}

fn handle_greater(&mut self) {
    if self.source.matches('=') {
        self.add_token(TokenData::GreaterEqual);
    } else {
        self.add_token(TokenData::Greater);
    }
}

fn handle_ampersand(&mut self) {
    if self.source.matches('&') {
        self.add_token(TokenData::DoubleAmpersand);
    } else {
        self.add_token(TokenData::Ampersand);
    }
}

fn handle_pipe(&mut self) {
    if self.source.matches('|') {
        self.add_token(TokenData::DoublePipe);
    } else {
        self.add_token(TokenData::Pipe);
    }
}

fn handle_number(&mut self, first_digit: char) {
    let mut number = String::new();

    number.push(first_digit);

    while let Some(character) = self.source.peek() {
        if !character.is_ascii_digit() {
```

```
                break;
            }

            number.push(character);
            self.source.advance();
        }

        if self.source.peek().is_some_and(|character| character == '.')
            && self
                .source
                .peek_after()
                .is_some_and(|character| character.is_ascii_digit())
        {
            number.push('.');
            self.source.advance();

            while let Some(character) = self.source.peek() {
                if !character.is_ascii_digit() {
                    break;
                }

                number.push(character);
                self.source.advance();
            }

            let number: f64 = number.parse().unwrap();

            self.add_token(TokenData::Float(number))
        } else {
            let number: i32 = number.parse().unwrap();

            self.add_token(TokenData::Integer(number));
        }
    }

    fn handle_word(&mut self, first_character: char) -> Result<(), LexerError> {
        let mut word = String::new();

        word.push(first_character);

        while let Some(character) = self.source.peek() {
            if character.is_ascii_alphanumeric() || character == '_' {
                word.push(character);
                self.source.advance();
            } else {
                break;
            }
        }

        match word.as_str() {
            // Literals
            "true" => self.add_token(TokenData::Boolean(true)),
            "false" => self.add_token(TokenData::Boolean(false)),

            _ => Err(LexerError::UnknownKeyword {
                location: self.current_token_start,
```

```
                keyword: word,
            })?,
        };

        Ok(())
    }
}
```

A.5 src/main.rs

```
use lexer::Lexer;
use parser::Parser;
use source::Source;
use token_stream::TokenStream;

mod expression;
mod lexer;
mod parser;
mod source;
mod token;
mod token_stream;

use std::{
    env,
    io::{self, BufRead, Write},
};

fn main() {
    // The first argument is always the executable path (e.g. /bin/slang)
    let args = env::args().skip(1);

    if args.len() == 0 {
        run_prompt();
    } else {
        println!("No arguments expected.");
    }
}

fn run_prompt() {
    let mut line = String::new();

    let mut stdin = io::stdin().lock();
    let mut stdout = io::stdout().lock();

    loop {
        line.clear();

        print!("\n> ");
        let _ = stdout.flush();
        let _ = stdin.read_line(&mut line);

        run(line.trim());
    }
}

fn run(source: &str) {
    let source = Source::new(source);
```

```
    let lexer = Lexer::new(source);

    let (tokens, errors) = lexer.lex();

    for error in &errors {
        eprintln!("{:?}", error);
    }

    if errors.len() != 0 {
        return;
    }

    let tokens = TokenStream::new(tokens);

    let parser = Parser::new(tokens);

    match parser.parse() {
        Ok(expression) => match expression.evaluate() {
            Ok(literal) => println!("{:?}", literal),
            Err(error) => println!("{}", error),
        },
        Err(error) => {
            println!("{}", error)
        }
    }
}
```

A.6 src/parser.rs

```
use std::{
    error::Error,
    fmt::{Debug, Display},
};

use crate::{
    expression::{BinaryOperator, Expression, Literal, UnaryOperator},
    source::GeneralLocation,
    token::{TokenData, TokenKind},
    token_stream::TokenStream,
};

pub enum ParserError {
    ExpectedToken {
        expected: Vec<TokenKind>,
        location: GeneralLocation,
    },
    UnsupportedUnaryExpression {
        operator: BinaryOperator,
        location: GeneralLocation,
    },
}

impl Display for ParserError {
    fn fmt(&self, f: &mut std::fmt::Formatter<'_>) -> std::fmt::Result {
        match self {
            Self::ExpectedToken { expected, location } => {
```

```
                write!(
                    f,
                    "{} Expected one of the following tokens: {:?}",
                    location, expected
                )
            }
            Self::UnsupportedUnaryExpression { operator, location } => {
                write!(
                    f,
                    "{} The unary `{}` operator is not supported.",
                    location,
                    operator.raw(),
                )
            }
        }
    }
}

impl Debug for ParserError {
    fn fmt(&self, f: &mut std::fmt::Formatter<'_>) -> std::fmt::Result {
        write!(f, "{}", self)
    }
}

impl Error for ParserError {}

pub struct Parser {
    tokens: TokenStream,
}

impl Parser {
    pub fn new(tokens: TokenStream) -> Self {
        Self { tokens }
    }

    pub fn parse(mut self) -> Result<Expression, ParserError> {
        self.expression()
    }

    fn expression(&mut self) -> Result<Expression, ParserError> {
        self.ternary()
    }

    fn ternary(&mut self) -> Result<Expression, ParserError> {
        let mut expression = self.logical()?;

        if self.tokens.matches(&[TokenKind::QuestionMark]).is_some() {
            let left = self.logical()?;

            self.tokens.consume(TokenKind::Colon)?;

            let right = self.logical()?;

            expression = Expression::Ternary {
                condition: Box::new(expression),
                left: Box::new(left),
```

```
                right: Box::new(right),
            }
        }

        Ok(expression)
    }

    fn logical(&mut self) -> Result<Expression, ParserError> {
        let mut expression = self.equality()?;

        while let Some((operator, _)) = self
            .tokens
            .binary_operator(&[BinaryOperator::AND, BinaryOperator::OR])
        {
            expression = Expression::Binary {
                left: Box::new(expression),
                operator: operator,
                right: Box::new(self.equality()?),
            }
        }

        Ok(expression)
    }

    fn equality(&mut self) -> Result<Expression, ParserError> {
        let mut expression = self.comparison()?;

        while let Some((operator, _)) = self
            .tokens
            .binary_operator(&[BinaryOperator::NotEqualTo, BinaryOperator::EqualTo])
        {
            expression = Expression::Binary {
                left: Box::new(expression),
                operator,
                right: Box::new(self.comparison()?),
            }
        }

        Ok(expression)
    }

    fn comparison(&mut self) -> Result<Expression, ParserError> {
        let mut expression = self.bitwise()?;

        while let Some((operator, _)) = self.tokens.binary_operator(&[
            BinaryOperator::GreaterThan,
            BinaryOperator::GreaterThanOrEqualTo,
            BinaryOperator::LessThan,
            BinaryOperator::LessThanOrEqualTo,
        ]) {
            expression = Expression::Binary {
                left: Box::new(expression),
                operator,
                right: Box::new(self.bitwise()?),
            }
        }
```

```
        Ok(expression)
    }

    fn bitwise(&mut self) -> Result<Expression, ParserError> {
        let mut expression = self.term()?;

        while let Some((operator, _)) = self
            .tokens
            .binary_operator(&[BinaryOperator::BitwiseAND,
BinaryOperator::BitwiseOR])
        {
            expression = Expression::Binary {
                left: Box::new(expression),
                operator,
                right: Box::new(self.term()?),
            }
        }

        Ok(expression)
    }

    fn term(&mut self) -> Result<Expression, ParserError> {
        let mut expression = self.factor()?;

        while let Some((operator, _)) = self
            .tokens
            .binary_operator(&[BinaryOperator::Add, BinaryOperator::Subtract])
        {
            expression = Expression::Binary {
                left: Box::new(expression),
                operator,
                right: Box::new(self.factor()?),
            }
        }

        Ok(expression)
    }

    fn factor(&mut self) -> Result<Expression, ParserError> {
        let mut expression = self.unary()?;

        while let Some((operator, _)) = self
            .tokens
            .binary_operator(&[BinaryOperator::Multiply, BinaryOperator::Divide])
        {
            expression = Expression::Binary {
                left: Box::new(expression),
                operator,
                right: Box::new(self.unary()?),
            }
        }

        Ok(expression)
    }
```

```
fn unary(&mut self) -> Result<Expression, ParserError> {
    if let Some((operator, _)) = self
        .tokens
        .unary_operator(&[UnaryOperator::Minus, UnaryOperator::NOT])
    {
        Ok(Expression::Unary {
            operator: operator,
            operand: Box::new(self.primary()?),
        })
    } else if let Some((operator, location)) = self.tokens.binary_operator(&[
        BinaryOperator::Add,
        BinaryOperator::Multiply,
        BinaryOperator::Divide,
        BinaryOperator::NotEqualTo,
        BinaryOperator::EqualTo,
        BinaryOperator::GreaterThan,
        BinaryOperator::GreaterThanOrEqualTo,
        BinaryOperator::LessThan,
        BinaryOperator::LessThanOrEqualTo,
        BinaryOperator::BitwiseAND,
        BinaryOperator::BitwiseOR,
    ]) {
        let _ = self.primary();

        Err(ParserError::UnsupportedUnaryExpression {
            location: GeneralLocation::Location(location),
            operator: operator,
        })
    } else {
        self.primary()
    }
}

fn primary(&mut self) -> Result<Expression, ParserError> {
    let expected = [
        TokenKind::LeftParenthesis,
        TokenKind::Float,
        TokenKind::Integer,
        TokenKind::Boolean,
    ];

    if let Some(token) = self.tokens.matches(&expected) {
        Ok(Expression::Literal(match token.data() {
            TokenData::LeftParenthesis => {
                let expression = self.expression()?;

                self.tokens.consume(TokenKind::RightParenthesis)?;

                return Ok(Expression::Grouping(Box::new(expression)));
            }

            TokenData::Float(float) => Literal::Float(float),

            TokenData::Integer(integer) => Literal::Integer(integer),

            TokenData::Boolean(boolean) => Literal::Boolean(boolean),
```

```
                _ => unreachable!(),
            }))
        } else if let Some(token) = self.tokens.peek() {
            Err(ParserError::ExpectedToken {
                expected: expected.to_vec(),
                location: GeneralLocation::Location(token.location()),
            })
        } else {
            Err(ParserError::ExpectedToken {
                expected: expected.to_vec(),
                location: GeneralLocation::EndOfFile,
            })
        }
    }
}
```

A.7 src/source.rs

```
use std::fmt::Display;

#[derive(Clone, Copy, Debug)]
pub struct Location {
    pub index: usize,
    pub line: usize,
    pub column: usize,
}

impl Display for Location {
    fn fmt(&self, f: &mut std::fmt::Formatter<'_>) -> std::fmt::Result {
        write!(f, "[line {}, column {}]", self.line, self.column)
    }
}

impl Location {
    pub fn start() -> Self {
        Self {
            index: 0,
            line: 1,
            column: 1,
        }
    }
}

pub enum GeneralLocation {
    Location(Location),
    EndOfFile,
}

impl Display for GeneralLocation {
    fn fmt(&self, f: &mut std::fmt::Formatter<'_>) -> std::fmt::Result {
        match self {
            GeneralLocation::Location(location) => write!(f, "{}", location),
            GeneralLocation::EndOfFile => write!(f, "[end of file]"),
        }
    }
}
```

```rust
pub struct Source {
    text: Vec<char>,
    location: Location,
}

impl Source {
    pub fn new(text: &str) -> Self {
        Self {
            text: text.chars().collect(),
            location: Location::start(),
        }
    }

    pub fn peek(&self) -> Option<char> {
        self.text.get(self.location.index).copied()
    }

    pub fn peek_after(&self) -> Option<char> {
        self.text.get(self.location.index + 1).copied()
    }

    pub fn advance(&mut self) -> Option<char> {
        let next = self.peek();

        if let Some(character) = next {
            self.location.index += 1;
            self.location.column += 1;

            if character == '\n' {
                self.location.line += 1;
                self.location.column = 1;
            }
        }

        next
    }

    pub fn matches(&mut self, target: char) -> bool {
        if let Some(character) = self.peek() {
            if character == target {
                self.advance();
            }

            return character == target;
        }

        false
    }

    pub fn location(&self) -> Location {
        self.location
    }
}
```

A.8 src/token.rs

```rust
use crate::{
    expression::{BinaryOperator, UnaryOperator},
    source::Location,
};

#[derive(Debug)]
pub struct Token {
    data: TokenData,
    location: Location,
}

impl Token {
    pub fn new(data: TokenData, location: Location) -> Self {
        Self { data, location }
    }

    pub fn kind(&self) -> TokenKind {
        self.data.kind()
    }

    pub fn location(&self) -> Location {
        self.location
    }

    pub fn data(self) -> TokenData {
        self.data
    }
}

#[derive(Debug)]
pub enum TokenData {
    LeftParenthesis,
    RightParenthesis,
    QuestionMark,
    Colon,

    // Arithmetic operators
    Plus,
    Minus,
    Star,
    Slash,

    // Logical and bitwise operators
    Bang,
    BangEqual,
    DoubleEqual,
    Greater,
    GreaterEqual,
    Less,
    LessEqual,
    Ampersand,
    DoubleAmpersand,
    Pipe,
    DoublePipe,
```

```
    // Literals
    Float(f64),
    Integer(i32),
    Boolean(bool),
}

impl TokenData {
    pub fn kind(&self) -> TokenKind {
        match self {
            TokenData::LeftParenthesis => TokenKind::LeftParenthesis,
            TokenData::RightParenthesis => TokenKind::RightParenthesis,
            TokenData::QuestionMark => TokenKind::QuestionMark,
            TokenData::Colon => TokenKind::Colon,

            // Arithmetic operators
            TokenData::Plus => TokenKind::Plus,
            TokenData::Minus => TokenKind::Minus,
            TokenData::Star => TokenKind::Star,
            TokenData::Slash => TokenKind::Slash,

            // Logical and bitwise operators
            TokenData::Bang => TokenKind::Bang,
            TokenData::BangEqual => TokenKind::BangEqual,
            TokenData::DoubleEqual => TokenKind::DoubleEqual,
            TokenData::Greater => TokenKind::Greater,
            TokenData::GreaterEqual => TokenKind::GreaterEqual,
            TokenData::Less => TokenKind::Less,
            TokenData::LessEqual => TokenKind::LessEqual,
            TokenData::Ampersand => TokenKind::Ampersand,
            TokenData::DoubleAmpersand => TokenKind::DoubleAmpersand,
            TokenData::Pipe => TokenKind::Pipe,
            TokenData::DoublePipe => TokenKind::DoublePipe,

            // Literals
            TokenData::Float(_) => TokenKind::Float,
            TokenData::Integer(_) => TokenKind::Integer,
            TokenData::Boolean(_) => TokenKind::Boolean,
        }
    }
}

#[derive(Debug, PartialEq, Eq, Clone, Copy)]
pub enum TokenKind {
    LeftParenthesis,
    RightParenthesis,
    QuestionMark,
    Colon,

    // Arithmetic operators
    Plus,
    Minus,
    Star,
    Slash,

    // Logical and bitwise operators
    Bang,
```

```
    BangEqual,
    DoubleEqual,
    Greater,
    GreaterEqual,
    Less,
    LessEqual,
    Ampersand,
    DoubleAmpersand,
    Pipe,
    DoublePipe,

    // Literals
    Float,
    Integer,
    Boolean,
}

impl TokenKind {
    pub fn binary_operator(&self) -> Option<BinaryOperator> {
        Some(match self {
            Self::Plus => BinaryOperator::Add,
            Self::Minus => BinaryOperator::Subtract,
            Self::Star => BinaryOperator::Multiply,
            Self::Slash => BinaryOperator::Divide,

            Self::BangEqual => BinaryOperator::NotEqualTo,
            Self::DoubleEqual => BinaryOperator::EqualTo,
            Self::Greater => BinaryOperator::GreaterThan,
            Self::GreaterEqual => BinaryOperator::GreaterThanOrEqualTo,
            Self::Less => BinaryOperator::LessThan,
            Self::LessEqual => BinaryOperator::LessThanOrEqualTo,

            Self::Ampersand => BinaryOperator::BitwiseAND,
            Self::DoubleAmpersand => BinaryOperator::AND,
            Self::Pipe => BinaryOperator::BitwiseOR,
            Self::DoublePipe => BinaryOperator::OR,

            _ => return None,
        })
    }

    pub fn unary_operator(&self) -> Option<UnaryOperator> {
        Some(match self {
            Self::Minus => UnaryOperator::Minus,
            Self::Bang => UnaryOperator::NOT,

            _ => return None,
        })
    }
}
```

A.9 src/token_stream.rs

```
use std::collections::VecDeque;

use crate::{
    expression::{BinaryOperator, UnaryOperator},
```

```
    parser::ParserError,
    source::{GeneralLocation, Location},
    token::{Token, TokenKind},
};

pub struct TokenStream {
    tokens: VecDeque<Token>,
}

impl TokenStream {
    pub fn new(tokens: Vec<Token>) -> Self {
        Self {
            tokens: tokens.into(),
        }
    }

    pub fn peek(&self) -> Option<&Token> {
        self.tokens.get(0)
    }

    pub fn advance(&mut self) -> Option<Token> {
        self.tokens.pop_front()
    }

    pub fn matches(&mut self, targets: &[TokenKind]) -> Option<Token> {
        if let Some(next) = self.peek() {
            for target in targets {
                if &next.kind() == target {
                    return self.advance();
                }
            }
        }

        None
    }

    pub fn binary_operator(
        &mut self,
        targets: &[BinaryOperator],
    ) -> Option<(BinaryOperator, Location)> {
        if let Some(next) = self.peek() {
            let location = next.location();

            if let Some(operator) = next.kind().binary_operator() {
                for target in targets {
                    if target == &operator {
                        self.advance();
                        return Some((operator, location));
                    }
                }
            }
        }

        None
    }
```

```
    pub fn unary_operator(
        &mut self,
        targets: &[UnaryOperator],
    ) -> Option<(UnaryOperator, Location)> {
        if let Some(next) = self.peek() {
            let location = next.location();

            if let Some(operator) = next.kind().unary_operator() {
                for target in targets {
                    if target == &operator {
                        self.advance();
                        return Some((operator, location));
                    }
                }
            }
        }

        None
    }

    pub fn consume(&mut self, kind: TokenKind) -> Result<Token, ParserError> {
        if let Some(token) = self.matches(&[kind]) {
            Ok(token)
        } else if let Some(token) = self.peek() {
            Err(ParserError::ExpectedToken {
                expected: vec![kind],
                location: GeneralLocation::Location(token.location()),
            })
        } else {
            Err(ParserError::ExpectedToken {
                expected: vec![kind],
                location: GeneralLocation::EndOfFile,
            })
        }
    }
}
```

B Interpreter source code

The interpreter source code has the following directory structure:

```
.
├── Cargo.lock
├── Cargo.toml
└── src
    ├── environment.rs
    ├── expression.rs
    ├── heap
    │   ├── garbage_collected.rs
    │   ├── mod.rs
    │   ├── naive.rs
    │   └── reference_counted.rs
    ├── lexer.rs
    ├── main.rs
    ├── parser.rs
    ├── source.rs
    ├── stack.rs
    ├── statement.rs
    ├── stats.rs
    ├── token.rs
    ├── token_stream.rs
    └── value.rs

3 directories, 18 files
```

B.1 `Cargo.lock`

```
# This file is automatically @generated by Cargo.
# It is not intended for manual editing.
version = 4

[[package]]
name = "slang_interpreter"
version = "0.1.0"
```

B.2 `Cargo.toml`

```
[package]
name = "slang_interpreter"
version = "0.1.0"
edition = "2024"

[dependencies]
```

B.3 `src/heap/garbage_collected.rs`

```
use std::{cell::RefCell, rc::Rc};

use crate::{
    heap::{HeapObject, Object, Pointer},
    value::Value,
};

pub struct GarbageCollectedHeap {
    heap: Vec<Pointer>,
}
```

```
impl GarbageCollectedHeap {
    pub fn new() -> Self {
        Self { heap: Vec::new() }
    }

    pub fn allocate(&mut self, data: Object) -> Pointer {
        let data = data
            .into_iter()
            .map(|(key, value)| match value {
                Value::Object(object) => (key,
Value::ObjectReference(self.allocate(object))),
                value => (key, value),
            })
            .collect();

        let heap_object = HeapObject {
            data,
            marked: false,
            reference_count: 1,
        };

        let pointer = Pointer::new(RefCell::new(heap_object));
        self.heap.push(Rc::clone(&pointer));

        pointer
    }

    pub fn manage(&mut self, roots: &[Pointer]) {
        for root in roots {
            self.traverse(Rc::clone(&root));
        }

        self.heap.retain(|value| value.borrow().marked);

        for object in &self.heap {
            object.borrow_mut().marked = false;
        }
    }

    fn traverse(&mut self, root: Pointer) {
        {
            let root = root.borrow();

            if root.marked {
                return;
            }
        }

        {
            let mut root = root.borrow_mut();
            root.marked = true;
        }

        let root = root.borrow();

        for value in root.data.values() {
```

```
            if let Value::ObjectReference(pointer) = value {
                self.traverse(Rc::clone(&pointer));
            }
        }
    }

    pub fn objects_count(&self) -> usize {
        self.heap.len()
    }
}
```

B.4 src/heap/mod.rs

```
use std::{cell::RefCell, collections::HashMap, rc::Rc};

use crate::{
    heap::{
        garbage_collected::GarbageCollectedHeap, naive::NaiveHeap,
        reference_counted::ReferenceCountedHeap,
    },
    value::Value,
};

pub mod garbage_collected;
pub mod naive;
pub mod reference_counted;

pub type Object = HashMap<String, Value>;

pub type Pointer = Rc<RefCell<HeapObject>>;

#[derive(PartialEq)]
pub struct HeapObject {
    pub data: Object,
    pub marked: bool,
    pub reference_count: usize,
}

pub enum ManagedHeap {
    GarbageCollected(GarbageCollectedHeap),
    Naive(NaiveHeap),
    ReferenceCounted(ReferenceCountedHeap),
}

impl ManagedHeap {
    pub fn allocate(&mut self, data: Object) -> Pointer {
        match self {
            Self::GarbageCollected(heap) => heap.allocate(data),
            Self::Naive(heap) => heap.allocate(data),
            Self::ReferenceCounted(heap) => heap.allocate(data),
        }
    }

    pub fn objects_count(&self) -> usize {
        match self {
            Self::GarbageCollected(heap) => heap.objects_count(),
            Self::Naive(heap) => heap.objects_count(),
```

```
            Self::ReferenceCounted(heap) => heap.objects_count(),
        }
    }

    pub fn get_technique_code(&self) -> String {
        match self {
            Self::GarbageCollected(_) => "gc",
            Self::ReferenceCounted(_) => "rc",
            Self::Naive(_) => "na",
        }
        .to_string()
    }
}
```

B.5 src/heap/naive.rs

```
use std::{cell::RefCell, rc::Rc};

use crate::{
    heap::{HeapObject, Object, Pointer},
    value::Value,
};

pub struct NaiveHeap {
    heap: Vec<Pointer>,
}

impl NaiveHeap {
    pub fn new() -> Self {
        Self { heap: Vec::new() }
    }

    pub fn allocate(&mut self, data: Object) -> Pointer {
        let data = data
            .into_iter()
            .map(|(key, value)| match value {
                Value::Object(object) => (key,
Value::ObjectReference(self.allocate(object))),
                value => (key, value),
            })
            .collect();

        let heap_object = HeapObject {
            data,
            marked: false,
            reference_count: 1,
        };

        let pointer = Pointer::new(RefCell::new(heap_object));
        self.heap.push(Rc::clone(&pointer));

        pointer
    }

    pub fn objects_count(&self) -> usize {
        self.heap.len()
```

```
    }
}
```

B.6 src/heap/reference_counted.rs

```
use std::{cell::RefCell, rc::Rc};

use crate::{
    heap::{HeapObject, Object, Pointer},
    value::Value,
};

pub struct ReferenceCountedHeap {
    heap: Vec<Pointer>,
}

impl ReferenceCountedHeap {
    pub fn new() -> Self {
        Self { heap: Vec::new() }
    }

    pub fn allocate(&mut self, data: Object) -> Pointer {
        let data = data
            .into_iter()
            .map(|(key, value)| match value {
                Value::ObjectReference(pointer) => {
                    self.increment(Rc::clone(&pointer));
                    (key, Value::ObjectReference(pointer))
                }
                Value::Object(object) => (key,
Value::ObjectReference(self.allocate(object))),
                value => (key, value),
            })
            .collect();

        let heap_object = HeapObject {
            data,
            marked: false,
            reference_count: 1,
        };

        let pointer = Pointer::new(RefCell::new(heap_object));
        self.heap.push(Rc::clone(&pointer));

        pointer
    }

    pub fn increment(&mut self, object: Pointer) {
        object.borrow_mut().reference_count += 1;
    }

    pub fn decrement(&mut self, object: Pointer) {
        let count = object.borrow().reference_count;

        match count {
            0 => {
                self.heap
```

```
                    .retain(|object| object.borrow().reference_count > 0);
            }
            1 => {
                object.borrow_mut().reference_count -= 1;

                for value in object.borrow().data.values() {
                    if let Value::ObjectReference(pointer) = value {
                        self.decrement(Rc::clone(pointer));
                    }
                }

                self.heap
                    .retain(|object| object.borrow().reference_count > 0);
            }
            2.. => object.borrow_mut().reference_count -= 1,
        }
    }

    pub fn conditionally_decrement(&mut self, value: Value) {
        if let Value::ObjectReference(pointer) = value {
            self.decrement(pointer);
        }
    }

    pub fn objects_count(&self) -> usize {
        self.heap.len()
    }
}
```

B.7 `src/environment.rs`

```
//! Environments and scopes for the slang programming language.

use std::{cell::RefCell, collections::HashMap, mem, rc::Rc};

use crate::{
    heap::Pointer,
    value::{Function, NativeFunction, Value},
};

/// All errors which can occur while accessing the environment.
pub enum EnvironmentError {
    /// When there is an attempt to assign a value to a target which has not been
defined.
    UndefinedAssignmentTarget { identifier: String },
    /// When there is an attempt to get the value of a target which has not been
initialised.
    UninitialisedTarget { identifier: String },
    /// When there is an attempt to get the value of a target which has not been
defined.
    UndefinedTarget { identifier: String },
}

/// An [Environment] represents a set of scopes, stacked on top of one another.
///
/// Note that this is not the same thing as the stack: the stack has a frame for each
subroutine call, whereas the environment has a new scope for each block scope (e.g.
```

```
if-statements, while-loops).
pub struct Environment {
    /// The parent scope.
    parent: Option<MutEnvironment>,
    /// The current scope.
    scope: HashMap<String, Option<Value>>,
    /// Object references returned from functions.
    returned_object_references: Vec<Pointer>,
}

pub type MutEnvironment = Rc<RefCell<Environment>>;

impl Environment {
    /// Creates a new [Environment].
    pub fn new(parent: Option<MutEnvironment>) -> Self {
        let mut scope = HashMap::new();

        if parent.is_none() {
            [
                ("print", NativeFunction::Print),
                ("format", NativeFunction::Format),
                ("input", NativeFunction::Input),
                ("int", NativeFunction::Int),
                ("float", NativeFunction::Float),
            ]
            .into_iter()
            .for_each(|(identifier, function)| {
                scope.insert(
                    identifier.to_string(),
                    Some(Value::Function(Function::Native(function))),
                );
            });
        }

        let returned_object_references = Vec::new();

        Self {
            scope,
            parent,
            returned_object_references,
        }
    }

    /// Defines a new target and inserts it into the innermost scope.
    pub fn define(&mut self, identifier: String, value: Option<Value>) {
        self.scope.insert(identifier, value);
    }

    /// Assigns a value to an initialised target.
    ///
    /// In order to find the target to mutate, the program starts in the innermost
scope and works outwards until the target is found (or is not found anywhere).
    pub fn assign(
        &mut self,
        identifier: String,
        value: Option<Value>,
```

```
    ) -> Result<Option<Value>, EnvironmentError> {
        if let Some(target) = self.scope.get_mut(&identifier) {
            let mut value = value;

            mem::swap(target, &mut value);

            let previous = value;

            Ok(previous)
        } else if let Some(parent) = &self.parent {
            parent.borrow_mut().assign(identifier, value)
        } else {
            Err(EnvironmentError::UndefinedAssignmentTarget { identifier })
        }
    }

    /// Gets the value of a target.
    ///
    /// In order to find the target, the program starts in the innermost scope and
works outwards until the target is found (or is not found anywhere).
    pub fn get(&self, identifier: &str) -> Result<Value, EnvironmentError> {
        match self.scope.get(identifier) {
            Some(Some(value)) => Ok(value.clone()),
            Some(None) => Err(EnvironmentError::UninitialisedTarget {
                identifier: identifier.to_string(),
            }),
            None => {
                if let Some(parent) = &self.parent {
                    parent.borrow().get(identifier)
                } else {
                    Err(EnvironmentError::UndefinedTarget {
                        identifier: identifier.to_string(),
                    })
                }
            }
        }
    }

    /// Gets the outermost scope.
    ///
    /// Accepts an Rc<RefCell> to itself.
    pub fn global(&self, self_reference: MutEnvironment) -> MutEnvironment {
        if let Some(parent) = &self.parent {
            parent.borrow().global(Rc::clone(parent))
        } else {
            self_reference
        }
    }

    pub fn roots(&self) -> Vec<Pointer> {
        let mut roots = Vec::new();

        for value in self.scope.values() {
            if let Some(Value::ObjectReference(pointer)) = value {
                roots.push(pointer.clone());
            }
```

```
        }

        if let Some(parent) = &self.parent {
            roots.append(&mut parent.borrow().roots());
        }

        roots
    }

    pub fn returned_object_references(&self) -> &Vec<Pointer> {
        &self.returned_object_references
    }

    pub fn add_returned_object_reference(&mut self, pointer: Pointer) {
        self.returned_object_references.push(pointer);
    }

    pub fn parent(&self) -> Option<MutEnvironment> {
        match &self.parent {
            Some(parent) => Some(Rc::clone(&parent)),
            None => None,
        }
    }

    pub fn values(&self) -> Vec<Value> {
        self.scope
            .iter()
            .filter_map(|(_, value)| value.clone())
            .collect()
    }
}
```

B.8 src/expression.rs

```
//! Expressions within the slang programming language.

use std::{
    collections::HashMap,
    error::Error,
    fmt::{Debug, Display},
    io::{self, Write},
};

use crate::{
    environment::EnvironmentError,
    heap::{ManagedHeap, Pointer},
    stack::Stack,
    statement::ControlFlow,
    stats::Logger,
    value::{Function, NativeFunction, Type, Value},
};

/// All errors which can occur while evaluating an expression.
pub enum EvaluationError {
    /// When the value of the condition for a ternary expression does not have the
type of Boolean.
    NonBooleanTernaryCondition {
```

```
        condition: Type,
    },
    /// When the types of the operands for a binary operation are not valid.
    InvalidBinaryTypes {
        left: Type,
        operator: BinaryOperator,
        right: Option<Type>,
    },
    /// When the type of the operand for a unary operation is not valid.
    InvalidUnaryType {
        operator: UnaryOperator,
        operand: Type,
    },
    /// When a division by zero occurs.
    DivisionByZero,
    /// When there is an attempt to get the value of a variable which has not been
defined.
    UndefinedIdentifier {
        identifier: String,
    },
    /// When there is an attempt to get the value of a variable which has not been
initialised.
    UninitialisedTarget {
        identifier: String,
    },
    /// When the value of the condition for a control flow statement does not have
the type of Boolean.
    NonBooleanControlFlowCondition {
        condition: Type,
        control_flow: String,
    },
    AttemptedCallOfNonFunction {
        attempt: Type,
    },
    IncorrectArgumentCount {
        expected: usize,
        passed: usize,
    },
    AttemptToUseNothing,
    AttemptToAccessNonObject {
        attempt: Type,
    },
    UndefinedField(String),
    CastingError {
        from: Value,
        to: Type,
    },
}

impl From<EnvironmentError> for EvaluationError {
    fn from(value: EnvironmentError) -> Self {
        match value {
            EnvironmentError::UndefinedAssignmentTarget { identifier } => {
                Self::UndefinedIdentifier { identifier }
            }
            EnvironmentError::UndefinedTarget { identifier } => {
```

```
                Self::UndefinedIdentifier { identifier }
            }
            EnvironmentError::UninitialisedTarget { identifier } => {
                Self::UninitialisedTarget { identifier }
            }
        }
    }
}

impl Display for EvaluationError {
    fn fmt(&self, f: &mut std::fmt::Formatter<'_>) -> std::fmt::Result {
        write!(f, "[evaluation error] ")?;

        match self {
            Self::NonBooleanTernaryCondition { condition } => write!(
                f,
                "Expected Boolean operand for ternary condition, found {}.",
                condition
            ),
            Self::InvalidBinaryTypes {
                left,
                operator,
                right,
            } => write!(
                f,
                "The `{}` operator is not defined for {}{}.",
                operator.raw(),
                left,
                match right {
                    Some(right) => format!(" and {}", right),
                    None => "".to_string(),
                }
            ),
            Self::InvalidUnaryType { operator, operand } => write!(
                f,
                "The unary `{}` operator is not defined for {}.",
                operator.raw(),
                operand
            ),
            Self::DivisionByZero => {
                write!(f, "Division by zero.")
            }
            Self::UndefinedIdentifier { identifier } => {
                write!(f, "The identifier `{}` is not defined.", identifier)
            }
            Self::UninitialisedTarget { identifier } => {
                write!(f, "The target `{}` has not been initialised.", identifier)
            }
            Self::NonBooleanControlFlowCondition {
                condition,
                control_flow,
            } => {
                write!(
                    f,
                    "Expected Boolean {} condition, found {}.",
                    control_flow, condition
```

```
                )
            }
            Self::AttemptedCallOfNonFunction { attempt } => {
                write!(
                    f,
                    "Attempted to 'call' a value of type {} like a function.",
                    attempt
                )
            }
            Self::IncorrectArgumentCount { expected, passed } => {
                write!(
                    f,
                    "Expected {} arguments, but received {}.",
                    expected, passed
                )
            }
            Self::AttemptToUseNothing => write!(
                f,
                "Attempted to use the return value from a function, however the
function returned nothing."
            ),
            Self::AttemptToAccessNonObject { attempt } => write!(
                f,
                "Attempted to access a field of a value of type {}, like an object.",
                attempt
            ),
            Self::UndefinedField(identifier) => {
                write!(
                    f,
                    "Attempted to access a non-existent field `{}` on an object.",
                    identifier
                )
            }
            Self::CastingError { from, to } => {
                write!(f, "Unable to cast from {:?} to {}.", from, to)
            }
        }
    }
}

impl Debug for EvaluationError {
    fn fmt(&self, f: &mut std::fmt::Formatter<'_>) -> std::fmt::Result {
        write!(f, "{}", self)
    }
}

impl Error for EvaluationError {}

/// Represents all possible expressions within the slang programming language.
#[derive(Clone, PartialEq)]
pub enum Expression {
    /// Ternary expressions, in the form `condition ? if_true : if_false`.
    Ternary {
        condition: Box<Expression>,
        left: Box<Expression>,
        right: Box<Expression>,
```

```
    },
    /// Binary expressions, in the form `left operator right`.
    Binary {
        left: Box<Expression>,
        operator: BinaryOperator,
        right: Box<Expression>,
    },
    /// Unary expressions, in the form `operator operand`.
    Unary {
        operator: UnaryOperator,
        operand: Box<Expression>,
    },
    /// A function call.
    Call {
        function: Box<Expression>,
        arguments: Vec<Box<Expression>>,
    },
    /// An assignment expression, which yields the assigned value.
    Assignment {
        identifier: String,
        value: Box<Expression>,
    },
    /// An expression surrounded by parenthesis.
    Grouping {
        contained: Box<Expression>,
    },
    /// A literal value.
    Literal {
        value: Value,
    },
    /// A reference to a variable.
    Variable {
        identifier: String,
    },
    GetField {
        object: Box<Expression>,
        field: String,
    },
    SetField {
        object: Box<Expression>,
        field: String,
        value: Box<Expression>,
    },
    Object(HashMap<String, Expression>),
}

impl Expression {
    /// Evaluates an expression, returning an error if it is nothing.
    pub fn evaluate_not_nothing(
        self,
        stack: &mut Stack,
        heap: &mut ManagedHeap,
        logger: &mut Logger,
    ) -> Result<Value, EvaluationError> {
        self.evaluate(stack, heap, logger)
            .map(|value| match value {
```

```
                Some(value) => Ok(value),
                None => Err(EvaluationError::AttemptToUseNothing),
            })?
    }

    /// Evaluates the expression.
    pub fn evaluate(
        self,
        stack: &mut Stack,
        heap: &mut ManagedHeap,
        logger: &mut Logger,
    ) -> Result<Option<Value>, EvaluationError> {
        match self {
            Self::Ternary {
                condition,
                left,
                right,
            } => Expression::evaluate_ternary(stack, heap, logger, condition, left,
right),

            Self::Binary {
                left,
                operator,
                right,
            } => Expression::evaluate_binary(stack, heap, logger, left, operator,
right),

            Self::Unary { operator, operand } => {
                Expression::evaluate_unary(stack, heap, logger, operator, operand)
            }

            Self::Call {
                function,
                arguments,
            } => Expression::evaluate_call(stack, heap, logger, function, arguments),

            Self::Assignment { identifier, value } => {
                let next = value.evaluate(stack, heap, logger)?;

                let next = match next {
                    Some(Value::Object(data)) =>
Some(Value::ObjectReference(heap.allocate(data))),
                    Some(Value::ObjectReference(ref pointer)) => {
                        if let ManagedHeap::ReferenceCounted(heap) = heap {
                            heap.increment(Pointer::clone(pointer));
                        }

                        next
                    }
                    _ => next,
                };

                /*
                - if next is an Object, then allocate data on the heap and place the
returned pointer on the stack
                - if next is an ObjectReference, then place the pointer on the stack
```

```
(and increment reference count)
                - if next is anything else, just place it directly on the stack

                - previous cannot be an Object
                - if previous is an ObjectReference, then decrement it

                */

                let previous = stack.top().borrow_mut().assign(identifier,
next.clone())?;

                if let (Some(previous), ManagedHeap::ReferenceCounted(heap)) =
(previous, heap) {
                    heap.conditionally_decrement(previous);
                }

                Ok(next)
            }

            Self::Grouping { contained } => contained.evaluate(stack, heap, logger),

            Self::Literal { value } => Ok(Some(value)),

            Self::Variable { identifier } =>
Ok(Some(stack.top().borrow().get(&identifier)?)),

            Self::GetField { object, field } => {
                match object.evaluate_not_nothing(stack, heap, logger)? {
                    Value::ObjectReference(pointer) => {
                        if let Some(value) =
pointer.borrow().data.get(&field).cloned() {
                            Ok(Some(value))
                        } else {
                            Err(EvaluationError::UndefinedField(field))
                        }
                    }
                    Value::Object(fields) => {
                        if let Some(value) = fields.get(&field).cloned() {
                            Ok(Some(value))
                        } else {
                            Err(EvaluationError::UndefinedField(field))
                        }
                    }
                    attempt => Err(EvaluationError::AttemptToAccessNonObject {
                        attempt: attempt.slang_type(),
                    }),
                }
            }

            Self::SetField {
                object,
                field,
                value,
            } => match object.evaluate_not_nothing(stack, heap, logger)? {
                Value::ObjectReference(pointer) => {
                    let next = value.evaluate_not_nothing(stack, heap, logger)?;
```

```
                    let next = match next {
                        Value::Object(data) =>
Value::ObjectReference(heap.allocate(data)),
                        Value::ObjectReference(ref pointer) => {
                            if let ManagedHeap::ReferenceCounted(heap) = heap {
                                heap.increment(Pointer::clone(pointer));
                            }

                            next
                        }
                        _ => next,
                    };

                    let previous = pointer.borrow_mut().data.insert(field,
next.clone());

                    if let (ManagedHeap::ReferenceCounted(heap), Some(previous)) =
(heap, previous)
                    {
                        heap.conditionally_decrement(previous);
                    }

                    Ok(None)
                }
                attempt => Err(EvaluationError::AttemptToAccessNonObject {
                    attempt: attempt.slang_type(),
                }),
            },

            Self::Object(unevaluated_fields) => {
                let mut fields = HashMap::new();

                for (identifier, expression) in unevaluated_fields.into_iter() {
                    /* We evaluate the expression, and if it is an Object, then the
Object itself will be inserted into fields,
                    but if it is an ObjectReference then the pointer will be inserted
into fields. Note that that the reference count
                    is not incremented, but this is correct, as the Object being
evaluated has not yet been assigned to anything, so its children
                    should not have their reference counts incremented.
                    */
                    fields.insert(
                        identifier,
                        expression.evaluate_not_nothing(stack, heap, logger)?,
                    );
                }

                Ok(Some(Value::Object(fields)))
            }
        }
    }

    /// Evaluates a ternary expression.
    fn evaluate_ternary(
        stack: &mut Stack,
```

```
        heap: &mut ManagedHeap,
        logger: &mut Logger,
        condition: Box<Expression>,
        left: Box<Expression>,
        right: Box<Expression>,
    ) -> Result<Option<Value>, EvaluationError> {
        let condition = condition.evaluate_not_nothing(stack, heap, logger)?;

        if let Value::Boolean(condition) = condition {
            if condition {
                return left.evaluate(stack, heap, logger);
            } else {
                return right.evaluate(stack, heap, logger);
            }
        } else {
            return Err(EvaluationError::NonBooleanTernaryCondition {
                condition: condition.slang_type(),
            });
        }
    }

    /// Evaluates a binary expression.
    fn evaluate_binary(
        stack: &mut Stack,
        heap: &mut ManagedHeap,
        logger: &mut Logger,
        left: Box<Expression>,
        operator: BinaryOperator,
        right: Box<Expression>,
    ) -> Result<Option<Value>, EvaluationError> {
        Ok(Some(match operator {
            BinaryOperator::Add => match Self::binary_operands(left, right, stack,
heap, logger)? {
                (Value::String(left), Value::String(right)) => {
                    let mut new = left;
                    new.push_str(&right);
                    Value::String(new)
                }
                (Value::Integer(left), Value::Integer(right)) => Value::Integer(left
+ right),
                (Value::Float(left), Value::Float(right)) => Value::Float(left +
right),
                (left, right) => Err(EvaluationError::InvalidBinaryTypes {
                    left: left.slang_type(),
                    operator,
                    right: Some(right.slang_type()),
                })?,
            },

            BinaryOperator::Subtract => {
                match Self::binary_operands(left, right, stack, heap, logger)? {
                    (Value::Integer(left), Value::Integer(right)) =>
Value::Integer(left - right),
                    (Value::Float(left), Value::Float(right)) => Value::Float(left -
right),
                    (left, right) => Err(EvaluationError::InvalidBinaryTypes {
```

```
                        left: left.slang_type(),
                        operator,
                        right: Some(right.slang_type()),
                    })?,
                }
            }

            BinaryOperator::Multiply => {
                match Self::binary_operands(left, right, stack, heap, logger)? {
                    (Value::Integer(left), Value::Integer(right)) =>
Value::Integer(left * right),
                    (Value::Float(left), Value::Float(right)) => Value::Float(left *
right),
                    (left, right) => Err(EvaluationError::InvalidBinaryTypes {
                        left: left.slang_type(),
                        operator,
                        right: Some(right.slang_type()),
                    })?,
                }
            }

            BinaryOperator::Divide => {
                match Self::binary_operands(left, right, stack, heap, logger)? {
                    (Value::Integer(left), Value::Integer(right)) => {
                        if right == 0 {
                            return Err(EvaluationError::DivisionByZero);
                        }

                        Value::Integer(left / right)
                    }
                    (Value::Float(left), Value::Float(right)) => {
                        if right == 0.0 {
                            return Err(EvaluationError::DivisionByZero);
                        }

                        Value::Float(left / right)
                    }
                    (left, right) => Err(EvaluationError::InvalidBinaryTypes {
                        left: left.slang_type(),
                        operator,
                        right: Some(right.slang_type()),
                    })?,
                }
            }

            BinaryOperator::Exponent => {
                match Self::binary_operands(left, right, stack, heap, logger)? {
                    (Value::Integer(left), Value::Integer(right)) => {
                        if right < 0 {
                            if left == 0 {
                                return Err(EvaluationError::DivisionByZero);
                            }

                            Value::Integer(0)
                        } else {
                            Value::Integer(left.pow(right as u32))
```

```
                        }
                    }
                    (Value::Float(left), Value::Float(right)) =>
Value::Float(left.powf(right)),
                    (left, right) => Err(EvaluationError::InvalidBinaryTypes {
                        left: left.slang_type(),
                        operator: BinaryOperator::Exponent,
                        right: Some(right.slang_type()),
                    })?,
                }
            }

            BinaryOperator::EqualTo => {
                match Self::binary_operands(left, right, stack, heap, logger)? {
                    (Value::String(left), Value::String(right)) =>
Value::Boolean(left == right),
                    (Value::Integer(left), Value::Integer(right)) =>
Value::Boolean(left == right),
                    (Value::Float(left), Value::Float(right)) => Value::Boolean(left
== right),
                    (Value::Boolean(left), Value::Boolean(right)) =>
Value::Boolean(left == right),
                    (Value::Object(left), Value::Object(right)) =>
Value::Boolean(left == right),
                    (Value::Object(left), Value::ObjectReference(right)) => {
                        Value::Boolean(left == right.borrow().data)
                    }
                    (Value::ObjectReference(left), Value::Object(right)) => {
                        Value::Boolean(left.borrow().data == right)
                    }
                    (Value::ObjectReference(left), Value::ObjectReference(right)) =>
{
                        Value::Boolean(left == right)
                    }
                    (left, right) => Err(EvaluationError::InvalidBinaryTypes {
                        left: left.slang_type(),
                        operator,
                        right: Some(right.slang_type()),
                    })?,
                }
            }

            BinaryOperator::NotEqualTo => {
                match Self::binary_operands(left, right, stack, heap, logger)? {
                    (Value::String(left), Value::String(right)) =>
Value::Boolean(left != right),
                    (Value::Integer(left), Value::Integer(right)) =>
Value::Boolean(left != right),
                    (Value::Float(left), Value::Float(right)) =>
Value::Boolean(left != right),
                    (Value::Boolean(left), Value::Boolean(right)) =>
Value::Boolean(left != right),
                    (Value::Object(left), Value::Object(right)) =>
Value::Boolean(left != right),
                    (Value::Object(left), Value::ObjectReference(right)) => {
                        Value::Boolean(left != right.borrow().data)
```

```
                    }
                    (Value::ObjectReference(left), Value::Object(right)) => {
                        Value::Boolean(left.borrow().data != right)
                    }
                    (Value::ObjectReference(left), Value::ObjectReference(right)) =>
{
                        Value::Boolean(left != right)
                    }
                    (left, right) => Err(EvaluationError::InvalidBinaryTypes {
                        left: left.slang_type(),
                        operator,
                        right: Some(right.slang_type()),
                    })?,
                }
            }

            BinaryOperator::GreaterThan => {
                match Self::binary_operands(left, right, stack, heap, logger)? {
                    (Value::String(left), Value::String(right)) =>
Value::Boolean(left > right),
                    (Value::Integer(left), Value::Integer(right)) =>
Value::Boolean(left > right),
                    (Value::Float(left), Value::Float(right)) => Value::Boolean(left
> right),
                    (left, right) => Err(EvaluationError::InvalidBinaryTypes {
                        left: left.slang_type(),
                        operator,
                        right: Some(right.slang_type()),
                    })?,
                }
            }

            BinaryOperator::GreaterThanOrEqualTo => {
                match Self::binary_operands(left, right, stack, heap, logger)? {
                    (Value::String(left), Value::String(right)) =>
Value::Boolean(left >= right),
                    (Value::Integer(left), Value::Integer(right)) =>
Value::Boolean(left >= right),
                    (Value::Float(left), Value::Float(right)) => Value::Boolean(left
>= right),
                    (left, right) => Err(EvaluationError::InvalidBinaryTypes {
                        left: left.slang_type(),
                        operator,
                        right: Some(right.slang_type()),
                    })?,
                }
            }

            BinaryOperator::LessThan => {
                match Self::binary_operands(left, right, stack, heap, logger)? {
                    (Value::String(left), Value::String(right)) =>
Value::Boolean(left < right),
                    (Value::Integer(left), Value::Integer(right)) =>
Value::Boolean(left < right),
                    (Value::Float(left), Value::Float(right)) => Value::Boolean(left
< right),
```

```
                    (left, right) => Err(EvaluationError::InvalidBinaryTypes {
                        left: left.slang_type(),
                        operator,
                        right: Some(right.slang_type()),
                    })?,
                }
            }

            BinaryOperator::LessThanOrEqualTo => {
                match Self::binary_operands(left, right, stack, heap, logger)? {
                    (Value::String(left), Value::String(right)) =>
Value::Boolean(left <= right),
                    (Value::Integer(left), Value::Integer(right)) =>
Value::Boolean(left <= right),
                    (Value::Float(left), Value::Float(right)) => Value::Boolean(left
<= right),
                    (left, right) => Err(EvaluationError::InvalidBinaryTypes {
                        left: left.slang_type(),
                        operator,
                        right: Some(right.slang_type()),
                    })?,
                }
            }

            BinaryOperator::AND => match left.evaluate_not_nothing(stack, heap,
logger)? {
                Value::Boolean(left) => {
                    if left {
                        match right.evaluate_not_nothing(stack, heap, logger)? {
                            Value::Boolean(right) => Value::Boolean(left && right),
                            right => Err(EvaluationError::InvalidBinaryTypes {
                                left: Type::Boolean,
                                operator,
                                right: Some(right.slang_type()),
                            })?,
                        }
                    } else {
                        Value::Boolean(false)
                    }
                }
                left => Err(EvaluationError::InvalidBinaryTypes {
                    left: left.slang_type(),
                    operator,
                    right: None,
                })?,
            },

            BinaryOperator::OR => match left.evaluate_not_nothing(stack, heap,
logger)? {
                Value::Boolean(left) => {
                    if left {
                        Value::Boolean(true)
                    } else {
                        match right.evaluate_not_nothing(stack, heap, logger)? {
                            Value::Boolean(right) => Value::Boolean(left || right),
                            right => Err(EvaluationError::InvalidBinaryTypes {
```

```
                                left: Type::Boolean,
                                operator,
                                right: Some(right.slang_type()),
                            })?,
                        }
                    }
                }
                left => Err(EvaluationError::InvalidBinaryTypes {
                    left: left.slang_type(),
                    operator,
                    right: None,
                })?,
            },

            BinaryOperator::BitwiseAND => {
                match Self::binary_operands(left, right, stack, heap, logger)? {
                    (Value::Integer(left), Value::Integer(right)) =>
Value::Integer(left & right),
                    (Value::Boolean(left), Value::Boolean(right)) =>
Value::Boolean(left & right),
                    (left, right) => Err(EvaluationError::InvalidBinaryTypes {
                        left: left.slang_type(),
                        operator,
                        right: Some(right.slang_type()),
                    })?,
                }
            }

            BinaryOperator::BitwiseOR => {
                match Self::binary_operands(left, right, stack, heap, logger)? {
                    (Value::Integer(left), Value::Integer(right)) =>
Value::Integer(left | right),
                    (Value::Boolean(left), Value::Boolean(right)) =>
Value::Boolean(left | right),
                    (left, right) => Err(EvaluationError::InvalidBinaryTypes {
                        left: left.slang_type(),
                        operator,
                        right: Some(right.slang_type()),
                    })?,
                }
            }
        }))
    }

    /// Evaluates a unary expression.
    fn evaluate_unary(
        stack: &mut Stack,
        heap: &mut ManagedHeap,
        logger: &mut Logger,
        operator: UnaryOperator,
        operand: Box<Expression>,
    ) -> Result<Option<Value>, EvaluationError> {
        let operand = operand.evaluate_not_nothing(stack, heap, logger)?;

        Ok(Some(match operator {
            UnaryOperator::Minus => match operand {
```

```
                Value::Integer(operand) => Value::Integer(-operand),
                Value::Float(operand) => Value::Float(-operand),
                _ => Err(EvaluationError::InvalidUnaryType {
                    operator,
                    operand: operand.slang_type(),
                })?,
            },
            UnaryOperator::NOT => match operand {
                Value::Integer(operand) => Value::Integer(!operand),
                Value::Boolean(operand) => Value::Boolean(!operand),
                _ => Err(EvaluationError::InvalidUnaryType {
                    operator,
                    operand: operand.slang_type(),
                })?,
            },
        }))
    }

    /// Evaluates a function call.
    fn evaluate_call(
        stack: &mut Stack,
        heap: &mut ManagedHeap,
        logger: &mut Logger,
        function: Box<Expression>,
        arguments: Vec<Box<Expression>>,
    ) -> Result<Option<Value>, EvaluationError> {
        match function.evaluate_not_nothing(stack, heap, logger)? {
            Value::Function(Function::UserDefined { parameters, block }) => {
                if parameters.len() != arguments.len() {
                    return Err(EvaluationError::IncorrectArgumentCount {
                        expected: parameters.len(),
                        passed: arguments.len(),
                    });
                }

                let mut evaluated_arguments = Vec::new();

                for argument in arguments.into_iter() {
                    let argument = argument.evaluate_not_nothing(stack, heap,
logger)?;

                    evaluated_arguments.push(match argument {
                        Value::Object(data) =>
Value::ObjectReference(heap.allocate(data)),
                        Value::ObjectReference(ref pointer) => {
                            if let ManagedHeap::ReferenceCounted(heap) = heap {
                                heap.increment(Pointer::clone(pointer));
                            }

                            argument
                        }
                        _ => argument,
                    });
                }

                let call_scope = stack.push();
```

```
                parameters
                    .into_iter()
                    .zip(evaluated_arguments.clone())
                    .for_each(|(parameter, argument)| {
                        call_scope.borrow_mut().define(parameter, Some(argument))
                    });

                let return_value =
                    block
                        .execute(stack, heap, logger)
                        .map(|control| match control {
                            ControlFlow::Break(value) => value,
                            ControlFlow::Continue => None,
                        });

                if let ManagedHeap::ReferenceCounted(heap) = heap {
                    for value in evaluated_arguments {
                        heap.conditionally_decrement(value);
                    }
                }

                stack.pop();

                /*
                    fu f() {
                        let x = {}; (rc = 1)
                        return x; (rc = 2)
                    } (rc = 1)

                    {
                        let a = f(); (rc = 1)

                        f(); (rc = 1)
                    } (both rc = 0)
                */

                // At this point, we can't be sure whether the value returned from
the function will actually be used.
                // If the return value is an object reference, then it is important
that we keep track of this object reference,
                // so that it can be decremented when we leave the scope/stack frame.
This returned object reference is added to a list
                // within the scope, and when that scope is exited, its reference
count is decremented.

                if let Ok(Some(Value::ObjectReference(pointer))) = &return_value {
                    stack.add_returned_object_reference(Pointer::clone(pointer));
                }

                return_value
            }
            Value::Function(Function::Native(function)) => match function {
                NativeFunction::Print => match &arguments[..] {
                    [] => {
                        println!();
```

```
            Ok(None)
        }
        [expression] => {
            println!(
                "{}",
                expression
                    .clone()
                    .evaluate_not_nothing(stack, heap, logger)?
            );
            Ok(None)
        }
        _ => Err(EvaluationError::IncorrectArgumentCount {
            expected: 1,
            passed: arguments.len(),
        }),
    },
    NativeFunction::Input => match &arguments[..] {
        [] => {
            let mut line = String::new();

            let _ = io::stdout().flush();
            let _ = io::stdin().read_line(&mut line);

            Ok(Some(Value::String(line.trim().to_string())))
        }
        [prompt] => {
            print!(
                "{}",
                prompt.clone().evaluate_not_nothing(stack, heap, logger)?
            );

            let mut line = String::new();

            let _ = io::stdout().flush();
            let _ = io::stdin().read_line(&mut line);

            Ok(Some(Value::String(line.trim().to_string())))
        }
        _ => Err(EvaluationError::IncorrectArgumentCount {
            expected: 1,
            passed: arguments.len(),
        }),
    },

    NativeFunction::Format => {
        let mut buffer = String::new();

        for argument in arguments {
            buffer.push_str(&format!(
                "{}",
                argument.evaluate_not_nothing(stack, heap, logger)?
            ));
        }

        Ok(Some(Value::String(buffer)))
    }
```

```
                NativeFunction::Int => match &arguments[..] {
                    [argument] => {
                        let argument =
                            argument.clone().evaluate_not_nothing(stack, heap,
logger)?;

                        match argument {
                            Value::Integer(integer) =>
Ok(Some(Value::Integer(integer))),
                            Value::Float(float) => {
                                if float.round() == float {
                                    let integer = float as i32;

                                    Ok(Some(Value::Integer(integer)))
                                } else {
                                    Err(EvaluationError::CastingError {
                                        from: argument,
                                        to: Type::Integer,
                                    })
                                }
                            }
                            Value::String(ref value) => {
                                if let Ok(integer) = value.parse() {
                                    Ok(Some(Value::Integer(integer)))
                                } else {
                                    Err(EvaluationError::CastingError {
                                        from: argument,
                                        to: Type::Integer,
                                    })
                                }
                            }
                            _ => Err(EvaluationError::CastingError {
                                from: argument,
                                to: Type::Integer,
                            }),
                        }
                    }
                    _ => Err(EvaluationError::IncorrectArgumentCount {
                        expected: 1,
                        passed: arguments.len(),
                    }),
                },
                NativeFunction::Float => match &arguments[..] {
                    [argument] => {
                        let argument =
                            argument.clone().evaluate_not_nothing(stack, heap,
logger)?;

                        match argument {
                            Value::Integer(integer) => Ok(Some(Value::Float(integer
as f64))),
                            Value::Float(float) => Ok(Some(Value::Float(float))),
                            Value::String(ref value) => {
                                if let Ok(float) = value.parse() {
                                    Ok(Some(Value::Float(float)))
                                } else {
```

```
                                Err(EvaluationError::CastingError {
                                    from: argument,
                                    to: Type::Float,
                                })
                            }
                        }
                        _ => Err(EvaluationError::CastingError {
                            from: argument,
                            to: Type::Float,
                        }),
                    }
                }
                _ => Err(EvaluationError::IncorrectArgumentCount {
                    expected: 1,
                    passed: arguments.len(),
                }),
            },
        },
        other => Err(EvaluationError::AttemptedCallOfNonFunction {
            attempt: other.slang_type(),
        }),
    }
}

/// Evaluates a set of binary operands, ensuring that they are not nothing.
fn binary_operands(
    left: Box<Expression>,
    right: Box<Expression>,
    stack: &mut Stack,
    heap: &mut ManagedHeap,
    logger: &mut Logger,
) -> Result<(Value, Value), EvaluationError> {
    Ok((
        left.evaluate_not_nothing(stack, heap, logger)?,
        right.evaluate_not_nothing(stack, heap, logger)?,
    ))
}
}

/// All valid binary operators.
#[derive(Debug, PartialEq, Eq, Clone, Copy)]
pub enum BinaryOperator {
    // Arithmetic operators
    Add,
    Subtract,
    Multiply,
    Divide,
    Exponent,

    // Logical operators
    EqualTo,
    NotEqualTo,
    GreaterThan,
    GreaterThanOrEqualTo,
    LessThan,
    LessThanOrEqualTo,
```

```
    AND,
    OR,

    // Bitwise operators
    BitwiseAND,
    BitwiseOR,
}

impl BinaryOperator {
    /// How the binary operator will appear in source code.
    pub fn raw(&self) -> String {
        match self {
            Self::Add => "+",
            Self::Subtract => "-",
            Self::Multiply => "*",
            Self::Divide => "/",
            Self::Exponent => "^",

            Self::EqualTo => "==",
            Self::NotEqualTo => "!=",
            Self::GreaterThan => ">",
            Self::GreaterThanOrEqualTo => ">=",
            Self::LessThan => "<",
            Self::LessThanOrEqualTo => "<=",
            Self::AND => "&&",
            Self::OR => "||",

            Self::BitwiseAND => "&",
            Self::BitwiseOR => "|",
        }
        .to_string()
    }
}

/// All valid unary operators.
#[derive(Debug, PartialEq, Eq, Clone, Copy)]
pub enum UnaryOperator {
    Minus,
    NOT,
}

impl UnaryOperator {
    /// How the unary operator will appear in source code.
    pub fn raw(&self) -> String {
        match self {
            Self::Minus => "-",
            Self::NOT => "!",
        }
        .to_string()
    }
}
```

B.9 src/lexer.rs

```
//! The lexer for the slang programming language.

use std::{
```

```
    error::Error,
    fmt::{Debug, Display},
};

use crate::{
    source::{Location, Source},
    token::{Token, TokenData},
};

/// All the errors which can occur while lexing.
pub enum LexerError {
    /// A string without the enclosing `"`.
    UnterminatedString(Location),
    /// A block comment without the enclosing `*/`.
    UnterminatedBlockComment(Location),
    /// An unexpected character. Optionally specify which character was expected.
    UnexpectedCharacter {
        location: Location,
        character: char,
        expected: Option<char>,
    },
}

impl Display for LexerError {
    fn fmt(&self, f: &mut std::fmt::Formatter<'_>) -> std::fmt::Result {
        match self {
            Self::UnterminatedString(location) => {
                write!(f, "{} Unterminated string.", location)
            }
            Self::UnterminatedBlockComment(location) => {
                write!(f, "{} Unterminated block comment.", location)
            }
            Self::UnexpectedCharacter {
                location,
                character,
                expected,
            } => write!(
                f,
                "{} Unexpected character: `{}`{}",
                location,
                character,
                match expected {
                    Some(expected) => format!(" (expected `{}`)", expected),
                    None => String::new(),
                }
            ),
        }
    }
}

impl Debug for LexerError {
    fn fmt(&self, f: &mut std::fmt::Formatter<'_>) -> std::fmt::Result {
        write!(f, "{}", self)
    }
}
```

```
impl Error for LexerError {}

/// An instance of a lexer, for a specific source code string.
pub struct Lexer {
    source: Source,
    tokens: Vec<Token>,
    current_token_start: Location,
}

impl Lexer {
    /// Creates a new lexer for a specific source.
    pub fn new(source: Source) -> Self {
        Self {
            source: source,
            tokens: Vec::new(),
            current_token_start: Location::start(),
        }
    }

    /// Attempts to lexically analyse the source code to produce a sequence of
tokens.
    ///
    /// Will consume the entire source code, returning all valid tokens, and any
errors.
    pub fn lex(mut self) -> (Vec<Token>, Vec<LexerError>) {
        let mut errors = Vec::new();

        while let Some(character) = self.source.advance() {
            let result = match character {
                '(' => Ok(self.add_token(TokenData::LeftParenthesis)),
                ')' => Ok(self.add_token(TokenData::RightParenthesis)),
                '{' => Ok(self.add_token(TokenData::LeftBrace)),
                '}' => Ok(self.add_token(TokenData::RightBrace)),
                ',' => Ok(self.add_token(TokenData::Comma)),
                '.' => Ok(self.add_token(TokenData::Dot)),
                ';' => Ok(self.add_token(TokenData::Semicolon)),
                '?' => Ok(self.add_token(TokenData::QuestionMark)),
                ':' => Ok(self.add_token(TokenData::Colon)),

                // Arithmetic operators
                '+' => Ok(self.add_token(TokenData::Plus)),
                '-' => Ok(self.add_token(TokenData::Minus)),
                '*' => Ok(self.add_token(TokenData::Star)),
                '/' => self.handle_slash(),
                '^' => Ok(self.add_token(TokenData::Exponent)),

                // Logical and bitwise operators
                '!' => Ok(self.handle_bang()),
                '=' => Ok(self.handle_equal()),
                '>' => Ok(self.handle_greater()),
                '<' => Ok(self.handle_less()),
                '&' => Ok(self.handle_ampersand()),
                '|' => Ok(self.handle_pipe()),

                // Literals (not including booleans)
                '"' => self.handle_string(),
```

```
                character if character.is_ascii_digit() =>
Ok(self.handle_number(character)),

                // Identifiers and keywords
                character if character.is_ascii_alphabetic() || character == '_' => {
                    Ok(self.handle_word(character))
                }

                // Whitespace
                ' ' | '\r' | '\t' | '\n' => Ok(()),

                // Unexpected characters
                _ => Err(LexerError::UnexpectedCharacter {
                    location: self.current_token_start,
                    character: character,
                    expected: None,
                }),
            };

            if let Err(error) = result {
                errors.push(error);
            }

            self.current_token_start = self.source.location();
        }

        (self.tokens, errors)
    }

    /// Adds a token to the internal list of tokens.
    fn add_token(&mut self, data: TokenData) {
        self.tokens.push(Token::new(data, self.current_token_start));
    }

    /// Called when a `!` character is encountered.
    fn handle_bang(&mut self) {
        if self.source.matches('=') {
            self.add_token(TokenData::BangEqual);
        } else {
            self.add_token(TokenData::Bang);
        }
    }

    /// Called when a `=` character is encountered.
    fn handle_equal(&mut self) {
        if self.source.matches('=') {
            self.add_token(TokenData::DoubleEqual);
        } else {
            self.add_token(TokenData::Equal);
        }
    }

    /// Called when a `<` character is encountered.
    fn handle_less(&mut self) {
        if self.source.matches('=') {
            self.add_token(TokenData::LessEqual);
```

```
    } else {
        self.add_token(TokenData::Less);
    }
}

/// Called when a `>` character is encountered.
fn handle_greater(&mut self) {
    if self.source.matches('=') {
        self.add_token(TokenData::GreaterEqual);
    } else {
        self.add_token(TokenData::Greater);
    }
}

/// Called when a `&` character is encountered.
fn handle_ampersand(&mut self) {
    if self.source.matches('&') {
        self.add_token(TokenData::DoubleAmpersand);
    } else {
        self.add_token(TokenData::Ampersand);
    }
}

/// Called when a `|` character is encountered.
fn handle_pipe(&mut self) {
    if self.source.matches('|') {
        self.add_token(TokenData::DoublePipe);
    } else {
        self.add_token(TokenData::Pipe);
    }
}

/// Called when a `/` character is encountered.
fn handle_slash(&mut self) -> Result<(), LexerError> {
    // Block comments
    if self.source.matches('*') {
        while self.source.peek().is_some_and(|character| character != '*')
            || self
                .source
                .peek_after()
                .is_some_and(|character| character != '/')
        {
            self.source.advance();
        }

        if self.source.peek().is_some() && self.source.peek_after().is_some() {
            self.source.advance();
            self.source.advance();

            return Ok(());
        } else {
            return Err(LexerError::UnterminatedBlockComment(
                self.current_token_start,
            ));
        }
    }
```

```
        // Single line comments
        else if self.source.matches('/') {
            while self
                .source
                .peek()
                .is_some_and(|character| character != '\n')
            {
                self.source.advance();
            }
        } else {
            self.add_token(TokenData::Slash);
        }

        Ok(())
    }

    /// Called when a `"` character is encountered.
    fn handle_string(&mut self) -> Result<(), LexerError> {
        let mut string = String::new();

        while let Some(character) = self.source.peek() {
            if character == '"' {
                break;
            }

            string.push(character);
            self.source.advance();
        }

        if self.source.at_end() {
            return Err(LexerError::UnterminatedString(self.current_token_start));
        }

        // Consume the enclosing "
        self.source.advance();

        self.add_token(TokenData::String(string));

        Ok(())
    }

    /// Called when a digit is encountered.
    fn handle_number(&mut self, first_digit: char) {
        let mut number = String::new();

        number.push(first_digit);
        while let Some(character) = self.source.peek() {
            if !character.is_ascii_digit() {
                break;
            }

            number.push(character);
            self.source.advance();
        }

        if self.source.peek().is_some_and(|character| character == '.')
```

```
            && self
                .source
                .peek_after()
                .is_some_and(|character| character.is_ascii_digit())
        {
            number.push('.');
            self.source.advance();

            while let Some(character) = self.source.peek() {
                if !character.is_ascii_digit() {
                    break;
                }

                number.push(character);
                self.source.advance();
            }

            let number: f64 = number.parse().unwrap();

            self.add_token(TokenData::Float(number))
        } else {
            let number: i32 = number.parse().unwrap();

            self.add_token(TokenData::Integer(number));
        }
    }

    /// Called when the start of an identifier or keyword is encountered.
    fn handle_word(&mut self, first_character: char) {
        let mut word = String::new();

        word.push(first_character);

        while let Some(character) = self.source.peek() {
            if character.is_ascii_alphanumeric() || character == '_' {
                word.push(character);
                self.source.advance();
            } else {
                break;
            }
        }

        match word.as_str() {
            // Literals
            "true" => self.add_token(TokenData::Boolean(true)),
            "false" => self.add_token(TokenData::Boolean(false)),

            // Control flow
            "if" => self.add_token(TokenData::If),
            "else" => self.add_token(TokenData::Else),
            "while" => self.add_token(TokenData::While),
            "return" => self.add_token(TokenData::Return),

            // Identifier related
            "let" => self.add_token(TokenData::Let),
            "fu" => self.add_token(TokenData::Fu),
```

```
            _ => self.add_token(TokenData::Identifier(word)),
        };
    }
}
```

B.10 src/main.rs

```
//! Environments and scopes for the slang programming language.

use std::{cell::RefCell, collections::HashMap, mem, rc::Rc};

use crate::{
    heap::Pointer,
    value::{Function, NativeFunction, Value},
};

/// All errors which can occur while accessing the environment.
pub enum EnvironmentError {
    /// When there is an attempt to assign a value to a target which has not been
defined.
    UndefinedAssignmentTarget { identifier: String },
    /// When there is an attempt to get the value of a target which has not been
initialised.
    UninitialisedTarget { identifier: String },
    /// When there is an attempt to get the value of a target which has not been
defined.
    UndefinedTarget { identifier: String },
}

/// An [Environment] represents a set of scopes, stacked on top of one another.
///
/// Note that this is not the same thing as the stack: the stack has a frame for each
subroutine call, whereas the environment has a new scope for each block scope (e.g.
if-statements, while-loops).
pub struct Environment {
    /// The parent scope.
    parent: Option<MutEnvironment>,
    /// The current scope.
    scope: HashMap<String, Option<Value>>,
    /// Object references returned from functions.
    returned_object_references: Vec<Pointer>,
}

pub type MutEnvironment = Rc<RefCell<Environment>>;

impl Environment {
    /// Creates a new [Environment].
    pub fn new(parent: Option<MutEnvironment>) -> Self {
        let mut scope = HashMap::new();

        if parent.is_none() {
            [
                ("print", NativeFunction::Print),
                ("format", NativeFunction::Format),
                ("input", NativeFunction::Input),
                ("int", NativeFunction::Int),
```

```
                ("float", NativeFunction::Float),
            ]
            .into_iter()
            .for_each(|(identifier, function)| {
                scope.insert(
                    identifier.to_string(),
                    Some(Value::Function(Function::Native(function))),
                );
            });
        }

        let returned_object_references = Vec::new();

        Self {
            scope,
            parent,
            returned_object_references,
        }
    }

    /// Defines a new target and inserts it into the innermost scope.
    pub fn define(&mut self, identifier: String, value: Option<Value>) {
        self.scope.insert(identifier, value);
    }

    /// Assigns a value to an initialised target.
    ///
    /// In order to find the target to mutate, the program starts in the innermost
scope and works outwards until the target is found (or is not found anywhere).
    pub fn assign(
        &mut self,
        identifier: String,
        value: Option<Value>,
    ) -> Result<Option<Value>, EnvironmentError> {
        if let Some(target) = self.scope.get_mut(&identifier) {
            let mut value = value;

            mem::swap(target, &mut value);

            let previous = value;

            Ok(previous)
        } else if let Some(parent) = &self.parent {
            parent.borrow_mut().assign(identifier, value)
        } else {
            Err(EnvironmentError::UndefinedAssignmentTarget { identifier })
        }
    }

    /// Gets the value of a target.
    ///
    /// In order to find the target, the program starts in the innermost scope and
works outwards until the target is found (or is not found anywhere).
    pub fn get(&self, identifier: &str) -> Result<Value, EnvironmentError> {
        match self.scope.get(identifier) {
            Some(Some(value)) => Ok(value.clone()),
```

```
            Some(None) => Err(EnvironmentError::UninitialisedTarget {
                identifier: identifier.to_string(),
            }),
            None => {
                if let Some(parent) = &self.parent {
                    parent.borrow().get(identifier)
                } else {
                    Err(EnvironmentError::UndefinedTarget {
                        identifier: identifier.to_string(),
                    })
                }
            }
        }
    }

    /// Gets the outermost scope.
    ///
    /// Accepts an Rc<RefCell> to itself.
    pub fn global(&self, self_reference: MutEnvironment) -> MutEnvironment {
        if let Some(parent) = &self.parent {
            parent.borrow().global(Rc::clone(parent))
        } else {
            self_reference
        }
    }

    pub fn roots(&self) -> Vec<Pointer> {
        let mut roots = Vec::new();

        for value in self.scope.values() {
            if let Some(Value::ObjectReference(pointer)) = value {
                roots.push(pointer.clone());
            }
        }

        if let Some(parent) = &self.parent {
            roots.append(&mut parent.borrow().roots());
        }

        roots
    }

    pub fn returned_object_references(&self) -> &Vec<Pointer> {
        &self.returned_object_references
    }

    pub fn add_returned_object_reference(&mut self, pointer: Pointer) {
        self.returned_object_references.push(pointer);
    }

    pub fn parent(&self) -> Option<MutEnvironment> {
        match &self.parent {
            Some(parent) => Some(Rc::clone(&parent)),
            None => None,
        }
    }
}
```

```
    pub fn values(&self) -> Vec<Value> {
        self.scope
            .iter()
            .filter_map(|(_, value)| value.clone())
            .collect()
    }
}
```

B.11 src/parser.rs

```
//! The parser for the slang programming language.

use std::{
    error::Error,
    fmt::{Debug, Display},
};

use crate::{
    expression::{BinaryOperator, Expression, UnaryOperator},
    source::{GeneralLocation, Location},
    statement::Statement,
    token::{TokenData, TokenKind},
    token_stream::TokenStream,
    value::Value,
};

/// All errors which can occur while parsing.
pub enum ParserError {
    /// When a token was expected but not found.
    ExpectedToken {
        expected: Vec<TokenKind>,
        location: GeneralLocation,
    },
    /// When a unary expression with an unsupported unary operator is encountered.
    UnsupportedUnaryExpression {
        operator: BinaryOperator,
        location: GeneralLocation,
    },
    /// When there is an attempt to assign a value to something which is not
assignable.
    InvalidAssignmentTarget(Location),
}

impl Display for ParserError {
    fn fmt(&self, f: &mut std::fmt::Formatter<'_>) -> std::fmt::Result {
        match self {
            Self::ExpectedToken { expected, location } => {
                write!(
                    f,
                    "{} Expected one of the following tokens: {:?}",
                    location, expected
                )
            }
            Self::UnsupportedUnaryExpression { operator, location } => {
                write!(
                    f,
```

```
                    "{} The unary `{}` operator is not supported.",
                    location,
                    operator.raw(),
                )
            }
            Self::InvalidAssignmentTarget(location) => {
                write!(f, "{} Invalid assignment target.", location)
            }
        }
    }
}

impl Debug for ParserError {
    fn fmt(&self, f: &mut std::fmt::Formatter<'_>) -> std::fmt::Result {
        write!(f, "{}", self)
    }
}

impl Error for ParserError {}

/// A parser for a specific token stream.
pub struct Parser {
    tokens: TokenStream,
}

impl Parser {
    /// Creates a new parser for a specific token stream.
    pub fn new(tokens: TokenStream) -> Self {
        Self { tokens }
    }

    /// Attempts to parse the token stream. Corresponds to `program` in the grammar.
    ///
    /// Consumes the entire token stream. Will attempt to find all errors, while
minimising cascading errors.
    pub fn parse(mut self) -> Result<Vec<Statement>, Vec<ParserError>> {
        let mut statements: Vec<Statement> = Vec::new();
        let mut errors: Vec<ParserError> = Vec::new();

        while !self.tokens.at_end() {
            match self.statement() {
                Ok(statement) => statements.push(statement),
                Err(error) => {
                    errors.push(error);
                    self.synchronize();
                }
            }
        }

        if errors.is_empty() {
            Ok(statements)
        } else {
            Err(errors)
        }
    }
```

```
    /// Consumes tokens until the end of a statement is reached.
    fn synchronize(&mut self) {
        self.tokens.advance();

        while let Some(token) = self.tokens.peek() {
            match token.kind() {
                TokenKind::Semicolon => {
                    self.tokens.advance();
                    return;
                }

                TokenKind::Fu
                | TokenKind::Let
                | TokenKind::If
                | TokenKind::While
                | TokenKind::Return => return,

                _ => {
                    self.tokens.advance();
                }
            }
        }
    }

    /// Attempts to parse a statement. Corresponds to `statement` in the grammar.
    fn statement(&mut self) -> Result<Statement, ParserError> {
        match self.tokens.peek().map(|token| token.kind()) {
            Some(TokenKind::Let) => self.variable_declaration(),
            Some(TokenKind::Fu) => self.function_definition(),
            Some(TokenKind::Return) => self.return_statement(),
            Some(TokenKind::If) => self.if_statement(),
            Some(TokenKind::While) => self.while_loop(),
            Some(TokenKind::LeftBrace) => self.block(),
            _ => self.expression_statement(),
        }
    }

    /// Attempts to parse a variable declaration. Corresponds to
`variableDeclaration` in the grammar.
    fn variable_declaration(&mut self) -> Result<Statement, ParserError> {
        self.tokens.consume(TokenKind::Let)?;

        let identifier = self.tokens.consume_identifier()?;

        let initialiser = if self.tokens.matches(&[TokenKind::Equal]) {
            Some(self.expression()?)
        } else {
            None
        };

        self.tokens.consume(TokenKind::Semicolon)?;

        Ok(Statement::VariableDeclaration {
            identifier,
            initialiser,
        })
```

```
    }

    /// Attempts to parse a function definition. Corresponds to `functionDefinition`
in the grammar.
    fn function_definition(&mut self) -> Result<Statement, ParserError> {
        self.tokens.consume(TokenKind::Fu)?;

        let identifier = self.tokens.consume_identifier()?;

        self.tokens.consume(TokenKind::LeftParenthesis)?;

        let mut parameters = Vec::new();

        if let Ok(parameter) = self.tokens.consume_identifier() {
            parameters.push(parameter);

            while self.tokens.matches(&[TokenKind::Comma]) {
                parameters.push(self.tokens.consume_identifier()?);
            }
        }

        self.tokens.consume(TokenKind::RightParenthesis)?;

        let block = Box::new(self.block()?);

        Ok(Statement::FunctionDefinition {
            identifier,
            parameters,
            block,
        })
    }

    /// Attempts to parse a return statement. Corresponds to `returnStatement` in the
grammar.
    fn return_statement(&mut self) -> Result<Statement, ParserError> {
        self.tokens.consume(TokenKind::Return)?;

        if self.tokens.matches(&[TokenKind::Semicolon]) {
            Ok(Statement::Return(None))
        } else {
            let expression = self.expression()?;
            self.tokens.consume(TokenKind::Semicolon)?;
            Ok(Statement::Return(Some(expression)))
        }
    }

    /// Attempts to parse an if-statement. Corresponds to `ifStatement` in the
grammar.
    fn if_statement(&mut self) -> Result<Statement, ParserError> {
        self.tokens.consume(TokenKind::If)?;

        let condition = self.expression()?;

        let execute_if_true = Box::new(self.block()?);

        let execute_if_false = if self.tokens.matches(&[TokenKind::Else]) {
```

```
            match self
                .tokens
                .peek()
                .map(|token| (token.kind(), token.location()))
            {
                Some((TokenKind::If, _)) => Some(Box::new(self.if_statement()?)),
                Some((TokenKind::LeftBrace, _)) => Some(Box::new(self.block()?)),
                Some((_, location)) => Err(ParserError::ExpectedToken {
                    expected: vec![TokenKind::If, TokenKind::LeftBrace],
                    location: GeneralLocation::Location(location),
                })?,
                None => Err(ParserError::ExpectedToken {
                    expected: vec![TokenKind::If, TokenKind::LeftBrace],
                    location: GeneralLocation::EndOfFile,
                })?,
            }
        } else {
            None
        };

        Ok(Statement::IfStatement {
            condition,
            execute_if_true,
            execute_if_false,
        })
    }

    /// Attempts to parse a while-loop. Corresponds to `whileLoop` in the grammar.
    fn while_loop(&mut self) -> Result<Statement, ParserError> {
        self.tokens.consume(TokenKind::While)?;

        let condition = self.expression()?;

        let block = Box::new(self.block()?);

        Ok(Statement::WhileLoop { condition, block })
    }

    /// Attempts to parse a block statement. Corresponds to `block` in the grammar.
    fn block(&mut self) -> Result<Statement, ParserError> {
        self.tokens.consume(TokenKind::LeftBrace)?;

        let mut statements = Vec::new();

        while self
            .tokens
            .peek()
            .is_some_and(|token| token.kind() != TokenKind::RightBrace)
        {
            statements.push(self.statement()?);
        }

        self.tokens.consume(TokenKind::RightBrace)?;

        Ok(Statement::Block(statements))
    }
```

```
    /// Attempts to parse an expression statement. Corresponds to
`expressionStatement` in the grammar.
    fn expression_statement(&mut self) -> Result<Statement, ParserError> {
        let expression = self.expression()?;

        self.tokens.consume(TokenKind::Semicolon)?;

        Ok(Statement::Expression(expression))
    }

    /// Attempts to parse an expression. Corresponds to `expression` in the grammar.
    fn expression(&mut self) -> Result<Expression, ParserError> {
        self.assignment()
    }

    /// Attempts to parse an assignment. Corresponds to `assignment` in the grammar.
    fn assignment(&mut self) -> Result<Expression, ParserError> {
        let expression = self.ternary()?;

        if let Some(equals) = self.tokens.only_take(&[TokenKind::Equal]) {
            let value = self.assignment()?;

            match expression {
                Expression::GetField { object, field } => Ok(Expression::SetField {
                    object,
                    field,
                    value: Box::new(value),
                }),
                Expression::Variable { identifier } => Ok(Expression::Assignment {
                    identifier,
                    value: Box::new(value),
                }),
                _ => Err(ParserError::InvalidAssignmentTarget(equals.location())),
            }
        } else {
            Ok(expression)
        }
    }

    /// Attempts to parse a ternary expression. Corresponds to `ternary` in the
grammar.
    fn ternary(&mut self) -> Result<Expression, ParserError> {
        let mut expression = self.logical()?;

        if self.tokens.matches(&[TokenKind::QuestionMark]) {
            let left = self.logical()?;

            self.tokens.consume(TokenKind::Colon)?;

            let right = self.logical()?;

            expression = Expression::Ternary {
                condition: Box::new(expression),
                left: Box::new(left),
                right: Box::new(right),
```

```
            }
        }

        Ok(expression)
    }

    /// Attempts to parse a logical expression. Corresponds to `logical` in the
grammar.
    fn logical(&mut self) -> Result<Expression, ParserError> {
        let mut expression = self.equality()?;

        while let Some((operator, _)) = self
            .tokens
            .binary_operator(&[BinaryOperator::AND, BinaryOperator::OR])
        {
            expression = Expression::Binary {
                left: Box::new(expression),
                operator: operator,
                right: Box::new(self.equality()?),
            }
        }

        Ok(expression)
    }

    /// Attempts to parse an equality expression. Corresponds to `equality` in the
grammar.
    fn equality(&mut self) -> Result<Expression, ParserError> {
        let mut expression = self.comparison()?;

        while let Some((operator, _)) = self
            .tokens
            .binary_operator(&[BinaryOperator::NotEqualTo, BinaryOperator::EqualTo])
        {
            expression = Expression::Binary {
                left: Box::new(expression),
                operator,
                right: Box::new(self.comparison()?),
            }
        }

        Ok(expression)
    }

    /// Attempts to parse a comparison expression. Corresponds to `comparison` in the
grammar.
    fn comparison(&mut self) -> Result<Expression, ParserError> {
        let mut expression = self.bitwise()?;

        while let Some((operator, _)) = self.tokens.binary_operator(&[
            BinaryOperator::GreaterThan,
            BinaryOperator::GreaterThanOrEqualTo,
            BinaryOperator::LessThan,
            BinaryOperator::LessThanOrEqualTo,
        ]) {
            expression = Expression::Binary {
```

```
                left: Box::new(expression),
                operator,
                right: Box::new(self.bitwise()?),
            }
        }

        Ok(expression)
    }

    /// Attempts to parse a bitwise expression. Corresponds to `bitwise` in the
grammar.
    fn bitwise(&mut self) -> Result<Expression, ParserError> {
        let mut expression = self.term()?;

        while let Some((operator, _)) = self
            .tokens
            .binary_operator(&[BinaryOperator::BitwiseAND,
BinaryOperator::BitwiseOR])
        {
            expression = Expression::Binary {
                left: Box::new(expression),
                operator,
                right: Box::new(self.term()?),
            }
        }

        Ok(expression)
    }

    /// Attempts to parse a term. Corresponds to `term` in the grammar.
    fn term(&mut self) -> Result<Expression, ParserError> {
        let mut expression = self.factor()?;

        while let Some((operator, _)) = self
            .tokens
            .binary_operator(&[BinaryOperator::Add, BinaryOperator::Subtract])
        {
            expression = Expression::Binary {
                left: Box::new(expression),
                operator,
                right: Box::new(self.factor()?),
            }
        }

        Ok(expression)
    }

    /// Attempts to parse a factor. Corresponds to `factor` in the grammar.
    fn factor(&mut self) -> Result<Expression, ParserError> {
        let mut expression = self.unary()?;

        while let Some((operator, _)) = self
            .tokens
            .binary_operator(&[BinaryOperator::Multiply, BinaryOperator::Divide])
        {
            expression = Expression::Binary {
```

```
                left: Box::new(expression),
                operator,
                right: Box::new(self.unary()?),
            }
        }

        Ok(expression)
    }

    /// Attempts to parse a unary expression. Corresponds to `unary` in the grammar.
    fn unary(&mut self) -> Result<Expression, ParserError> {
        if let Some((operator, _)) = self
            .tokens
            .unary_operator(&[UnaryOperator::Minus, UnaryOperator::NOT])
        {
            Ok(Expression::Unary {
                operator: operator,
                operand: Box::new(self.exponent()?),
            })
        } else if let Some((operator, location)) = self.tokens.binary_operator(&[
            BinaryOperator::Add,
            BinaryOperator::Multiply,
            BinaryOperator::Divide,
            BinaryOperator::NotEqualTo,
            BinaryOperator::EqualTo,
            BinaryOperator::GreaterThan,
            BinaryOperator::GreaterThanOrEqualTo,
            BinaryOperator::LessThan,
            BinaryOperator::LessThanOrEqualTo,
            BinaryOperator::BitwiseAND,
            BinaryOperator::BitwiseOR,
        ]) {
            let _ = self.exponent();

            Err(ParserError::UnsupportedUnaryExpression {
                location: GeneralLocation::Location(location),
                operator: operator,
            })
        } else {
            self.exponent()
        }
    }

    /// Attempts to parse an exponent expression. Corresponds to `exponent` in the
grammar.
    fn exponent(&mut self) -> Result<Expression, ParserError> {
        let mut expression = self.call()?;

        if self.tokens.matches(&[TokenKind::Exponent]) {
            expression = Expression::Binary {
                left: Box::new(expression),
                operator: BinaryOperator::Exponent,
                right: Box::new(self.exponent()?),
            }
        }
```

```
        Ok(expression)
    }

    /// Attempt to parse a call expression. Corresponds to `call` in the grammar.
    fn call(&mut self) -> Result<Expression, ParserError> {
        let mut expression = self.primary()?;

        while let Some(token) = self
            .tokens
            .only_take(&[TokenKind::LeftParenthesis, TokenKind::Dot])
        {
            match token.kind() {
                TokenKind::LeftParenthesis => {
                    let mut arguments = Vec::new();

                    if self
                        .tokens
                        .peek()
                        .is_some_and(|token| token.kind() !=
TokenKind::RightParenthesis)
                    {
                        arguments.push(Box::new(self.expression()?));

                        while self.tokens.matches(&[TokenKind::Comma]) {
                            arguments.push(Box::new(self.expression()?));
                        }
                    }

                    self.tokens.consume(TokenKind::RightParenthesis)?;

                    expression = Expression::Call {
                        function: Box::new(expression),
                        arguments,
                    }
                }
                TokenKind::Dot => {
                    let field = self.tokens.consume_identifier()?;

                    expression = Expression::GetField {
                        object: Box::new(expression),
                        field,
                    }
                }
                _ => unreachable!(),
            }
        }

        Ok(expression)
    }

    /// Attempts to parse a primary expression. Corresponds to `primary` in the
grammar.
    fn primary(&mut self) -> Result<Expression, ParserError> {
        let expected = [
            TokenKind::LeftParenthesis,
            TokenKind::String,
```

```
            TokenKind::Float,
            TokenKind::Integer,
            TokenKind::Boolean,
            TokenKind::Identifier,
            TokenKind::LeftBrace,
        ];

        if let Some(token) = self.tokens.only_take(&expected) {
            Ok(Expression::Literal {
                value: match token.data() {
                    TokenData::LeftParenthesis => {
                        let expression = self.expression()?;

                        self.tokens.consume(TokenKind::RightParenthesis)?;

                        return Ok(Expression::Grouping {
                            contained: Box::new(expression),
                        });
                    }

                    TokenData::String(string) => Value::String(string),

                    TokenData::Float(float) => Value::Float(float),

                    TokenData::Integer(integer) => Value::Integer(integer),

                    TokenData::Boolean(boolean) => Value::Boolean(boolean),

                    TokenData::Identifier(identifier) => {
                        return Ok(Expression::Variable { identifier });
                    }

                    TokenData::LeftBrace => {
                        let mut fields = Vec::new();

                        if self
                            .tokens
                            .peek()
                            .is_some_and(|token| token.kind() !=
TokenKind::RightBrace)
                        {
                            let identifier = self.tokens.consume_identifier()?;
                            self.tokens.consume(TokenKind::Colon)?;
                            let expression = self.expression()?;
                            fields.push((identifier, expression));

                            while self.tokens.matches(&[TokenKind::Comma]) {
                                let identifier = self.tokens.consume_identifier()?;
                                self.tokens.consume(TokenKind::Colon)?;
                                let expression = self.expression()?;
                                fields.push((identifier, expression));
                            }
                        }

                        self.tokens.consume(TokenKind::RightBrace)?;
```

```
                        return Ok(Expression::Object(fields.into_iter().collect()));
                    }

                    _ => unreachable!(),
                },
            })
        } else if let Some(token) = self.tokens.peek() {
            Err(ParserError::ExpectedToken {
                expected: expected.to_vec(),
                location: GeneralLocation::Location(token.location()),
            })
        } else {
            Err(ParserError::ExpectedToken {
                expected: expected.to_vec(),
                location: GeneralLocation::EndOfFile,
            })
        }
    }
}
```

B.12 src/source.rs

```
//! Code relating to the raw source code string.

use std::fmt::Display;

/// Represents the location of a character within a source code string.
#[derive(Clone, Copy, Debug)]
pub struct Location {
    /// The zero-indexed position of the character.
    index: usize,
    /// The line (`>= 1`) which the character appears on.
    line: usize,
    /// The column (`>= 1`) which the character appears in.
    column: usize,
}

impl Display for Location {
    fn fmt(&self, f: &mut std::fmt::Formatter<'_>) -> std::fmt::Result {
        write!(f, "[line {}, column {}]", self.line, self.column)
    }
}

impl Location {
    /// Returns a location representing the first character in a source code string,
with index `0`, line `1`, column `1`.
    pub fn start() -> Self {
        Self {
            index: 0,
            line: 1,
            column: 1,
        }
    }
}

/// Can represent either a specific location, or the end of a source code string.
pub enum GeneralLocation {
```

```
    /// A specific location in the source code.
    Location(Location),
    /// The end of the source code string.
    EndOfFile,
}

impl Display for GeneralLocation {
    fn fmt(&self, f: &mut std::fmt::Formatter<'_>) -> std::fmt::Result {
        match self {
            GeneralLocation::Location(location) => write!(f, "{}", location),
            GeneralLocation::EndOfFile => write!(f, "[end of file]"),
        }
    }
}

/// A wrapper around the source code string.
pub struct Source {
    /// The source code string.
    text: Vec<char>,
    /// The location of the next character.
    location: Location,
}

impl Source {
    /// Creates a new source from a string input.
    pub fn new(text: &str) -> Self {
        Self {
            text: text.chars().collect(),
            location: Location::start(),
        }
    }

    /// Returns the next character in the string, without advancing the position.
    pub fn peek(&self) -> Option<char> {
        self.text.get(self.location.index).copied()
    }

    /// Returns the (next + 1)th character in the string, without advancing the
position.
    pub fn peek_after(&self) -> Option<char> {
        self.text.get(self.location.index + 1).copied()
    }

    /// Returns the next character in the string, and advances the position.
    pub fn advance(&mut self) -> Option<char> {
        let next = self.peek();

        if let Some(character) = next {
            self.location.index += 1;
            self.location.column += 1;

            if character == '\n' {
                self.location.line += 1;
                self.location.column = 1;
            }
        }
```

```
        next
    }

    /// Conditionally advances the position if the next character matches a target.
    ///
    /// If the next character is equal to `target`, then the position is advanced,
and `true` is returned. Otherwise, the position is not advanced, and `false` is
returned.
    pub fn matches(&mut self, target: char) -> bool {
        if let Some(character) = self.peek() {
            if character == target {
                self.advance();
            }

            return character == target;
        }

        false
    }

    /// Returns whether the source code string has been consumed fully.
    pub fn at_end(&self) -> bool {
        self.location.index >= self.text.len()
    }

    /// Returns the location of the next character.
    pub fn location(&self) -> Location {
        self.location
    }
}
```

B.13 src/stack.rs

```
use std::{cell::RefCell, rc::Rc};

use crate::{
    environment::{Environment, MutEnvironment},
    heap::{ManagedHeap, Pointer},
};

pub struct Stack {
    stack: Vec<MutEnvironment>,
}

impl Stack {
    pub fn new() -> Self {
        Stack {
            stack: vec![Rc::new(RefCell::new(Environment::new(None)))],
        }
    }

    pub fn top(&mut self) -> MutEnvironment {
        if let Some(top) = self.stack.last() {
            Rc::clone(top)
        } else {
            let top = Rc::new(RefCell::new(Environment::new(None)));
```

```
            self.stack.push(Rc::clone(&top));

            top
        }
    }

    pub fn enter_scope(&mut self) {
        if let Some(top) = self.stack.last_mut() {
            *top = Rc::new(RefCell::new(Environment::new(Some(Rc::clone(top)))))
        }
    }

    pub fn add_returned_object_reference(&mut self, pointer: Pointer) {
        self.top()
            .borrow_mut()
            .add_returned_object_reference(pointer);
    }

    pub fn exit_scope(&mut self, heap: &mut ManagedHeap) {
        if let Some(top) = self.stack.last_mut() {
            // When exiting a scope, ensure that any object references given to use
by functions are decremented.
            if let ManagedHeap::ReferenceCounted(heap) = heap {
                for pointer in top.borrow().returned_object_references() {
                    heap.decrement(Pointer::clone(pointer));
                }
            }

            let parent = if let Some(parent) = top.borrow().parent() {
                parent
            } else {
                return;
            };

            *top = parent;
        }
    }

    pub fn push(&mut self) -> MutEnvironment {
        let global = match self.stack.first() {
            Some(first) => Some(first.borrow().global(Rc::clone(first))),
            None => None,
        };

        let environment = Rc::new(RefCell::new(Environment::new(global)));

        self.stack.push(Rc::clone(&environment));

        environment
    }

    pub fn pop(&mut self) {
        self.stack.pop();
    }
```

```
    pub fn roots(&self) -> Vec<Pointer> {
        let mut roots = Vec::new();

        for environment in &self.stack {
            roots.append(&mut environment.borrow().roots());
        }

        roots
    }

    pub fn frames_count(&self) -> usize {
        self.stack.len()
    }
}
```

B.14 src/statement.rs

```
//! Statements within the slang programming language.

use crate::{
    expression::{EvaluationError, Expression},
    heap::{ManagedHeap, Pointer},
    stack::Stack,
    stats::Logger,
    value::{Function, Value},
};

/// Used to signal whether a block should be exited early.
pub enum ControlFlow {
    /// Signals that execution of the block should continue.
    Continue,
    /// Signals that execution of the block should terminate, with an optional value
returned.
    Break(Option<Value>),
}

/// Represents a statement.
#[derive(Clone, PartialEq)]
pub enum Statement {
    /// A variable declaration.
    VariableDeclaration {
        identifier: String,
        initialiser: Option<Expression>,
    },
    /// An if-statement.
    IfStatement {
        condition: Expression,
        execute_if_true: Box<Statement>,
        execute_if_false: Option<Box<Statement>>,
    },
    /// A function definition.
    FunctionDefinition {
        identifier: String,
        parameters: Vec<String>,
        block: Box<Statement>,
    },
    /// A return statement.
```

```
    Return(Option<Expression>),
    WhileLoop {
        condition: Expression,
        block: Box<Statement>,
    },
    /// A block.
    Block(Vec<Statement>),
    /// An expression statement.
    Expression(Expression),
}

impl Statement {
    /// Executes a statement and inserts a log entry.
    pub fn execute(
        self,
        stack: &mut Stack,
        heap: &mut ManagedHeap,
        logger: &mut Logger,
    ) -> Result<ControlFlow, EvaluationError> {
        stack.top().borrow_mut().define(
            String::from("STACK_FRAMES_COUNT"),
            Some(Value::Integer(stack.frames_count() as i32)),
        );

        stack.top().borrow_mut().define(
            String::from("HEAP_OBJECTS_COUNT"),
            Some(Value::Integer(heap.objects_count() as i32)),
        );

        stack.top().borrow_mut().define(
            String::from("MEMORY_MANAGEMENT"),
            Some(Value::String(heap.get_technique_code())),
        );

        logger.new_entry(heap.objects_count(), stack.frames_count());

        match self {
            Self::VariableDeclaration {
                identifier,
                initialiser,
            } => {
                let initialiser = match initialiser {
                    Some(initialiser) => {
                        Some(initialiser.evaluate_not_nothing(stack, heap, logger)?)
                    }
                    None => None,
                };

                let previous = stack.top().borrow().get(&identifier);

                let initialiser = match initialiser {
                    Some(Value::Object(data)) =>
Some(Value::ObjectReference(heap.allocate(data))),
                    Some(Value::ObjectReference(ref pointer)) => {
                        if let ManagedHeap::ReferenceCounted(heap) = heap {
                            heap.increment(Pointer::clone(pointer));
```

```
                        }

                        initialiser
                    }
                    _ => initialiser,
                };

                if let (Ok(previous), ManagedHeap::ReferenceCounted(heap)) =
(previous, heap) {
                    heap.conditionally_decrement(previous);
                }

                stack.top().borrow_mut().define(identifier, initialiser);
                Ok(ControlFlow::Continue)
            }
            Self::FunctionDefinition {
                identifier,
                parameters,
                block,
            } => {
                stack.top().borrow_mut().define(
                    identifier,
                    Some(Value::Function(Function::UserDefined { parameters,
block })),
                );
                Ok(ControlFlow::Continue)
            }
            Self::IfStatement {
                condition,
                execute_if_true,
                execute_if_false,
            } => {
                let condition = condition.evaluate_not_nothing(stack, heap, logger)?;

                if let Value::Boolean(condition) = condition {
                    if condition {
                        execute_if_true.execute(stack, heap, logger)
                    } else {
                        match execute_if_false {
                            Some(if_false) => if_false.execute(stack, heap, logger),
                            None => Ok(ControlFlow::Continue),
                        }
                    }
                } else {
                    Err(EvaluationError::NonBooleanControlFlowCondition {
                        condition: condition.slang_type(),
                        control_flow: "if-statement".to_string(),
                    })
                }
            }
            Self::WhileLoop { condition, block } => {
                while match condition
                    .clone()
                    .evaluate_not_nothing(stack, heap, logger)?
                {
                    Value::Boolean(condition) => condition,
```

```
                    condition => Err(EvaluationError::NonBooleanControlFlowCondition
{
                        condition: condition.slang_type(),
                        control_flow: "while-loop".to_string(),
                    })?,
                } {
                    match block.clone().execute(stack, heap, logger)? {
                        ControlFlow::Break(value) => return
Ok(ControlFlow::Break(value)),
                        ControlFlow::Continue => continue,
                    }
                }

                Ok(ControlFlow::Continue)
            }
            Self::Block(statements) => {
                stack.enter_scope();

                let mut non_definitions = Vec::new();

                for statement in statements {
                    match statement {
                        Statement::FunctionDefinition { .. } => {
                            statement.execute(stack, heap, logger)?;
                        }
                        _ => non_definitions.push(statement),
                    }
                }

                let mut return_value = ControlFlow::Continue;

                for statement in non_definitions {
                    match statement.execute(stack, heap, logger)? {
                        ControlFlow::Break(value) => {
                            return_value = ControlFlow::Break(value);
                            break;
                        }
                        ControlFlow::Continue => continue,
                    }
                }

                if let ManagedHeap::ReferenceCounted(heap) = heap {
                    if let ControlFlow::Break(Some(Value::ObjectReference(value))) =
&return_value {
                        heap.increment(Pointer::clone(value));
                    }

                    for value in stack.top().borrow().values() {
                        heap.conditionally_decrement(value);
                    }
                }

                stack.exit_scope(heap);

                if let ManagedHeap::GarbageCollected(heap) = heap {
                    let mut roots = stack.roots();
```

```
                    if let ControlFlow::Break(Some(Value::ObjectReference(pointer)))
= &return_value
                    {
                        roots.push(Pointer::clone(pointer));
                    }

                    heap.manage(&roots);
                }

                Ok(return_value)
            }
            Self::Expression(expression) => match expression.evaluate(stack, heap,
logger) {
                Ok(_) => Ok(ControlFlow::Continue),
                Err(error) => Err(error),
            },
            Self::Return(expression) => match expression {
                Some(expression) => Ok(ControlFlow::Break(
                    expression.evaluate(stack, heap, logger)?,
                )),
                None => Ok(ControlFlow::Break(None)),
            },
        }
    }
}
```

B.15 src/stats.rs

```
use std::{
    fs::{self, File},
    io::{self, BufRead},
    path::Path,
    time::{Duration, Instant},
};

pub struct Logger {
    start: Instant,
    entries: Vec<Entry>,
}

fn get_memory_usage() -> Option<usize> {
    let path = Path::new("/proc/self/status");

    if let Ok(file) = File::open(path) {
        let reader = io::BufReader::new(file);

        for line in reader.lines() {
            let line = line;

            if let Ok(line) = line {
                if !line.starts_with("VmRSS:") {
                    continue;
                }

                let parts: Vec<&str> = line.split_whitespace().collect();
```

```
                // Get the VmRSS value, which is in kilobytes.
                if let Some(rss_kb) = parts.get(1) {
                    return Some(rss_kb.parse::<usize>().unwrap_or(0) * 1000);
                }
            }
        }
    }

    None
}

impl Logger {
    pub fn new() -> Self {
        Self {
            start: Instant::now(),
            entries: Vec::new(),
        }
    }

    pub fn new_entry(&mut self, heap_objects_count: usize, stack_frames_count: usize)
{
        let memory_usage = get_memory_usage();

        self.entries.push(Entry {
            elapsed: self.start.elapsed(),
            heap_objects_count,
            stack_frames_count,
            memory_usage,
        });
    }

    pub fn write_to_csv(self, source_code_filename: &str) {
        let mut contents =
String::from("elapsed,heap_objects_count,stack_frames_count,interpreter_memory_usage");

        for entry in self.entries {
            let memory_usage = if let Some(memory_usage) = entry.memory_usage {
                format!("{}", memory_usage)
            } else {
                String::from("unable to calculate")
            };

            contents.push_str(
                format!(
                    "\n{},{},{},{}",
                    entry.elapsed.as_secs_f64(),
                    entry.heap_objects_count,
                    entry.stack_frames_count,
                    memory_usage,
                )
                .as_str(),
            );
        }

        let filename = format!("{}.csv", source_code_filename);
```

```
        let _ = fs::write(filename, contents);
    }
}

struct Entry {
    elapsed: Duration,
    heap_objects_count: usize,
    stack_frames_count: usize,
    memory_usage: Option<usize>,
}
```

B.16 src/token_stream.rs

```
//! All code relating to the stream of tokens given to the parser.

use std::collections::VecDeque;

use crate::{
    expression::{BinaryOperator, UnaryOperator},
    parser::ParserError,
    source::{GeneralLocation, Location},
    token::{Token, TokenData, TokenKind},
};

/// A wrapper around a queue of tokens.
pub struct TokenStream {
    tokens: VecDeque<Token>,
}

impl TokenStream {
    /// Creates a new token stream from a list of tokens.
    pub fn new(tokens: Vec<Token>) -> Self {
        Self {
            tokens: tokens.into(),
        }
    }

    /// Returns a reference to the next token in the stream.
    pub fn peek(&self) -> Option<&Token> {
        self.tokens.get(0)
    }

    /// Consumes the next token and returns it.
    pub fn advance(&mut self) -> Option<Token> {
        self.tokens.pop_front()
    }

    /// Consumes and returns the next token only if it matches a target.
    pub fn only_take(&mut self, targets: &[TokenKind]) -> Option<Token> {
        if let Some(next) = self.peek() {
            for target in targets {
                if &next.kind() == target {
                    return self.advance();
                }
            }
        }
```

```
        None
    }

    /// Consumes the next token only if it matches a target. The token is not
returned.
    pub fn matches(&mut self, targets: &[TokenKind]) -> bool {
        if let Some(next) = self.peek() {
            for target in targets {
                if &next.kind() == target {
                    self.advance();
                    return true;
                }
            }
        }

        false
    }

    /// Consumes the next token only if it is a binary operator and matches a target.
    pub fn binary_operator(
        &mut self,
        targets: &[BinaryOperator],
    ) -> Option<(BinaryOperator, Location)> {
        if let Some(next) = self.peek() {
            let location = next.location();

            if let Some(operator) = next.kind().binary_operator() {
                for target in targets {
                    if target == &operator {
                        self.advance();
                        return Some((operator, location));
                    }
                }
            }
        }

        None
    }

    /// Consumes the next token only if it is a unary operator and matches a target.
    pub fn unary_operator(
        &mut self,
        targets: &[UnaryOperator],
    ) -> Option<(UnaryOperator, Location)> {
        if let Some(next) = self.peek() {
            let location = next.location();

            if let Some(operator) = next.kind().unary_operator() {
                for target in targets {
                    if target == &operator {
                        self.advance();
                        return Some((operator, location));
                    }
                }
            }
        }
```

```
        None
    }

    /// Consumes the next token only if it is an identifier. Will return an error if
it is not an identifier.
    pub fn consume_identifier(&mut self) -> Result<String, ParserError> {
        let token = self.peek().cloned();

        match token.map(|token| (token.location(), token.data())) {
            Some((_, TokenData::Identifier(identifier))) => {
                self.advance();
                Ok(identifier)
            }
            Some((location, _)) => Err(ParserError::ExpectedToken {
                expected: vec![TokenKind::Identifier],
                location: GeneralLocation::Location(location),
            }),
            None => Err(ParserError::ExpectedToken {
                expected: vec![TokenKind::Identifier],
                location: GeneralLocation::EndOfFile,
            }),
        }
    }

    /// Consumes the next token only if it is of a certain kind. Will return an error
if it is not of that kind.
    pub fn consume(&mut self, kind: TokenKind) -> Result<Token, ParserError> {
        if let Some(token) = self.only_take(&[kind]) {
            Ok(token)
        } else if let Some(token) = self.peek() {
            Err(ParserError::ExpectedToken {
                expected: vec![kind],
                location: GeneralLocation::Location(token.location()),
            })
        } else {
            Err(ParserError::ExpectedToken {
                expected: vec![kind],
                location: GeneralLocation::EndOfFile,
            })
        }
    }

    /// Returns whether all of the tokens have been consumed.
    pub fn at_end(&self) -> bool {
        self.tokens.is_empty()
    }
}
```

B.17 `src/token.rs`

```
//! Tokens within the slang programming language.

use crate::{
    expression::{BinaryOperator, UnaryOperator},
    source::Location,
};
```

```rust
/// The smallest meaningful unit of the language.
#[derive(Debug, Clone)]
pub struct Token {
    /// The contained data, including the token type, and any associated data.
    data: TokenData,
    /// The location of its first character.
    location: Location,
}

impl Token {
    /// Creates a new Token.
    pub fn new(data: TokenData, location: Location) -> Self {
        Self { data, location }
    }

    /// Returns the kind of the token.
    pub fn kind(&self) -> TokenKind {
        self.data.kind()
    }

    /// Returns the location of the token's first character.
    pub fn location(&self) -> Location {
        self.location
    }

    /// Consumes the token and returns its data.
    pub fn data(self) -> TokenData {
        self.data
    }
}

/// The data contained within a token.
///
/// This is similar to [TokenKind], however contains more information. For example,
the [TokenData::Integer] variant has an [i32] field which stores the integer that
token represents, however [TokenKind::Integer] has no contained fields, and is simply
a flag stating that the token represents an integer.
#[derive(Debug, Clone)]
pub enum TokenData {
    /// The `(` character.
    LeftParenthesis,
    /// The `)` character.
    RightParenthesis,
    /// The `{` character.
    LeftBrace,
    /// The `}` character.
    RightBrace,
    /// The `,` character.
    Comma,
    /// The `.` character.
    Dot,
    /// The `;` character.
    Semicolon,
    /// The `?` character.
    QuestionMark,
```

```
    /// The `:` character.
    Colon,

    // Arithmetic operators
    /// The `+` character.
    Plus,
    /// The `-` character.
    Minus,
    /// The `*` character.
    Star,
    /// The `/` character.
    Slash,
    /// The `^` character.
    Exponent,

    // Logical and bitwise operators
    /// The `!` character.
    Bang,
    /// The `!=` string.
    BangEqual,
    /// The `=` character.
    Equal,
    /// The `==` string.
    DoubleEqual,
    /// The `>` character.
    Greater,
    /// The `>=` string.
    GreaterEqual,
    /// The `<` character.
    Less,
    /// The `<=` string.
    LessEqual,
    /// The `&` character.
    Ampersand,
    /// The `&&` string.
    DoubleAmpersand,
    /// The `|` character.
    Pipe,
    /// The `||` string.
    DoublePipe,

    // Literals
    /// String literals enclosed in `"`.
    String(String),
    /// Floating point numbers, denoted with a `.` separating the integer and
fractional parts.
    Float(f64),
    /// Integers.
    Integer(i32),
    /// Either `true` or `false`.
    Boolean(bool),

    // Control flow
    /// The `if` string.
    If,
    /// The `else` string.
```

```
    Else,
    /// The `while` string.
    While,
    /// The `return` string.
    Return,

    // Identifier related
    /// The `let` string.
    Let,
    /// The `fu` string.
    Fu,
    /// All valid identifiers.
    ///
    /// Must start with either an alphabetic character or an underscore, with all
subsequent characters being alphanumeric or underscores.
    Identifier(String),
}

impl TokenData {
    /// Returns the [TokenKind] of some [TokenData].
    pub fn kind(&self) -> TokenKind {
        match self {
            TokenData::LeftParenthesis => TokenKind::LeftParenthesis,
            TokenData::RightParenthesis => TokenKind::RightParenthesis,
            TokenData::LeftBrace => TokenKind::LeftBrace,
            TokenData::RightBrace => TokenKind::RightBrace,
            TokenData::Comma => TokenKind::Comma,
            TokenData::Dot => TokenKind::Dot,
            TokenData::Semicolon => TokenKind::Semicolon,
            TokenData::QuestionMark => TokenKind::QuestionMark,
            TokenData::Colon => TokenKind::Colon,

            // Arithmetic operators
            TokenData::Plus => TokenKind::Plus,
            TokenData::Minus => TokenKind::Minus,
            TokenData::Star => TokenKind::Star,
            TokenData::Slash => TokenKind::Slash,
            TokenData::Exponent => TokenKind::Exponent,

            // Logical and bitwise operators
            TokenData::Bang => TokenKind::Bang,
            TokenData::BangEqual => TokenKind::BangEqual,
            TokenData::Equal => TokenKind::Equal,
            TokenData::DoubleEqual => TokenKind::DoubleEqual,
            TokenData::Greater => TokenKind::Greater,
            TokenData::GreaterEqual => TokenKind::GreaterEqual,
            TokenData::Less => TokenKind::Less,
            TokenData::LessEqual => TokenKind::LessEqual,
            TokenData::Ampersand => TokenKind::Ampersand,
            TokenData::DoubleAmpersand => TokenKind::DoubleAmpersand,
            TokenData::Pipe => TokenKind::Pipe,
            TokenData::DoublePipe => TokenKind::DoublePipe,

            // Literals
            TokenData::String(_) => TokenKind::String,
            TokenData::Float(_) => TokenKind::Float,
```

```
            TokenData::Integer(_) => TokenKind::Integer,
            TokenData::Boolean(_) => TokenKind::Boolean,

            // Control flow
            TokenData::If => TokenKind::If,
            TokenData::Else => TokenKind::Else,
            TokenData::While => TokenKind::While,
            TokenData::Return => TokenKind::Return,

            // Identifier related
            TokenData::Let => TokenKind::Let,
            TokenData::Fu => TokenKind::Fu,
            TokenData::Identifier(_) => TokenKind::Identifier,
        }
    }
}

/// A flag signalling the type of a token, without any additional data.
#[derive(Debug, PartialEq, Eq, Clone, Copy)]
pub enum TokenKind {
    /// The `(` character.
    LeftParenthesis,
    /// The `)` character.
    RightParenthesis,
    /// The `{` character.
    LeftBrace,
    /// The `}` character.
    RightBrace,
    /// The `,` character.
    Comma,
    /// The `.` character.
    Dot,
    /// The `;` character.
    Semicolon,
    /// The `?` character.
    QuestionMark,
    /// The `:` character.
    Colon,

    // Arithmetic operators
    /// The `+` character.
    Plus,
    /// The `-` character.
    Minus,
    /// The `*` character.
    Star,
    /// The `/` character.
    Slash,
    /// The `^` character.
    Exponent,

    // Logical and bitwise operators
    /// The `!` character.
    Bang,
    /// The `!=` string.
    BangEqual,
```

```
    /// The `=` character.
    Equal,
    /// The `==` string.
    DoubleEqual,
    /// The `>` character.
    Greater,
    /// The `>=` string.
    GreaterEqual,
    /// The `<` character.
    Less,
    /// The `<=` string.
    LessEqual,
    /// The `&` character.
    Ampersand,
    /// The `&&` string.
    DoubleAmpersand,
    /// The `|` character.
    Pipe,
    /// The `||` string.
    DoublePipe,

    // Literals
    /// String literals enclosed in `"`.
    String,
    /// Floating point numbers, denoted with a `.` separating the integer and
fractional parts.
    Float,
    /// Integers.
    Integer,
    /// Either `true` or `false`.
    Boolean,

    // Control flow
    /// The `if` string.
    If,
    /// The `else` string.
    Else,
    /// The `while` string.
    While,
    /// The `return` string.
    Return,

    // Identifier related
    /// The `let` string.
    Let,
    /// The `fu` string.
    Fu,
    /// All valid identifiers.
    ///
    /// Must start with either an alphabetic character or an underscore, with all
subsequent characters being alphanumeric or underscores.
    Identifier,
}

impl TokenKind {
    /// Attempts to cast itself to a [BinaryOperator], returning [None] if it does
```

```
not represent a binary operator.
    pub fn binary_operator(&self) -> Option<BinaryOperator> {
        Some(match self {
            Self::Plus => BinaryOperator::Add,
            Self::Minus => BinaryOperator::Subtract,
            Self::Star => BinaryOperator::Multiply,
            Self::Slash => BinaryOperator::Divide,

            Self::DoubleEqual => BinaryOperator::EqualTo,
            Self::BangEqual => BinaryOperator::NotEqualTo,
            Self::Greater => BinaryOperator::GreaterThan,
            Self::GreaterEqual => BinaryOperator::GreaterThanOrEqualTo,
            Self::Less => BinaryOperator::LessThan,
            Self::LessEqual => BinaryOperator::LessThanOrEqualTo,

            Self::Ampersand => BinaryOperator::BitwiseAND,
            Self::DoubleAmpersand => BinaryOperator::AND,
            Self::Pipe => BinaryOperator::BitwiseOR,
            Self::DoublePipe => BinaryOperator::OR,

            _ => return None,
        })
    }

    /// Attempts to cast itself to a [UnaryOperator], returning [None] if it does not
represent a unary operator.
    pub fn unary_operator(&self) -> Option<UnaryOperator> {
        Some(match self {
            Self::Minus => UnaryOperator::Minus,
            Self::Bang => UnaryOperator::NOT,

            _ => return None,
        })
    }
}
```

B.18 `src/value.rs`

```
use std::fmt::{Debug, Display};

use crate::{
    heap::{Object, Pointer},
    statement::Statement,
};

#[derive(Clone, PartialEq)]
pub enum NativeFunction {
    Print,
    Format,
    Input,
    Int,
    Float,
}

#[derive(Clone, PartialEq)]
pub enum Function {
    UserDefined {
```

```
        parameters: Vec<String>,
        block: Box<Statement>,
    },
    Native(NativeFunction),
}

#[derive(Clone, PartialEq)]
pub enum Value {
    String(String),
    Float(f64),
    Integer(i32),
    Boolean(bool),
    Function(Function),
    ObjectReference(Pointer),
    Object(Object),
}

impl Display for Value {
    fn fmt(&self, f: &mut std::fmt::Formatter<'_>) -> std::fmt::Result {
        match self {
            Self::String(value) => write!(f, "{}", value),
            Self::Float(value) => write!(f, "{}", value),
            Self::Integer(value) => write!(f, "{}", value),
            Self::Boolean(value) => write!(f, "{}", value),
            Self::Function(function) => match function {
                Function::Native(_) => write!(f, "<native function>"),
                Function::UserDefined {
                    parameters,
                    block: _,
                } => write!(f, "<function with {} named parameters>",
parameters.len()),
            },
            Self::Object(fields) => {
                write!(
                    f,
                    "{{ {} }}",
                    fields
                        .iter()
                        .map(|(identifier, _expression)| format!("{}", identifier))
                        .collect::<Vec<String>>()
                        .join(", ")
                )
            }
            Self::ObjectReference(_) => {
                write!(f, "<object reference>")
            }
        }
    }
}

impl Debug for Value {
    fn fmt(&self, f: &mut std::fmt::Formatter<'_>) -> std::fmt::Result {
        match self {
            Self::String(value) => write!(f, "\"{}\"", value),
            _ => write!(f, "{}", self),
        }?;
```

```
        write!(f, " (of type {})", self.slang_type())
    }
}

impl Value {
    pub fn slang_type(&self) -> Type {
        match self {
            Self::String(_) => Type::String,
            Self::Float(_) => Type::Float,
            Self::Integer(_) => Type::Integer,
            Self::Boolean(_) => Type::Boolean,
            Self::Function(_) => Type::Function,
            Self::Object(_) => Type::Object,
            Self::ObjectReference(_) => Type::Object,
        }
    }
}

#[derive(Debug)]
pub enum Type {
    String,
    Float,
    Integer,
    Boolean,
    Function,
    Object,
}

impl Display for Type {
    fn fmt(&self, f: &mut std::fmt::Formatter<'_>) -> std::fmt::Result {
        match self {
            Self::String => write!(f, "String"),
            Self::Float => write!(f, "Float"),
            Self::Integer => write!(f, "Integer"),
            Self::Boolean => write!(f, "Boolean"),
            Self::Function => write!(f, "Function"),
            Self::Object => write!(f, "Object"),
        }
    }
}
```

C Tests

C.1 t00.slang

```
// This is a line comment.

/*
This is a block comment. Anything in here will be ignored.

print("This will not be run.");
*/

if true {
    // Blocks are enclosed in braces.
}

// Parameter lists to subroutines are enclosed in parenthesis.
fu greet(firstName, lastName, age) {
    // Statements are terminated with semicolons.
    print(format("Hello ", firstName, " ", lastName, ". You are ", age, " years
old."));
}

// Arithmetic expressions are enclosed in infix notation.
let x = 1 + 2 * 3 - 4;
```

C.2 t00.out

C.3 t01.slang

```
let x = 0;
print(x);

x = "Hello";
print(x);

x = 0.0;
print(x);

x = print;
x(x);

x = {a: 1, b: 2, c: 3};
print(x);
```

C.4 t01.out

```
0
Hello
0
<native function>
<object reference>
```

C.5 t02.slang

```
fu test(x) {
    if x < 0 {
        return;
```

```
    } else if x < 10 {
        return x;
    } else {
        return "Very big.";
    }
}

print(test(0));
print(test(100));
print(test(-10));
```

C.6 t02.out

```
0
Very big.
[evaluation error] Attempted to use the return value from a function, however the
function returned nothing.
```

C.7 t03.slang

```
let x = true && "false";
```

C.8 t03.out

```
[evaluation error] The `&&` operator is not defined for Boolean and String.
```

C.9 t04.slang

```
// Addition
print(1 + 2);
print(1.2 + 3.4);
print("Hello, " + "world!");

// Subtraction
print(1 - 2);
print(1.2 - 3.4);

// Multiplication
print(3 * 4);
print(3.4 * 5.6);

// Division
print(10/3);
print(10.0/3.0);

// Exponent
print(2^8);
```

C.10 t04.out

```
3
4.6
Hello, world!
-1
-2.2
12
19.04
3
3.3333333333333335
256
```

C.11 t05.slang

```
print(1 / 0);
```

C.12 t05.out

```
[evaluation error] Division by zero.
```

C.13 t06.slang

```
print(-5);
print(-10.2);
print(-(-(10 / 4)));
```

C.14 t06.out

```
-5
-10.2
2
```

C.15 t07.slang

```
// Greater than
print(1 > 2);
print(2.0 > 1.0);
print("Hello" > "World");

// Less than
print(10 < 20);
print(1 * 10^6 < 1 * 10^5);
print("Computer" < "Science");

// Greater than or equal to
print(10 >= 20);
print(2.0 >= 2.0);
print("Hello" >= "World");

// Less than or equal to
print(-100 <= 10 * -10);
print(1 * 10^6 <= 1 * 10^2^2);
print("racecar" <= "racecar");
```

C.16 t07.out

```
false
true
false
true
false
true
false
true
false
true
false
true
```

C.17 t08.slang

```
// Equal to
print(1 == 1);
print(1 == 0);
```

```
print(1.0 == 1.0);
print(1.0 == 0.0);

print("Hello" == "Hello");
print("Hello" == "World");

let x = {label: "test"};
let y = x;
print(x == y);
print(x == {label: "test"});
print(x == {label: "test", label2: "another test"});

print(true == true);
print(true == false);

// Not equal to
print(1 != 1);
print(1 != 0);

print(1.0 != 1.0);
print(1.0 != 0.0);

print("Hello" != "Hello");
print("Hello" != "World");

let x = {label: "test"};
let y = x;
print(x != y);
print(x != {label: "test"});
print(x != {label: "test", label2: "another test"});

print(true != true);
print(true != false);
```

C.18 `t08.out`

```
true
false
true
false
true
false
true
true
false
true
false
false
true
false
true
false
true
false
false
true
```

```
false
true
```

C.19 t09.slang

```
// AND
print(false && false);
print(false && true);
print(true && false);
print(true && true);

// OR
print(false || false);
print(false || true);
print(true || false);
print(true || true);
```

C.20 t09.out

```
false
false
false
true
false
true
true
true
```

C.21 t10.slang

```
// Bitwise AND
print(false & false);
print(false & true);
print(true & false);
print(true & true);

print(1 & 1);
print(2^7 - 1 & 2 + 4);

// Bitwise OR
print(false | false);
print(false | true);
print(true | false);
print(true | true);

print(1 | 0);
print(2^7 - 1 | 2 + 4);
```

C.22 t10.out

```
false
false
false
true
1
6
false
true
true
true
```

```
1
127
```

C.23 t11.slang

```
print(!true);
print(!false);

print(!1);
print(!128);
```

C.24 t11.out

```
false
true
-2
-129
```

C.25 t12.slang

```
print(true ? 1 : 2);

print(true && false ? "Hello" : 0);
```

C.26 t12.out

```
1
0
```

C.27 t13.slang

```
print(true && (1 == 1) ? 1 + 2 * 3 - 4 : 3^3^(2 + -1));
```

C.28 t13.out

```
3
```

C.29 t14.slang

```
let letter = {raw: "a", position: 0, distanceFromEnd: 26 - 0};

print(letter);
```

C.30 t14.out

```
<object reference>
```

C.31 t15.slang

```
let person = {
    name: "Mr. Person",
    age: 100,
    address: "1 Big Road, City"
};

print(format(person.name, " lives at ", person.address, "."));
```

C.32 t15.out

```
Mr. Person lives at 1 Big Road, City.
```

C.33 t16.slang

```
let x;

print(x);
```

C.34 t16.out

```
[evaluation error] The target `x` has not been initialised.
```

C.35 t17.slang

```
let x = 10;

print(x);
```

C.36 t17.out

```
10
```

C.37 t18.slang

```
let x = 10;

{
    let x = 5;
    print(x);
}

print(x);
```

C.38 t18.out

```
5
10
```

C.39 t19.slang

```
let x = 10;

{
    x = 5;
    print(x);
}

print(x);
```

C.40 t19.out

```
5
5
```

C.41 t20.slang

```
if true {
    print("Condition was true.");
}
```

C.42 t20.out

```
Condition was true.
```

C.43 t21.slang

```
if false {
    print("Condition was true.");
} else {
    print("Condition was not true.");
}
```

C.44 t21.out

```
Condition was not true.
```

C.45 t22.slang

```
let x = 0;

while x < 10 {
    print(x);
    x = x + 1;
}
```

C.46 t22.out

```
0
1
2
3
4
5
6
7
8
9
```

C.47 t23.slang

```
fu greet(person) {
    let greeting = format("Hello, ", person.name, ", nice to meet you.");
    print(greeting);
}

let person = {name: "Tim Apple"};

greet(person);
```

C.48 t23.out

```
Hello, Tim Apple, nice to meet you.
```

C.49 t24.slang

```
fu add(a, b) {
    return a + b;
}

print(add(100, 1));

print(add(100));
```

C.50 t24.out

```
101
[evaluation error] Expected 2 arguments, but received 1.
```

C.51 t25.slang

```
fu fib(n) {
    if n <= 1 { return n; }
    else { return fib(n - 1) + fib(n - 2); }
}
```

```
print(fib(10));
```

C.52 t25.out

```
55
```

C.53 t26.slang

```
fu test(x) {
    if x <= 1{
        return;
    } else if x <= 10 {
        return "small";
    } else {
        return true;
    }
}

print(test(100));
print(test(5));
print(test(0));
```

C.54 t26.out

```
true
small
[evaluation error] Attempted to use the return value from a function, however the
function returned nothing.
```

C.55 t27.slang

```
// With 0 arguments
print();

// Printing strings
print("Hello, world!");

// Printing numbers
print(123456789);
print(0.1 + 0.2);

// Printing booleans
print(true);
```

C.56 t27.out

```
Hello, world!
123456789
0.30000000000000004
true
```

C.57 t28.slang

```
let person = {
    name: "John Smith",
    age: 30,
    address: "house road"
};

let info = format(person.name, " (age ", person.age, ") lives at ", person.address);
```

```
print(info);
```

C.58 t28.out

```
John Smith (age 30) lives at house road
```

C.59 t29.slang

```
let name = input("What is your name? ");

print(format("Nice to meet you, ", name));


let continue = true;

while continue {
    if input("Do you want to continue? (y/n) ") != "y" {
        continue = false;
    }
}

print("Done.");
```

C.60 t29.out

```
What is your name? Sam
Nice to meet you, Sam
Do you want to continue? (y/n) y
Do you want to continue? (y/n) y
Do you want to continue? (y/n) y
Do you want to continue? (y/n) n
Done.
```

C.61 t30.slang

```
let x = int("123456789");
print(x - 987654321);

let y = int(10.0);
print(y);

let z = int("ten");
print(z);
```

C.62 t30.out

```
-864197532
10
[evaluation error] Unable to cast from "ten" (of type String) to Integer.
```

C.63 t31.slang

```
let x = float("123456789");
print(x - 987654321.0);

let y = float("10.5");
print(y / 10.0);

let z = float("ten");
print(z);
```

C.64 t31.out

```
-864197532
1.05
[evaluation error] Unable to cast from "ten" (of type String) to Float.
```

C.65 t32.slang

```
print("Hello, world!);
```

C.66 t32.out

```
[line 1, column 7] Unterminated string.
```

C.67 t33.slang

```
// Do not run this file. Enter, one-by-one, the following lines of code into REPL
mode.

print("Hello, world);

print('Hello, world!');

let a = 5 + 3

let b = +5;

let 5 = 2;

print(5 ? 1 : 2);

print(1 + 2.0);

print(-"hello");

print(1 / 0);

print(name);

let c;
print(c);

if 1 { print("Condition was true"); }

"hello"(2);

fu greet(name) { print(format("Hello, ", name)); }
greet("Steve", "Jobs");

let d = print();

let e = "name".firstChar;

let f = {a: "A", b: "B"};
print(f.c);

let g = int("five");
```

C.68 `t33.out`

```
> print("Hello, world);
[line 1, column 7] Unterminated string.
>
> print('Hello, world!');
[line 1, column 7] Unexpected character: `'`
[line 1, column 21] Unexpected character: `'`
>
> let a = 5 + 3
[end of file] Expected one of the following tokens: [Semicolon]
>
> let b = +5;
[line 1, column 9] The unary `+` operator is not supported.
>
> let 5 = 2;
[line 1, column 5] Expected one of the following tokens: [Identifier]
>
> print(5 ? 1 : 2);
[evaluation error] Expected Boolean operand for ternary condition, found Integer.
>
> print(1 + 2.0);
[evaluation error] The `+` operator is not defined for Integer and Float.
>
> print(-"hello");
[evaluation error] The unary `-` operator is not defined for String.
>
> print(1/0);
[evaluation error] Division by zero.
>
> print(name);
[evaluation error] The identifier `name` is not defined.
>
> let c;
> print(c);
[evaluation error] The target `c` has not been initialised.
>
> if 1 { print("Condition was true"); }
[evaluation error] Expected Boolean if-statement condition, found Integer.
>
> "hello"(2);
[evaluation error] Attempted to 'call' a value of type String like a function.
>
> fu greet(name) { print(format("Hello, ", name)); }
> greet("Steve", "Jobs");
[evaluation error] Expected 1 arguments, but received 2.
>
> let d = print();
[evaluation error] Attempted to use the return value from a function, however the
function returned nothing.
>
> let e = "name".firstChar;
[evaluation error] Attempted to access a field of a value of type String, like an
object.
>
> let f = {a: "A", b: "B"};
> print(f.c);
```

```
[evaluation error] Attempted to access a non-existent field `c` on an object.
>
> let g = int("five");
[evaluation error] Unable to cast from "five" (of type String) to Integer.
>
```

C.69 t34.slang

```
fu layer1() {
    print(format("start of layer1 - stack frames count: ", STACK_FRAMES_COUNT));
    layer2();
    print(format("end of layer1 - stack frames count: ", STACK_FRAMES_COUNT));
}

fu layer2() {
    print(format("start of layer2 - stack frames count: ", STACK_FRAMES_COUNT));
    layer3();
    print(format("end of layer2 - stack frames count: ", STACK_FRAMES_COUNT));
}

fu layer3() {
    print(format("layer3 - stack frames count: ", STACK_FRAMES_COUNT));
}

print(format("start of outer layer - stack frames count: ", STACK_FRAMES_COUNT));
layer1();
print(format("end of outer layer - stack frames count: ", STACK_FRAMES_COUNT));
```

C.70 t34.out

```
start of outer layer - stack frames count: 1
start of layer1 - stack frames count: 2
start of layer2 - stack frames count: 3
layer3 - stack frames count: 4
end of layer2 - stack frames count: 3
end of layer1 - stack frames count: 2
end of outer layer - stack frames count: 1
```

C.71 t35.slang

```
// Must be run in naive allocation ("na") mode.

print(format("Current objects count: ", HEAP_OBJECTS_COUNT));

let x = {};
print("Object x created.");

print(format("Current objects count: ", HEAP_OBJECTS_COUNT));

let y = 10;
print("Integer x created.");

print(format("Current objects count: ", HEAP_OBJECTS_COUNT));
```

C.72 t35.out

```
Current objects count: 0
Object x created.
Current objects count: 1
```

```
Integer x created.
Current objects count: 1
```

C.73 t36.slang

```
// Must be run in naive allocation ("na") mode.

print(format("Current objects count: ", HEAP_OBJECTS_COUNT));

{
    print("Scope entered.");
    print(format("Current objects count: ", HEAP_OBJECTS_COUNT));
    let x = {};
    print("Object x created.");
    print(format("Current objects count: ", HEAP_OBJECTS_COUNT));
}

print("Scope exited.");
print(format("Current objects count: ", HEAP_OBJECTS_COUNT));

fu create_object() {
    print("New stack frame entered");
    let y = {};
    print("Object y created.");
    print(format("Current objects count: ", HEAP_OBJECTS_COUNT));
}

create_object();

print("Stack frame exited.");
print(format("Current objects count: ", HEAP_OBJECTS_COUNT));
```

C.74 t36.out

```
Current objects count: 0
Scope entered.
Current objects count: 0
Object x created.
Current objects count: 1
Scope exited.
Current objects count: 1
New stack frame entered
Object y created.
Current objects count: 2
Stack frame exited.
Current objects count: 2
```

C.75 t37.slang

```
// Must be run in reference counting ("rc") mode.

print(format("Current objects count: ", HEAP_OBJECTS_COUNT));

{
    print("Scope entered.");
    print(format("Current objects count: ", HEAP_OBJECTS_COUNT));
    let x = {};
    print("Object x created.");
    print(format("Current objects count: ", HEAP_OBJECTS_COUNT));
```

```
}

print("Scope exited.");
print(format("Current objects count: ", HEAP_OBJECTS_COUNT));

fu create_object() {
    print("New stack frame entered");
    let y = {};
    print("Object y created.");
    print(format("Current objects count: ", HEAP_OBJECTS_COUNT));
}

create_object();

print("Stack frame exited.");
print(format("Current objects count: ", HEAP_OBJECTS_COUNT));
```

C.76 `t37.out`

```
Current objects count: 0
Scope entered.
Current objects count: 0
Object x created.
Current objects count: 1
Scope exited.
Current objects count: 0
New stack frame entered
Object y created.
Current objects count: 1
Stack frame exited.
Current objects count: 0
```

C.77 `t38.slang`

```
// Must be run in reference counting ("rc") mode.

fu create_cycle(x) {
    let y = {other: x};

    print("Object y created.");
    print(format("Current objects count: ", HEAP_OBJECTS_COUNT));

    x.other = y;

    print("Cycle created.");
    print(format("Current objects count: ", HEAP_OBJECTS_COUNT));
}

{
    print(format("Current objects count: ", HEAP_OBJECTS_COUNT));

    let x = {};
    print("Object x created.");

    print(format("Current objects count: ", HEAP_OBJECTS_COUNT));

    create_cycle(x);
```

```
    print("Subroutine create_cycle exited.");
    print(format("Current objects count: ", HEAP_OBJECTS_COUNT));
}

print("Scope exited.");
print(format("Current objects count: ", HEAP_OBJECTS_COUNT));
```

C.78 `t38.out`

```
Current objects count: 0
Object x created.
Current objects count: 1
Object y created.
Current objects count: 2
Cycle created.
Current objects count: 2
Subroutine create_cycle exited.
Current objects count: 2
Scope exited.
Current objects count: 2
```

C.79 `t39.slang`

```
// Must be run in reference counting ("rc") mode.

fu create_object() {
    print(format("(a) Current objects count: ", HEAP_OBJECTS_COUNT));
    let y = {};
    print(format("(b) Current objects count: ", HEAP_OBJECTS_COUNT));

    return y;
}

{
    print(format("(1) Current objects count: ", HEAP_OBJECTS_COUNT));

    create_object();

    print(format("(2) Current objects count: ", HEAP_OBJECTS_COUNT));
}

print(format("(3) Current objects count: ", HEAP_OBJECTS_COUNT));

{
    print(format("(4) Current objects count: ", HEAP_OBJECTS_COUNT));

    let x = create_object();

    print(format("(5) Current objects count: ", HEAP_OBJECTS_COUNT));
}

print(format("(6) Current objects count: ", HEAP_OBJECTS_COUNT));
```

C.80 `t39.out`

```
(1) Current objects count: 0
(a) Current objects count: 0
(b) Current objects count: 1
(2) Current objects count: 1
```

```
(3) Current objects count: 0
(4) Current objects count: 0
(a) Current objects count: 0
(b) Current objects count: 1
(5) Current objects count: 1
(6) Current objects count: 0
```

C.81 t40.slang

```
// Must be run in reference counting ("rc") mode.

{
    print(format("Current objects count: ", HEAP_OBJECTS_COUNT));

    let x = {};
    print("Object x created.");
    print(format("Current objects count: ", HEAP_OBJECTS_COUNT));

    x = 5;
    print("x reassigned to 5.");

    print(format("Current objects count: ", HEAP_OBJECTS_COUNT));

    let y = {};
    print("Object y created.");
    print(format("Current objects count: ", HEAP_OBJECTS_COUNT));

    let y = 10;
    print("y redefined as 10.");

    print(format("Current objects count: ", HEAP_OBJECTS_COUNT));
}

print(format("Current objects count: ", HEAP_OBJECTS_COUNT));
```

C.82 t40.out

```
Current objects count: 0
Object x created.
Current objects count: 1
x reassigned to 5.
Current objects count: 0
Object y created.
Current objects count: 1
y redefined as 10.
Current objects count: 0
Current objects count: 0
```

C.83 t41.slang

```
// Must be run in garbage collection ("gc") mode.

print(format("Current objects count: ", HEAP_OBJECTS_COUNT));

{
    print("Scope entered.");
    print(format("Current objects count: ", HEAP_OBJECTS_COUNT));
    let x = {};
    print("Object x created.");
```

```
    print(format("Current objects count: ", HEAP_OBJECTS_COUNT));
}

print("Scope exited.");
print(format("Current objects count: ", HEAP_OBJECTS_COUNT));

fu create_object() {
    print("New stack frame entered");
    let y = {};
    print("Object y created.");
    print(format("Current objects count: ", HEAP_OBJECTS_COUNT));
}

create_object();

print("Stack frame exited.");
print(format("Current objects count: ", HEAP_OBJECTS_COUNT));
```

C.84 `t41.out`

```
Current objects count: 0
Scope entered.
Current objects count: 0
Object x created.
Current objects count: 1
Scope exited.
Current objects count: 0
New stack frame entered
Object y created.
Current objects count: 1
Stack frame exited.
Current objects count: 0
```

C.85 `t42.slang`

```
// Must be run in garbage collection ("gc") mode.

fu create_cycle(x) {
    let y = {other: x};

    print("Object y created.");
    print(format("Current objects count: ", HEAP_OBJECTS_COUNT));

    x.other = y;

    print("Cycle created.");
    print(format("Current objects count: ", HEAP_OBJECTS_COUNT));
}

{
    print(format("Current objects count: ", HEAP_OBJECTS_COUNT));

    let x = {};
    print("Object x created.");

    print(format("Current objects count: ", HEAP_OBJECTS_COUNT));

    create_cycle(x);
```

```
    print("Subroutine create_cycle exited.");
    print(format("Current objects count: ", HEAP_OBJECTS_COUNT));
}

print("Scope exited.");
print(format("Current objects count: ", HEAP_OBJECTS_COUNT));
```

C.86 `t42.out`

```
Object x created.
Current objects count: 1
Object y created.
Current objects count: 2
Cycle created.
Current objects count: 2
Subroutine create_cycle exited.
Current objects count: 2
Scope exited.
Current objects count: 0
```

C.87 `t43.slang`

```
// Must be run in garbage collection ("gc") mode.

fu create_object() {
    print(format("(a) Current objects count: ", HEAP_OBJECTS_COUNT));
    let y = {};
    print(format("(b) Current objects count: ", HEAP_OBJECTS_COUNT));

    return y;
}

{
    print(format("(1) Current objects count: ", HEAP_OBJECTS_COUNT));

    create_object();

    print(format("(2) Current objects count: ", HEAP_OBJECTS_COUNT));
}

print(format("(3) Current objects count: ", HEAP_OBJECTS_COUNT));

{
    print(format("(4) Current objects count: ", HEAP_OBJECTS_COUNT));

    let x = create_object();

    print(format("(5) Current objects count: ", HEAP_OBJECTS_COUNT));
}

print(format("(6) Current objects count: ", HEAP_OBJECTS_COUNT));
```

C.88 `t43.out`

```
(1) Current objects count: 0
(a) Current objects count: 0
(b) Current objects count: 1
(2) Current objects count: 1
```

```
(3) Current objects count: 0
(4) Current objects count: 0
(a) Current objects count: 0
(b) Current objects count: 1
(5) Current objects count: 1
(6) Current objects count: 0
```

C.89 t44.slang

```
// Must be run in garbage collection ("gc") mode.

{
    print(format("Current objects count: ", HEAP_OBJECTS_COUNT));

    let x = {};
    print("Object x created.");
    print(format("Current objects count: ", HEAP_OBJECTS_COUNT));

    x = 5;
    print("x reassigned to 5.");

    print(format("Current objects count: ", HEAP_OBJECTS_COUNT));

    let y = {};
    print("Object y created.");
    print(format("Current objects count: ", HEAP_OBJECTS_COUNT));

    let y = 10;
    print("y redefined as 10.");

    print(format("Current objects count: ", HEAP_OBJECTS_COUNT));
}

print(format("Current objects count: ", HEAP_OBJECTS_COUNT));
```

C.90 t44.out

```
Current objects count: 0
Object x created.
Current objects count: 1
x reassigned to 5.
Current objects count: 1
Object y created.
Current objects count: 2
y redefined as 10.
Current objects count: 2
Current objects count: 0
```

C.91 t45.slang

```
fu create_object() {
    let x = {};
    return x;
}

let i = 0;

while i < 100 {
    i = i + 1;
```

```
    let x = create_object();
}
```

C.92 `t52.slang`

```
print("(1)");

print(1 / 0);

print("(2)");
```

Bibliography

[1] A. V. Aho, J. D. Ullman, R. Sethi, and M. S. Lam, *Compilers: Principles, Techniques, and Tools, 2nd edition.*

[2] R. Nystrom, *Crafting Interpreters.* [Online]. Available: https://craftinginterpreters.com/

[3] S. Cutter, *slang.* [Online]. Available: https://github.com/sam-cutter/slang